DJANGO

CHEAT SHEETS

A Complete Reference for the Python Web Framework

URLs & Views · Class-Based Views · Templates & Partials · Models · Migrations · QuerySets & ORM · Forms · Admin · Auth · CSP & Security · Testing · Tasks & Async · DRF

68 REFERENCE SHEETS

CONTENTS

FOUNDATIONS

01–05	Introduction & Setup
06–08	URLs & Routing
09–15	Views
16–21	Templates
22–29	Models

DATA & USERS

30–31	Migrations
32–37	QuerySets & the ORM
38–43	Forms
44–46	Admin
47–50	Authentication & Authorization

PRODUCTION

51–54	Request Lifecycle & Services
55–57	Security
58–59	Testing
60–66	Advanced Topics
67–68	External Libraries & Deployment

DJ

Table of Contents

68 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 05

- 01 Introduction & Philoso…
- 02 Installation & Virtual E…
- 03 Projects vs Apps
- 04 settings.py Essentials
- 05 manage.py & Manage…

URLS & ROUTING 06 – 08

- 06 URL Patterns
- 07 Converters & Named U…
- 08 include() & Namespac…

VIEWS 09 – 15

- 09 Function-Based Views
- 10 The HttpRequest Obje…
- 11 Response Types
- 12 Class-Based Views — …
- 13 Generic Views — Displ…
- 14 Generic Views — Editing
- 15 View Mixins & Decorat…

TEMPLATES 16 – 21

- 16 Template Basics & Con…
- 17 Variables & Built-in Filt…
- 18 Built-in Tags & Control…
- 19 Inheritance, Includes &…
- 20 Static Files
- 21 Custom Tags & Filters

MODELS 22 – 29

- 22 Model Basics
- 23 Field Types
- 24 Field Options
- 25 ForeignKey Relationshi…
- 26 Many-to-Many & One…
- 27 Model Meta Options
- 28 Model Methods & Man…
- 29 Model Inheritance

MIGRATIONS 30 – 31

- 30 Migrations — Basics
- 31 Migrations — Data & A…

QUERYSETS & THE ORM 32 – 37

- 32 QuerySet Basics
- 33 Field Lookups
- 34 Q & F Expressions
- 35 Aggregation & Annotat…
- 36 Related Objects & Que…
- 37 Bulk Ops, Raw SQL & T…

FORMS 38 – 43

- 38 Form Basics
- 39 Form Fields & Widgets
- 40 Form Validation
- 41 ModelForm
- 42 Formsets
- 43 File & Image Uploads

ADMIN 44 – 46

- 44 Admin Basics
- 45 ModelAdmin Options
- 46 Inlines & Custom Actio…

AUTHENTICATION & AUTHORIZATION 47 – 50

- 47 Users & Authentication
- 48 Built-in Auth Views
- 49 Permissions & Groups
- 50 Custom User Model

REQUEST LIFECYCLE & SERVICES 51 – 54

- 51 Middleware
- 52 Sessions & Cookies
- 53 Messages Framework
- 54 Signals

SECURITY 55 – 57

- 55 CSRF, XSS & Clickjacki…
- 56 Content Security Policy
- 57 Security Settings & De…

TESTING 58 – 59

- 58 Testing Basics
- 59 Test Client & Fixtures

ADVANCED TOPICS 60 – 66

- 60 Pagination
- 61 Caching
- 62 Sending Email
- 63 Background Tasks
- 64 Async Django
- 65 Logging & Debugging
- 66 Internationalization & …

EXTERNAL LIBRARIES & DEPLOYMENT 67 – 68

- 67 Django REST Framework
- 68 Deployment

Introduction & Philosophy

WHAT DJANGO IS

Django is a **full-stack Python web framework**, first released in 2005 at the Lawrence Journal-World newspaper and now run by the Django Software Foundation. It ships with an ORM, templates, forms, an admin site, auth, sessions and security protection out of the box (**batteries included**). It follows the **MTV** pattern: **Models** hold data, **Templates** render HTML and **Views** connect the two. This set targets **Django 6.0 / 6.1** on Python 3.12+.

NOTES

- **DRY**: define a thing once (a model field drives the DB column, form field and admin widget)
- **Loose coupling**: URLs, views, templates and models are separate layers
- Used by Instagram, Mozilla, Pinterest, Disqus and the Washington Post
- Feature releases about every 8 months; every third one (x.2) is an **LTS** with 3 years of support
- Files are plain Python: `.py` for code, `.html` for templates

HELLO, WORLD IN ONE VIEW

```
# myapp/views.py
from django.http import HttpResponse

def hello(request):                          # every view takes a request
    return HttpResponse("Hello, World")      # ...and returns a response

# mysite/urls.py
from django.urls import path
from myapp.views import hello

urlpatterns = [
    path("", hello),                         # GET /  ->  Hello, World
]
```

CODE EXPLAINED

- → A **view** is a function (or class) that gets an `HttpRequest`
- → It must return an `HttpResponse` (or raise an exception)
- → `urlpatterns` maps URL paths to views
- → `runserver` is the dev server: auto-reloads on save, not for production
- → Real views usually call `render()` with a template instead of raw strings

RUN IT

```
python manage.py runserver        # http://127.0.0.1:8000/
```

REQUEST / RESPONSE CYCLE

STEP	WHAT HAPPENS
1. Request	Browser hits a URL; WSGI/ASGI server passes it in
2. Middleware	Request passes down the `MIDDLEWARE` list
3. URLconf	`urlpatterns` picks the view
4. View	Queries models, builds context
5. Template	Renders HTML from the context
6. Response	Back up through middleware to the client

MTV VS MVC

DJANGO	MVC NAME
Model	Model (data + rules)
Template	View (presentation)
View	Controller (logic)
URLconf	Router
Framework	Controller plumbing

RECENT VERSIONS

VERSION	NOTES
4.2 LTS	Apr 2023; support ended Apr 2026
5.2 LTS	Apr 2025; supported to Apr 2028
6.0	Dec 2025; partials, CSP, tasks
6.1	Aug 2026; fetch modes, `DB_CASCADE`
Python	6.x needs 3.12, 3.13 or 3.14

⚠ COMMON MISTAKES

- Using `runserver` in production: it is single-process and has no security hardening
- Calling the template layer the "view"; in Django the view is the Python function
- Pinning to a non-LTS release for a long-lived project, then falling out of support

✓ TIPS & BEST PRACTICES

- Pick the latest LTS (`5.2`) for stability, or the latest release (`6.1`) for new features
- Read the official tutorial (polls app) once; most docs assume you know it
- Run `python -m django --version` to check which Django a venv has

QUICK SUMMARY

- `MTV` : Model, Template, View
- `view(request)` : returns an `HttpResponse`
- `urlpatterns` : URL → view map
- `LTS` : every x.2 release, 3 years of fixes

Installation & Virtual Environments

INSTALLING DJANGO

Always install Django into a **virtual environment** so each project has its own Python packages and Django version. Use the standard-library `venv` module with `pip`, or the faster **uv** tool. Record dependencies in `requirements.txt` (pip) or `pyproject.toml` + `uv.lock` (uv) so the project can be rebuilt anywhere.

NOTES

- Django 6.x needs **Python 3.12+**; check with `python3 --version`
- Activate the venv before every `pip` or `manage.py` command
- Pin versions: `Django>=6.1,<6.2` avoids surprise upgrades
- Add `.venv/` to `.gitignore`; commit the requirements file instead
- Database drivers are separate: `psycopg[binary]` for PostgreSQL, `mysqlclient` for MySQL

VENV + PIP

```
python3 -m venv .venv                  # create the environment
source .venv/bin/activate              # macOS/Linux
.venv\Scripts\activate                 # Windows
python -m pip install --upgrade pip
pip install "Django>=6.1,<6.2"         # install Django
python -m django --version             # 6.1.1
pip freeze > requirements.txt          # save exact versions
pip install -r requirements.txt        # rebuild elsewhere
deactivate                             # leave the venv
```

UV (FAST ALTERNATIVE)

```
uv init mysite && cd mysite            # pyproject.toml + .venv
uv add django "psycopg[binary]"        # install + lock
uv run django-admin startproject config .
uv run python manage.py runserver      # no activate needed
uv sync                                # rebuild from uv.lock
```

CODE EXPLAINED

- → `python3 -m venv .venv` creates an isolated interpreter in `.venv/`
- → `source .venv/bin/activate` puts that interpreter first on `PATH`
- → Quote version specifiers so the shell doesn't treat `<` as a redirect
- → `pip freeze` lists every installed package with exact versions
- → `uv run` runs a command inside the project venv without activating it

CHECK THE INSTALL

COMMAND	OUTPUT
`django-admin version`	6.1.1
`pip show django`	Version, location
`which python`	…/.venv/bin/python
`pip list`	All packages

COMMON EXTRAS

PACKAGE	FOR
`psycopg[binary]`	PostgreSQL
`mysqlclient`	MySQL / MariaDB
`Pillow`	`ImageField`
`python-dotenv`	Load `.env` files
`gunicorn`	Production WSGI

DEPENDENCY FILES

FILE	TOOL
`requirements.txt`	pip; one pin per line
`pyproject.toml`	uv / Poetry / PEP 621
`uv.lock`	uv; exact resolved set
`.python-version`	pins the Python version

⚠ COMMON MISTAKES

- Installing Django globally with `sudo pip`, so projects fight over versions
- Running `manage.py` without activating the venv: `ModuleNotFoundError: django`
- Committing `.venv/` to git; it is machine-specific and huge

✓ TIPS & BEST PRACTICES

- Name the venv `.venv` so editors (VS Code, PyCharm) find it automatically
- Split `requirements/base.txt`, `dev.txt` and `prod.txt` for larger projects
- Use `uv` in CI: installs are many times faster than `pip`

QUICK SUMMARY

- `python3 -m venv .venv`: create
- `pip install django`: install
- `pip freeze > requirements.txt`: record
- `uv add django`: uv equivalent

Projects vs Apps

PROJECTS AND APPS

A **project** is the whole site: settings, root URLconf and WSGI/ASGI entry points. An **app** is a self-contained Python package that does one job (blog, accounts, payments) and can be reused across projects. A project contains many apps, and an app only becomes active once it is listed in `INSTALLED_APPS`.

NOTES

- `django-admin startproject config .` puts `manage.py` in the current folder
- Naming the project package `config` keeps it clearly separate from apps
- `startapp` creates the package but does **not** register it
- Each app has an `AppConfig` in `apps.py`; `ready()` runs at startup
- Templates and static files can live inside apps: `blog/templates/blog/`

CREATE AND REGISTER

```
django-admin startproject config .    # project package + manage.py
python manage.py startapp blog        # new app package
```

RESULTING LAYOUT

```
mysite/
├── manage.py               # command-line entry point
├── config/                 # the project package
│   ├── settings.py         # all configuration
│   ├── urls.py             # root URLconf
│   ├── asgi.py / wsgi.py   # server entry points
└── blog/                   # an app
    ├── apps.py             # AppConfig
    ├── models.py  views.py  admin.py  tests.py
    ├── migrations/         # schema history
    └── urls.py             # add yourself

# config/settings.py
INSTALLED_APPS = [
    "django.contrib.admin", "django.contrib.auth",  # ...4 more
    "blog",                         # or "blog.apps.BlogConfig"
]
```

CODE EXPLAINED

- → The trailing `.` stops Django nesting `mysite/mysite/`
- → `apps.py` holds `BlogConfig(AppConfig)` with `name = "blog"`
- → Listing `"blog"` finds the default `AppConfig` automatically
- → Apps are found by dotted Python path, so `blog` must be importable
- → Django creates `urls.py` for the project but not for apps

APP FILES

FILE	HOLDS
`models.py`	Database models
`views.py`	Request handlers
`admin.py`	Admin registrations
`apps.py`	`AppConfig`
`tests.py`	Tests (or `tests/`)
`forms.py`	Forms (add yourself)

APPCONFIG ATTRIBUTES

ATTRIBUTE	PURPOSE
`name`	Dotted path, e.g. `"blog"`
`label`	Short unique label
`verbose_name`	Name shown in admin
`default_auto_field`	PK type for the app
`ready()`	Startup hook (signals)

DEFAULT CONTRIB APPS

APP	PROVIDES
`admin`	Admin site
`auth`	Users, permissions
`contenttypes`	Generic model refs
`sessions`	Session storage
`messages`	Flash messages
`staticfiles`	CSS/JS handling

⚠ COMMON MISTAKES

- Forgetting to add the app to `INSTALLED_APPS`, so models and templates are ignored
- Giving an app the same name as a Python module (`email`, `test`), which shadows imports
- One giant app for everything; split by domain so pieces stay reusable

✓ TIPS & BEST PRACTICES

- Use `startproject config .` for a flat, readable layout
- Namespace app templates: `blog/templates/blog/post_list.html`
- Move to an `apps/` folder only when you have many apps

QUICK SUMMARY

- `startproject`: the site + settings
- `startapp`: one feature package
- `INSTALLED_APPS`: activates apps
- `AppConfig.ready()`: startup hook

settings.py Essentials

THE SETTINGS MODULE

All configuration lives in one Python module, `config/settings.py`. Django finds it through the `DJANGO_SETTINGS_MODULE` environment variable, which `manage.py`, `wsgi.py` and `asgi.py` set for you. Settings are plain UPPERCASE module variables; read them anywhere with `from django.conf import settings`.

NOTES

- `DEBUG = True` shows detailed error pages; **never** in production
- `SECRET_KEY` signs sessions, cookies and tokens; keep it out of git
- `ALLOWED_HOSTS` must list your domains when `DEBUG = False`
- `BASE_DIR` is the project root as a `pathlib.Path`
- 6.0: `DEFAULT_AUTO_FIELD` now defaults to `BigAutoField`

TYPICAL SETTINGS.PY

```
import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]        # from env
DEBUG = os.environ.get("DJANGO_DEBUG", "0") == "1"
ALLOWED_HOSTS = ["example.com", "www.example.com"]

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "mysite", "USER": "mysite",
        "PASSWORD": os.environ["DB_PASSWORD"],
        "HOST": "localhost", "PORT": "5432",
    }
}
LANGUAGE_CODE = "en-us"
TIME_ZONE = "America/Chicago"
USE_TZ = True                                       # store UTC

# anywhere else
from django.conf import settings
settings.DEBUG                                      # False
```

CODE EXPLAINED

- → `Path(__file__).resolve().parent.parent` → folder holding `manage.py`
- → `os.environ[...]` raises `KeyError` if the variable is missing (fail fast)
- → `DATABASES["default"]` is the connection used unless you pick another
- → `USE_TZ = True` stores aware datetimes in UTC; `TIME_ZONE` is for display
- → Import `django.conf.settings`, never `config.settings` directly

CORE SETTINGS

SETTING	PURPOSE
`INSTALLED_APPS`	Active apps
`MIDDLEWARE`	Request/response hooks
`ROOT_URLCONF`	`"config.urls"`
`TEMPLATES`	Template engines
`STATIC_URL`	`"static/"`
`DEFAULT_AUTO_FIELD`	PK type

DATABASE ENGINE VALUES

ENGINE	DATABASE
`...sqlite3`	SQLite (default)
`...postgresql`	PostgreSQL 14+
`...mysql`	MySQL 8.0.11+
`...oracle`	Oracle (oracledb)
`django.db.backends.`	full prefix

SELECTING SETTINGS

WAY	EXAMPLE
`Env var`	`DJANGO_SETTINGS_MODULE`
`Flag`	`--settings=config.dev`
`Split files`	`base.py` / `dev.py` / `prod.py`
`Show diff`	`manage.py diffsettings`

⚠ COMMON MISTAKES

- Committing `SECRET_KEY` or DB passwords; anyone with the repo can forge sessions
- `DEBUG = False` with an empty `ALLOWED_HOSTS`: every request returns 400
- Importing `config.settings` directly, which skips overrides from `--settings`

✓ TIPS & BEST PRACTICES

- Load secrets from environment variables or a `.env` file via `python-dotenv`
- Use `manage.py diffsettings` to see only what you changed from defaults
- Keep `USE_TZ = True`; convert to local time only when displaying

QUICK SUMMARY

- `DJANGO_SETTINGS_MODULE`: which settings file
- `DEBUG`: dev error pages, off in prod
- `DATABASES`: connection config
- `from django.conf import settings`: read values

manage.py & Management Commands

runserver · shell · check · createsuperuser · showmigrations · custom BaseCommand · handle()

RUNNING COMMANDS

`manage.py` is a thin wrapper around `django-admin` that sets `DJANGO_SETTINGS_MODULE` for your project. Every built-in task (dev server, migrations, shell, tests) is a **management command**, and you can add your own by placing a module in `<app>/management/commands/`.

NOTES

- `python manage.py help` lists every command; `help <cmd>` shows its options
- `shell` auto-imports all models (Django 5.2+)
- `check` runs system checks; add `--deploy` before going live
- Custom commands are great for cron jobs and data imports
- Call commands from code with `call_command("migrate")`

EVERYDAY COMMANDS

```
python manage.py runserver 0.0.0.0:8000   # dev server on all interfaces
python manage.py shell                    # Python REPL with Django loaded
python manage.py createsuperuser          # admin login
python manage.py makemigrations && python manage.py migrate
python manage.py check --deploy           # production checklist
python manage.py import_books books.csv --dry-run
```

CUSTOM COMMAND

```
# blog/management/commands/import_books.py
from django.core.management.base import BaseCommand, CommandError

class Command(BaseCommand):
    help = "Import books from a CSV file"

    def add_arguments(self, parser):          # argparse parser
        parser.add_argument("path")
        parser.add_argument("--dry-run", action="store_true")

    def handle(self, *args, **opts):
        if not opts["path"].endswith(".csv"):
            raise CommandError("Expected a .csv file")
        count = import_csv(opts["path"], save=not opts["dry_run"])
        self.stdout.write(self.style.SUCCESS(f"Imported {count}"))
```

CODE EXPLAINED

- → The file name becomes the command name: `import_books`
- → `management/` and `commands/` both need `__init__.py`
- → `add_arguments` gets an `argparse` parser; `--dry-run` → `opts["dry_run"]`
- → `CommandError` prints the message and exits with status 1
- → Write with `self.stdout.write` so tests can capture output

BUILT-IN COMMANDS

COMMAND	DOES
`runserver`	Dev server, auto-reload
`shell / dbshell`	Python / SQL prompt
`startapp`	New app skeleton
`showmigrations`	Applied migrations
`collectstatic`	Gather static files
`test`	Run the test suite

DATA COMMANDS

COMMAND	DOES
`dumpdata app`	Export JSON fixture
`loaddata file`	Import fixture
`flush`	Empty all tables
`changepassword`	Reset a password
`clearsessions`	Delete expired sessions

SELF.STYLE HELPERS

STYLE	USE
`SUCCESS`	Green confirmation
`WARNING`	Yellow warning
`ERROR`	Red error
`NOTICE`	Red notice
`call_command("x")`	Run from code

⚠ COMMON MISTAKES

- Using `print()` in commands; tests can't capture it, use `self.stdout.write`
- Missing `__init__.py` in `management/commands/`, so the command is not found
- Running `flush` on the wrong database; it deletes every row without a backup

✓ TIPS & BEST PRACTICES

- Use `runserver_plus` (django-extensions) for an in-browser debugger
- Wrap bulk imports in `transaction.atomic()` so a failure rolls everything back
- Schedule custom commands with cron or a task runner instead of web endpoints

QUICK SUMMARY

- `manage.py <cmd>`: run any command
- `BaseCommand.handle()`: command body
- `add_arguments(parser)`: CLI options
- `call_command()`: invoke from Python

URL Patterns

HOW URLS REACH VIEWS

Django reads `ROOT_URLCONF` (usually `config.urls`) and walks its `urlpatterns` list **top to bottom**, stopping at the first pattern that matches the request path. `path()` uses simple angle-bracket captures; `re_path()` takes a regular expression. The leading slash and the domain are never part of the match: `/blog/2026/` is matched as `blog/2026/`.

NOTES

- Patterns don't include the leading `/` or the query string
- First match wins, so put specific routes above catch-alls
- `APPEND_SLASH = True` (default) redirects `/about` → `/about/` when only the slashed version exists
- No match → `Http404` (debug page lists the patterns tried)
- Captured values arrive as keyword arguments to the view

CONFIG/URLS.PY

```python
from django.contrib import admin
from django.urls import path, re_path
from blog import views

urlpatterns = [
    path("", views.home),                          # /
    path("about/", views.about),                   # /about/
    path("posts/<int:pk>/", views.post_detail),    # /posts/7/ -> pk=7
    path("posts/<slug:slug>/", views.post_by_slug),
    re_path(r"^archive/(?P<year>[0-9]{4})/$", views.archive),
    path("admin/", admin.site.urls),
]

# blog/views.py
def post_detail(request, pk):                      # pk is an int
    ...
def archive(request, year):                        # year is a str
    ...
```

CODE EXPLAINED

- → `path(route, view)` → the route string is matched against the path
- → `<int:pk>` converts the segment and passes `pk=7`
- → `re_path` named groups `(?P<year>...)` become kwargs, always as `str`
- → `/posts/7/` hits `<int:pk>` first; `/posts/hello/` falls to `<slug>`
- → `admin.site.urls` is itself a list of patterns

PATH() ARGUMENTS

ARGUMENT	MEANING
`route`	Pattern string
`view`	Function, `.as_view()` or `include()`
`kwargs`	Extra dict passed to the view
`name`	Name for `reverse()` (Sheet 07)

MATCHING EXAMPLES

REQUEST	PATTERN HIT
`/`	`""`
`/about/`	`"about/"`
`/about`	301 → `/about/`
`/posts/7/?x=1`	`<int:pk>`, pk=7
`/archive/26/`	404 (needs 4 digits)

RELATED SETTINGS

SETTING	DEFAULT
`ROOT_URLCONF`	`"config.urls"`
`APPEND_SLASH`	`True`
`PREPEND_WWW`	`False`
`request.urlconf`	Per-request override

⚠ COMMON MISTAKES

- Starting a route with `/` (`"/about/"`); it never matches because the slash is stripped
- A catch-all like `<slug:slug>/` above `about/`, which swallows the specific route
- Expecting `re_path` captures to be `int`; they are always strings

✓ TIPS & BEST PRACTICES

- Prefer `path()` and converters; reach for `re_path()` only for real regex needs
- Always end routes with `/` and keep `APPEND_SLASH` on for consistent URLs
- Pass `kwargs={"template": ...}` to reuse one view on several URLs

QUICK SUMMARY

- `urlpatterns`: ordered list, first match wins
- `path("x/<int:pk>/", view)`: typed capture
- `re_path(r"^...$", view)`: regex capture
- `APPEND_SLASH`: adds the missing slash

Converters & Named URLs

CONVERTERS AND REVERSING

A **path converter** validates and converts a captured segment, so `<int:pk>` only matches digits and hands the view an `int`. Giving a pattern a `name=` lets you build its URL from code with `reverse()` or from templates with `{% url %}`, so changing a route never breaks links. Use `reverse_lazy()` where URLs are needed at import time.

NOTES

- Converters: `str` (default), `int`, `slug`, `uuid`, `path`
- `str` matches any non-empty text without `/`; `path` includes `/`
- Custom converters need `regex`, `to_python()` and `to_url()`
- `reverse()` raises `NoReverseMatch` if the name or args are wrong
- `redirect("post-detail", pk=7)` reverses for you

NAMING AND REVERSING

```python
# urls.py
from django.urls import path, register_converter
from . import views, converters

register_converter(converters.YearConverter, "yyyy")
urlpatterns = [
    path("posts/<int:pk>/", views.detail, name="post-detail"),
    path("archive/<yyyy:year>/", views.archive, name="archive"),
]

# converters.py
class YearConverter:
    regex = r"[0-9]{4}"
    def to_python(self, value): return int(value)
    def to_url(self, value): return f"{value:04d}"

from django.urls import reverse, reverse_lazy
reverse("post-detail", args=[7])          # '/posts/7/'
reverse("post-detail", kwargs={"pk": 7})  # '/posts/7/'
success_url = reverse_lazy("post-list")   # safe at class level
```

CODE EXPLAINED

- → `register_converter` adds `<yyyy:...>` to the pattern syntax
- → `to_python` feeds the view; `to_url` is used when reversing
- → Pass `args` **or** `kwargs` to `reverse`, never both
- → `reverse_lazy` delays lookup until the string is used
- → `{% url ... as var %}` stores the URL instead of printing it

IN TEMPLATES

```
<a href="{% url 'post-detail' post.pk %}">{{ post.title }}</a>
{% url 'archive' year=2026 as archive_url %}
```

BUILT-IN CONVERTERS

CONVERTER	MATCHES
`str`	Text, no /
`int`	0 or positive int
`slug`	my-post_1
`uuid`	Formatted UUID
`path`	Text incl. /

REVERSE HELPERS

HELPER	WHERE
`reverse()`	Views, functions
`reverse_lazy()`	Class attrs, settings
`{% url %}`	Templates
`redirect(name)`	Views (302)
`get_absolute_url()`	Models

COMMON ERRORS

ERROR	CAUSE
`NoReverseMatch`	Bad name or args
`ImproperlyConfigured`	`reverse` at import
`404`	Converter didn't match
`TypeError`	View missing kwarg

⚠ COMMON MISTAKES

- Hard-coding `/posts/7/` in templates; one route change breaks every link
- Calling `reverse()` in a class attribute; URLconf isn't loaded yet, use `reverse_lazy()`
- Using `<str:path>` for file paths; it stops at `/`, use the `path` converter

✓ TIPS & BEST PRACTICES

- Name every route; use `app:name` with namespaces (Sheet 08)
- Use `<uuid:...>` for public IDs so URLs don't reveal row counts
- Add `get_absolute_url()` on models and call it from templates

QUICK SUMMARY

- `<int:pk>`: validate + convert
- `name="..."`: reversible route
- `reverse()` / `{% url %}`: build URLs
- `reverse_lazy()`: import-time safe

include() & Namespaces

SPLITTING THE URLCONF

`include()` hands the rest of a path to another URLconf, so each app keeps its own `urls.py`. Setting `app_name` in that file creates an **application namespace**, and names are then reversed as `"blog:detail"`, which stops two apps' `detail` routes from clashing. Custom error views are set with `handler404` and friends in the root URLconf.

NOTES

- The prefix matched so far is stripped before the included patterns run
- `include()` also accepts a list of `path()` objects
- `namespace=` on `include()` creates an **instance** namespace
- Error handlers only work in the root URLconf with `DEBUG = False`
- `request.resolver_match` shows which pattern matched

ROOT AND APP URLCONFS

```python
# config/urls.py
from django.urls import include, path

urlpatterns = [
    path("blog/", include("blog.urls")),            # /blog/...
    path("api/v1/", include(("api.urls", "api"), namespace="v1")),
    path("shop/", include([
        path("", shop_home, name="shop-home"),
        path("cart/", cart, name="cart"),
    ])),
]
handler404 = "config.views.not_found"
handler500 = "config.views.server_error"

# blog/urls.py
from django.urls import path
from . import views

app_name = "blog"                                   # namespace
urlpatterns = [
    path("", views.post_list, name="list"),         # /blog/
    path("<int:pk>/", views.post_detail, name="detail"),
]

reverse("blog:detail", args=[3])                    # '/blog/3/'
```

CODE EXPLAINED

- → `include("blog.urls")` takes a dotted module path
- → A `(module, app_name)` tuple sets the namespace without editing the app
- → `app_name = "blog"` makes names reversible as `blog:...`
- → `handler404` takes a dotted path to a `view(request, exception)`
- → `handler500` views take only `request`

TEMPLATE

```django
<a href="{% url 'blog:detail' post.pk %}">Read more</a>
```

ERROR HANDLERS

VARIABLE	SIGNATURE
handler400	(request, exception)
handler403	(request, exception)
handler404	(request, exception)
handler500	(request)

NAMESPACE TYPES

TYPE	SET BY
Application	app_name in app urls
Instance	include(..., namespace=)
Nested	"api:v1:list"
Current	resolver_match.namespace

DEFAULT ERROR TEMPLATES

TEMPLATE	SHOWN FOR
404.html	Page not found
500.html	Server error (no context)
403.html	Permission denied
400.html	Bad request

⚠ COMMON MISTAKES

- Forgetting the trailing slash in `path("blog", include(...))`, so `/blog/1/` never matches
- Reversing `"detail"` when the app has `app_name`; you must use `"blog:detail"`
- Testing a custom 404 with `DEBUG = True`; Django shows its debug page instead

✓ TIPS & BEST PRACTICES

- Give every app its own `urls.py` with `app_name`
- Just add `templates/404.html`; you rarely need a custom `handler404`
- Keep `500.html` static: it renders with no context and no DB access

QUICK SUMMARY

- `include("app.urls")`: delegate a prefix
- `app_name`: namespace for `app:name`
- `namespace=`: instance namespace
- `handler404`: custom error view

Function-Based Views

VIEWS AS FUNCTIONS

A **function-based view** (FBV) is a plain function that takes an `HttpRequest` plus any URL captures and returns an `HttpResponse`. The `django.shortcuts` helpers cover the common cases: `render()` fills a template, `redirect()` sends the browser elsewhere, and `get_object_or_404()` fetches a row or raises a 404. Decorators restrict methods or require login.

NOTES

- Branch on `request.method` to handle GET and POST in one view
- After a successful POST, always **redirect** (Post/Redirect/Get)
- `render()` returns an `HttpResponse` with status 200 by default
- Raising `Http404` anywhere in a view returns the 404 page
- FBVs are explicit and easy to read; CBVs reduce repetition (Sheet 12)

LIST, DETAIL AND CREATE

```python
from django.shortcuts import render, redirect, get_object_or_404
from django.views.decorators.http import require_http_methods, require_GET
from .models import Post
from .forms import PostForm

@require_GET
def post_list(request):
    posts = Post.objects.filter(published=True)
    return render(request, "blog/post_list.html", {"posts": posts})

def post_detail(request, pk):
    post = get_object_or_404(Post, pk=pk, published=True)
    return render(request, "blog/post_detail.html", {"post": post})

@require_http_methods(["GET", "POST"])
def post_create(request):
    form = PostForm(request.POST or None)
    if request.method == "POST" and form.is_valid():
        post = form.save()
        return redirect(post)                    # uses get_absolute_url()
    return render(request, "blog/post_form.html", {"form": form})
```

CODE EXPLAINED

→ `render(request, template, context)` → `HttpResponse` with HTML

→ `get_object_or_404(Model, **lookup)` → object or `Http404`

→ `PostForm(request.POST or None)` is unbound on GET, bound on POST

→ `redirect(obj)` calls `obj.get_absolute_url()`

→ Wrong method → `HttpResponseNotAllowed` (405)

DJANGO.SHORTCUTS

FUNCTION	RETURNS
`render()`	`HttpResponse` (HTML)
`redirect()`	302 redirect
`get_object_or_404()`	Object or 404
`get_list_or_404()`	List or 404 if empty
`aget_object_or_404()`	Async version

REDIRECT() TARGETS

ARGUMENT	GOES TO
`obj`	its `get_absolute_url()`
`"blog:list"`	Reversed name
`"/login/"`	Literal URL
`permanent=True`	301 instead of 302

METHOD DECORATORS

DECORATOR	ALLOWS
`@require_GET`	GET only
`@require_POST`	POST only
`@require_safe`	GET, HEAD
`@require_http_methods`	Listed methods
`@login_required`	Logged-in users

⚠ COMMON MISTAKES

- Returning `render()` after a successful POST; refreshing resubmits the form
- `Post.objects.get(pk=pk)` without handling `DoesNotExist`, which gives a 500
- Forgetting `return` before `render()`; the view returns `None` and errors

✓ TIPS & BEST PRACTICES

- Filter in `get_object_or_404` too: `pk=pk, author=request.user` blocks other users
- Use `require_POST` on views that change data (delete, like, publish)
- Keep views thin; move business logic into model methods or services

QUICK SUMMARY

- `def view(request, **kw)`: returns a response
- `render()`: template → HTML
- `redirect()`: after POST
- `get_object_or_404()`: fetch or 404

The HttpRequest Object

WHAT A VIEW RECEIVES

Every view's first argument is an `HttpRequest`. It carries the HTTP method, URL, query string (`GET`), form data (`POST`), uploads (`FILES`), cookies, headers and anything middleware attached, such as `request.user` and `request.session`. `GET` and `POST` are **QueryDicts**: read-only, dict-like and able to hold several values per key.

NOTES

- `request.POST` only has form-encoded data; JSON bodies are in `request.body`
- `request.GET.get("q", "")` avoids `KeyError` on missing params
- `getlist("tag")` returns every value for repeated keys
- `request.headers` is case-insensitive: `headers["User-Agent"]`
- `request.user` is `AnonymousUser` when nobody is logged in

READING REQUEST DATA

```python
import json

def search(request):                    # GET /search/?q=django&tag=web&tag=py
    request.method                      # 'GET'
    request.path                        # '/search/'
    request.get_full_path()             # '/search/?q=django&tag=web&tag=py'
    request.GET.get("q", "")            # 'django'
    request.GET["tag"]                  # 'py'  (last value)
    request.GET.getlist("tag")          # ['web', 'py']
    request.GET.urlencode()             # 'q=django&tag=web&tag=py'
    request.headers["Accept-Language"]   # 'en-US,en;q=0.9'
    request.COOKIES.get("theme")        # 'dark' or None
    request.user.is_authenticated       # True / False
    request.build_absolute_uri()        # 'https://example.com/search/?q=...'
    request.is_secure()                 # True over HTTPS

def api_create(request):                # POST with a JSON body
    data = json.loads(request.body)  # bytes -> dict
    ip = request.META.get("REMOTE_ADDR")
```

CODE EXPLAINED

- → `request.GET[key]` returns the **last** value; `getlist` returns all
- → `get_full_path()` includes the query string; `path` does not
- → `request.body` is raw `bytes`; read it before touching `POST` for JSON
- → `META` holds CGI-style keys like `REMOTE_ADDR`, `HTTP_HOST`
- → Make a mutable copy with `request.GET.copy()`

ATTRIBUTES

ATTRIBUTE	HOLDS
`method`	"GET", "POST"…
`GET / POST`	QueryDict
`FILES`	Uploaded files
`COOKIES`	dict of str
`session`	Session dict
`resolver_match`	Matched URL info

METHODS

METHOD	RETURNS
`get_host()`	`"example.com"`
`get_port()`	`"443"`
`build_absolute_uri()`	Full URL
`is_secure()`	HTTPS?
`get_signed_cookie()`	Verified value
`accepts("text/html")`	Bool

QUERYDICT

CALL	RESULT
`qd.get(k, d)`	Last value or default
`qd.getlist(k)`	All values
`qd.copy()`	Mutable copy
`qd.dict()`	Plain dict (last values)
`qd.lists()`	`(key, [values])` pairs

⚠ COMMON MISTAKES

- Expecting JSON in `request.POST`; it's empty for `application/json`, parse `request.body`
- Using `request.GET["q"]` without a default; a missing param raises `MultiValueDictKeyError`
- Trusting `REMOTE_ADDR` behind a proxy; it is the proxy's IP, not the client's

✓ TIPS & BEST PRACTICES

- Validate query params with a `Form` instead of manual parsing
- Use `request.headers` rather than `META["HTTP_..."]` names
- `request.accepts("application/json")` lets one view serve HTML and JSON

QUICK SUMMARY

- `request.GET` / `POST`: QueryDicts
- `request.body`: raw bytes (JSON)
- `request.user`: current user
- `build_absolute_uri()`: full URL

Response Types

RETURNING RESPONSES

Every view returns an `HttpResponse` or a subclass. The base class sends a string or bytes with a content type and status code; subclasses cover JSON, file downloads, streamed output and redirects. Headers are set like dictionary items, and cookies with `set_cookie()`. Raising `Http404` or `PermissionDenied` returns the matching error page instead.

NOTES

- Default `content_type` is `text/html; charset=utf-8`
- `JsonResponse` needs a `dict` unless you pass `safe=False`
- `FileResponse` streams efficiently and sets `Content-Length`
- `StreamingHttpResponse` has no `.content`; it iterates a generator
- Use `HTTPStatus.CREATED` instead of magic numbers like `201`

RESPONSE EXAMPLES

```python
from http import HTTPStatus
from django.http import (HttpResponse, JsonResponse, FileResponse,
    StreamingHttpResponse, HttpResponseRedirect, Http404)
from django.core.exceptions import PermissionDenied

HttpResponse("<h1>Hi</h1>")                          # 200 text/html
HttpResponse("ok", content_type="text/plain")
HttpResponse(status=HTTPStatus.NO_CONTENT)           # 204
JsonResponse({"id": 1, "ok": True})                  # application/json
JsonResponse([1, 2, 3], safe=False)                  # list needs safe=False
JsonResponse({"error": "bad"}, status=400)
FileResponse(open("report.pdf", "rb"), as_attachment=True,
             filename="report.pdf")                  # download
StreamingHttpResponse((f"{row}\n" for row in rows),
                      content_type="text/csv")
HttpResponseRedirect("/login/")                      # 302

resp = HttpResponse("Hi")
resp["Cache-Control"] = "no-store"                   # set a header
resp.set_cookie("theme", "dark", max_age=3600)
raise Http404("Post not found")                      # 404 page
raise PermissionDenied                               # 403 page
```

CODE EXPLAINED

- → `status=` takes an int or `HTTPStatus` member
- → `safe=False` allows non-dict top-level JSON
- → `FileResponse` closes the file for you when done
- → Generators keep memory flat for big CSV exports
- → `resp["Header"] = value` sets any response header

RESPONSE CLASSES

CLASS	STATUS
`HttpResponse`	200
`HttpResponseRedirect`	302
`HttpResponsePermanentRedirect`	301
`HttpResponseNotModified`	304
`HttpResponseBadRequest`	400
`HttpResponseNotAllowed`	405

RESPONSE ATTRIBUTES

ATTRIBUTE	MEANING
`content`	Body bytes
`status_code`	Int code
`headers`	Case-insensitive dict
`charset`	`"utf-8"`
`streaming`	Is it streamed?

EXCEPTIONS → PAGES

RAISE	RESULT
`Http404`	404 page
`PermissionDenied`	403 page
`BadRequest`	400 page
`SuspiciousOperation`	400 + log
`Anything else`	500 page

⚠ COMMON MISTAKES

- Passing a list to `JsonResponse` without `safe=False`: `TypeError`
- Building JSON with `HttpResponse(json.dumps(...))` and forgetting `content_type`
- Opening files with `open(path)` in text mode for `FileResponse`; use `"rb"`

✓ TIPS & BEST PRACTICES

- Use `HttpResponse(status=204)` for successful actions with no body
- Serve user uploads through `FileResponse` only after a permission check
- For big exports use `StreamingHttpResponse` so the worker doesn't buffer everything

QUICK SUMMARY

- `HttpResponse(body, status=)`: base response
- `JsonResponse(dict)`: JSON
- `FileResponse(f, as_attachment=True)`: download
- `raise Http404`: error page

Class-Based Views — Basics

VIEWS AS CLASSES

A **class-based view** (CBV) maps each HTTP method to a method on a class: `get()`, `post()`, `put()` and so on. `as_view()` turns the class into a callable for the URLconf; on each request it creates a **new instance**, stores `request`, `args` and `kwargs` on `self`, then `dispatch()` calls the method that matches `request.method`. Unhandled methods return 405.

NOTES

- Class attributes are defaults; pass overrides to `as_view(template_name=...)`
- Each request gets a fresh instance, so `self` state is per-request
- URL captures are in `self.kwargs["pk"]`
- `setup()` runs before `dispatch()`; override it to set shared attributes
- Generic CBVs (Sheets 13–14) build on these base classes

VIEW, TEMPLATEVIEW, REDIRECTVIEW

```python
from django.http import HttpResponse, JsonResponse
from django.views import View
from django.views.generic import TemplateView, RedirectView

class PingView(View):
    http_method_names = ["get", "post"]
    def get(self, request, *args, **kwargs):
        return JsonResponse({"pong": True})
    def post(self, request, *args, **kwargs):
        return HttpResponse(status=204)

class AboutView(TemplateView):
    template_name = "about.html"
    def get_context_data(self, **kwargs):
        ctx = super().get_context_data(**kwargs)     # includes URL kwargs
        ctx["team"] = Member.objects.all()
        return ctx

# urls.py
urlpatterns = [
    path("ping/", PingView.as_view()),
    path("about/", AboutView.as_view(), name="about"),
    path("terms/", TemplateView.as_view(template_name="terms.html")),
    path("old/", RedirectView.as_view(pattern_name="about", permanent=True)),
]
```

CODE EXPLAINED

- → Always register `ClassName.as_view()`, never the bare class
- → `http_method_names` limits allowed methods (others → 405)
- → `get_context_data` must call `super()` to keep base context
- → `as_view(template_name=...)` sets attributes without subclassing
- → `RedirectView(pattern_name=...)` reverses a named URL

REQUEST FLOW

STEP	METHOD
1	`as_view()` callable runs
2	`setup(request, *a, **kw)`
3	`dispatch()` picks handler
4	`get()` / `post()` / …
405	`http_method_not_allowed()`

ON SELF

ATTRIBUTE	HOLDS
`self.request`	HttpRequest
`self.kwargs`	URL captures
`self.args`	Positional captures
`self.object`	Set by detail views

FBV VS CBV

USE	WHEN
FBV	One-off logic, clarity
View	Several methods, shared code
TemplateView	Static pages + context
Generic CBV	CRUD on a model

⚠ COMMON MISTAKES

- `path("x/", PingView)` without `.as_view()`: `TypeError` on every request
- Storing per-user data on a class attribute; it is shared by all requests
- Overriding `get_context_data` without `super()`, which drops `view` and URL kwargs

✓ TIPS & BEST PRACTICES

- Start with FBVs; switch to CBVs when you see repeated GET/POST structure
- Use ccbv.co.uk to see every attribute and method a CBV inherits
- Override `dispatch()` for checks that apply to every HTTP method

QUICK SUMMARY

- `View`: method → handler
- `as_view()`: URLconf callable
- `TemplateView`: template + context
- `RedirectView`: simple redirects

Generic Views — Display

LISTVIEW AND DETAILVIEW

`ListView` shows a list of objects and `DetailView` shows one object fetched by `pk` or `slug` from the URL. Set `model` (or `queryset`) and Django picks a default template name and context variable. Override `get_queryset()` to filter per request and `get_context_data()` to add extra variables.

NOTES

- Default template: `<app>/<model>_list.html` and `<app>/<model>_detail.html`
- Context: `object_list` / `post_list`, and `object` / `post`
- `DetailView` raises 404 if no object matches
- `paginate_by` adds `page_obj`, `paginator` and `is_paginated`
- `slug_field` and `slug_url_kwarg` customize slug lookups

DISPLAY VIEWS

```python
from django.views.generic import ListView, DetailView
from .models import Post

class PostListView(ListView):
    model = Post
    paginate_by = 20
    ordering = ["-created"]
    context_object_name = "posts"              # instead of post_list
    def get_queryset(self):
        qs = super().get_queryset().filter(published=True)
        if q := self.request.GET.get("q"):
            qs = qs.filter(title__icontains=q)
        return qs

class PostDetailView(DetailView):
    model = Post                               # finds pk or slug in URL
    template_name = "blog/post.html"
    def get_queryset(self):                    # restrict what can be seen
        return Post.objects.filter(published=True)
    def get_context_data(self, **kwargs):
        ctx = super().get_context_data(**kwargs)
        ctx["related"] = self.object.related_posts()[:3]
        return ctx
```

CODE EXPLAINED

- → `super().get_queryset()` applies `model` and `ordering`
- → `self.request.GET` gives filters; `self.kwargs` gives URL captures
- → Detail lookups go through `get_queryset()`, so filtering there limits access
- → `self.object` is set before `get_context_data` runs
- → Override `get_object()` for lookups that aren't pk/slug

LISTVIEW ATTRIBUTES

ATTRIBUTE	PURPOSE
`model / queryset`	Source rows
`ordering`	Default order
`paginate_by`	Page size
`allow_empty`	404 if empty when False
`context_object_name`	Context var name

DETAILVIEW ATTRIBUTES

ATTRIBUTE	DEFAULT
`pk_url_kwarg`	`"pk"`
`slug_url_kwarg`	`"slug"`
`slug_field`	`"slug"`
`template_name_suffix`	`"_detail"`
`query_pk_and_slug`	`False`

HOOKS TO OVERRIDE

METHOD	RETURN
`get_queryset()`	QuerySet
`get_object()`	One instance
`get_context_data()`	dict
`get_template_names()`	list of names

⚠ COMMON MISTAKES

- Filtering in `get_context_data` instead of `get_queryset`; pagination then counts the wrong rows
- Template named `post.html` when the view expects `blog/post_detail.html`
- Forgetting `ordering` with `paginate_by`: `UnorderedObjectListWarning` and unstable pages

✓ TIPS & BEST PRACTICES

- Set `context_object_name` so templates read naturally (`posts`, `post`)
- Add `select_related()` in `get_queryset()` to avoid N+1 queries
- Scope detail views to the user: `Post.objects.filter(author=self.request.user)`

QUICK SUMMARY

- `ListView`: many objects (+ pagination)
- `DetailView`: one object by pk/slug
- `get_queryset()`: filter per request
- `get_context_data()`: extra context

Generic Views — Editing

CREATE, UPDATE, DELETE

The editing views handle the full form cycle: show an empty or bound form on GET, validate on POST, save and redirect on success, or re-render with errors. `CreateView` and `UpdateView` build a **ModelForm** from `model` + `fields`, `DeleteView` asks for confirmation then deletes, and `FormView` works with any plain `Form`.

NOTES

- Default template: `<app>/<model>_form.html` for create and update
- `DeleteView` uses `<model>_confirm_delete.html` and deletes on POST
- Success redirect: `success_url`, else the object's `get_absolute_url()`
- Override `form_valid()` to set fields the user shouldn't edit
- Context has `form`, and `object` on update/delete

EDITING VIEWS

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.urls import reverse_lazy
from django.views.generic.edit import CreateView, UpdateView, DeleteView, FormView
class PostCreateView(LoginRequiredMixin, CreateView):
    model = Post
    fields = ["title", "body", "tags"]
    def form_valid(self, form):
        form.instance.author = self.request.user    # set hidden field
        return super().form_valid(form)             # saves + redirects

class PostUpdateView(LoginRequiredMixin, UpdateView):
    model = Post
    fields = ["title", "body"]
    def get_queryset(self):                         # owner-only edits
        return Post.objects.filter(author=self.request.user)

class PostDeleteView(LoginRequiredMixin, DeleteView):
    model = Post
    success_url = reverse_lazy("blog:list")

class ContactView(FormView):
    form_class, template_name = ContactForm, "contact.html"
    success_url = reverse_lazy("thanks")
```

CODE EXPLAINED

- → `fields` (or `form_class`) is required on Create/UpdateView
- → `form.instance` is the unsaved model object
- → `super().form_valid()` saves and returns the redirect
- → Owner-only edits: filter `get_queryset()` by `request.user`
- → `reverse_lazy` because URLs aren't loaded at class creation

WHICH VIEW

VIEW	DOES
`CreateView`	New object
`UpdateView`	Edit object
`DeleteView`	Confirm + delete
`FormView`	Any Form

COMMON OVERRIDES

METHOD	USE FOR
`form_valid()`	Extra save logic
`form_invalid()`	Custom error handling
`get_success_url()`	Dynamic redirect
`get_form_kwargs()`	Pass args to form
`get_initial()`	Prefill values

ATTRIBUTES

ATTRIBUTE	PURPOSE
`fields`	ModelForm fields
`form_class`	Custom form
`success_url`	Redirect target
`initial`	Initial data dict
`template_name_suffix`	`"_form"`

⚠ COMMON MISTAKES

- Using `fields = "__all__"`, which exposes fields like `author` or `is_staff` to users
- Calling `form.save()` in `form_valid` and then `super()`, saving twice
- No `get_queryset()` filter on Update/DeleteView, so any user can edit any row

✓ TIPS & BEST PRACTICES

- Put `LoginRequiredMixin` **first** in the base class list
- Add `SuccessMessageMixin` + `success_message` for flash confirmations
- Override `get_form_kwargs()` to pass `request.user` into a custom form

QUICK SUMMARY

- `CreateView` / `UpdateView`: ModelForm flow
- `DeleteView`: confirm then delete
- `form_valid()`: hook before save
- `success_url`: where to go next

View Mixins & Decorators

REUSING VIEW BEHAVIOR

Mixins add behavior to class-based views through multiple inheritance, and **decorators** wrap function views. Auth mixins check access in `dispatch()` before the handler runs. Because Python resolves methods left to right (the MRO), access mixins must come **before** the view class. `method_decorator` lets you apply any function decorator to a CBV.

NOTES

- Order: `class V(LoginRequiredMixin, PermissionRequiredMixin, UpdateView)`
- Anonymous users are redirected to `LOGIN_URL` with `?next=`
- Logged-in users without permission get a 403
- Django 5.1+: `LoginRequiredMiddleware` + `@login_not_required` makes login the default
- Write your own mixin when several views share the same override

MIXINS AND DECORATORS

```python
from django.contrib.auth.decorators import login_required, permission_required
from django.contrib.auth.mixins import (LoginRequiredMixin,
    PermissionRequiredMixin, UserPassesTestMixin)
from django.utils.decorators import method_decorator
from django.views.decorators.cache import cache_page

class ReportView(LoginRequiredMixin, PermissionRequiredMixin, ListView):
    permission_required = "reports.view_report"
    raise_exception = True                    # 403 instead of redirect

class PostEditView(UserPassesTestMixin, UpdateView):
    model = Post
    def test_func(self):
        return self.get_object().author == self.request.user

@method_decorator(cache_page(60 * 15), name="dispatch")
class HomeView(TemplateView):
    template_name = "home.html"

class OwnerQuerysetMixin:                     # custom mixin
    def get_queryset(self):
        return super().get_queryset().filter(owner=self.request.user)

@login_required
@permission_required("reports.view_report", raise_exception=True)
def report(request): ...
```

CODE EXPLAINED

- → `permission_required` is `"app_label.codename"`
- → `raise_exception = True` returns 403 even for anonymous users
- → `test_func()` returns `True` to allow access
- → `method_decorator(dec, name="dispatch")` covers every method
- → Custom mixins call `super()` so they chain with the view

AUTH MIXINS

MIXIN	CHECKS
`LoginRequiredMixin`	Logged in
`PermissionRequiredMixin`	Has permission(s)
`UserPassesTestMixin`	`test_func()`
`AccessMixin`	Base for custom ones

ACCESSMIXIN OPTIONS

ATTRIBUTE	DEFAULT
`login_url`	`LOGIN_URL`
`redirect_field_name`	next
`raise_exception`	`False`

FBV DECORATORS

DECORATOR	EFFECT
`@login_required`	Redirect if anonymous
`@permission_required`	Perm check
`@user_passes_test(fn)`	Custom check
`@cache_page(s)`	Cache response
`@csrf_exempt`	Skip CSRF

⚠ COMMON MISTAKES

- Putting `LoginRequiredMixin` after the view class; the view's `dispatch` runs first
- `test_func` calling `get_object()` on a CreateView, where there is no object yet
- Decorating a CBV class directly with `@login_required`; use `method_decorator`

✓ TIPS & BEST PRACTICES

- Combine mixins into one base class (`StaffOnlyMixin`) to keep views short
- Prefer permission checks over `is_staff` checks for fine-grained access
- Use `LoginRequiredMiddleware` for sites where nearly every page needs login

QUICK SUMMARY

- Mixins go **left** of the view class
- `LoginRequiredMixin`: auth gate
- `UserPassesTestMixin.test_func()`: custom rule
- `method_decorator`: FBV decorator on a CBV

Template Basics & Configuration

THE TEMPLATE SYSTEM

The **Django Template Language** (DTL) turns a text file plus a **context** dict into HTML. The `TEMPLATES` setting picks the engine and where to look: `DIRS` lists project-level folders and `APP_DIRS = True` also searches each installed app's `templates/` folder. Views usually call `render()`, which loads, renders and wraps the result in an `HttpResponse`.

NOTES

- Templates hold presentation only; no arbitrary Python runs in them
- Lookup order: `DIRS` first, then each app in `INSTALLED_APPS` order
- Namespace app templates: `blog/templates/blog/list.html`
- Missing variables render as an empty string (not an error)
- Jinja2 is a built-in alternative backend

CONFIGURATION AND RENDERING

```
# settings.py
TEMPLATES = [{
    "BACKEND": "django.template.backends.django.DjangoTemplates",
    "DIRS": [BASE_DIR / "templates"],          # project-wide templates
    "APP_DIRS": True,                          # + <app>/templates/
    "OPTIONS": {"context_processors": [
        "django.template.context_processors.request",
        "django.contrib.auth.context_processors.auth",
        "django.contrib.messages.context_processors.messages",
    ]},
}]

# views.py
from django.shortcuts import render
from django.template.loader import render_to_string

def home(request):
    ctx = {"title": "Welcome", "items": Item.objects.all()[:5]}
    return render(request, "home.html", ctx)

html = render_to_string("emails/welcome.txt", {"user": u})  # str only
```

CODE EXPLAINED

- → `DIRS` takes a list of paths; checked before app folders
- → Context processors add variables (`request` , `user` , `messages`) to every `render()`
- → `render()` → `HttpResponse` ; `render_to_string()` → `str`
- → `{{ }}` outputs a value; `{% %}` runs a tag
- → Context values can be any Python object

TEMPLATES/HOME.HTML

```
<h1>{{ title }}</h1>
{% for item in items %}<p>{{ item.name }}</p>{% endfor %}
```

SYNTAX

SYNTAX	MEANING
`{{ var }}`	Output (escaped)
`{% tag %}`	Logic / tags
`{{ v|filter }}`	Transform value
`{# note #}`	One-line comment
`{% load lib %}`	Load tag library

LOADER FUNCTIONS

FUNCTION	RETURNS
`get_template(n)`	Template object
`select_template([..])`	First that exists
`render_to_string()`	str
`render()`	HttpResponse
`TemplateResponse`	Lazy response

TEMPLATES KEYS

KEY	PURPOSE
`BACKEND`	Engine class
`DIRS`	Extra folders
`APP_DIRS`	Search apps
`OPTIONS`	Processors, builtins
`NAME`	Engine alias

⚠ COMMON MISTAKES

- Putting `blog/templates/list.html` without the inner `blog/` ; another app's `list.html` may win
- Forgetting `APP_DIRS: True` , so app templates raise `TemplateDoesNotExist`
- Doing heavy queries inside templates via method calls; do them in the view

✓ TIPS & BEST PRACTICES

- Keep a project-level `templates/` for `base.html` , `404.html` and overrides
- Read the "Template-loader postmortem" on the debug page to see every path tried
- Add `string_if_invalid` in dev `OPTIONS` to spot missing variables

QUICK SUMMARY

- `TEMPLATES` : engine + search paths
- `render(request, name, ctx)` : HTML response
- `{{ }}` / `{% %}` : output / logic
- Context processors: global variables

Variables & Built-in Filters

VARIABLES AND FILTERS

`{{ var }}` prints a context value. The **dot** does dictionary lookup, attribute lookup, method call (no arguments) and list index, in that order. **Filters** transform a value with `|` and can be chained; some take one argument after a colon. All output is **auto-escaped** for HTML unless you mark it safe.

NOTES

- `{{ user.profile.name }}`: attribute chain
- `{{ items.0 }}`: list index; `{{ d.key }}`: dict key
- Methods are called without arguments: `{{ post.get_absolute_url }}`
- Chain filters left to right: `{{ name|lower|truncatechars:20 }}`
- `|safe` turns off escaping: only for trusted HTML

FILTERS IN USE

```
{{ post.title|title }}                    {# Hello World #}
{{ post.body|truncatewords:30 }}          {# first 30 words... #}
{{ post.created|date:"M j, Y" }}          {# Oct 11, 2026 #}
{{ post.created|timesince }}              {# 3 days, 2 hours #}
{{ comments|length }}                     {# 12 #}
{{ count }} comment{{ count|pluralize }}  {# 1 comment / 2 comments #}
{{ price|floatformat:2 }}                 {# 19.90 #}
{{ bio|default:"No bio yet" }}            {# when falsy #}
{{ middle_name|default_if_none:"" }}      {# only when None #}
{{ body|linebreaks }}                     {# \n -> <p> / <br> #}
{{ tags|join:", " }}                      {# django, python #}
{{ html_snippet|safe }}                   {# no escaping #}
{{ data|json_script:"page-data" }}        {# safe <script> JSON #}
{{ url|urlize }}                          {# clickable links #}
{{ size|filesizeformat }}                 {# 1.2 MB #}
```

CODE EXPLAINED

→ Filter args follow a colon and are quoted if strings

→ `default` covers `""`, `0`, `None`, `[]`; `default_if_none` only `None`

→ `date` uses PHP-style format letters (`Y-m-d`, `M j`)

→ `json_script` outputs JSON in a `<script>` tag safely

→ `safe` disables escaping for that one value

TEXT FILTERS

FILTER	EXAMPLE → OUTPUT
`lower / upper`	Ann → ann
`title`	hi there → Hi There
`truncatechars:5`	Django → Djan…
`slugify`	Hi There! → hi-there
`striptags`	removes HTML tags
`wordcount`	a b c → 3

DATA FILTERS

FILTER	DOES
`first / last`	List ends
`slice:":3"`	First 3 items
`dictsort:"name"`	Sort dicts
`add:1`	Add numbers
`yesno:"y,n"`	Bool → text
`divisibleby:2`	Bool

DATE FORMAT LETTERS

CODE	OUTPUT
`Y / y`	2026 / 26
`m / n / M / F`	01 / 1 / Jan / January
`d / j`	05 / 5
`H:i`	14:30
`D / l`	Sat / Saturday

⚠ COMMON MISTAKES

- `{{ post.get_comments(5) }}`: templates can't pass method arguments
- Using `|safe` on user input, which opens an XSS hole
- Spaces around the colon: `{{ x|date : "Y" }}` is a syntax error

✓ TIPS & BEST PRACTICES

- Pass data to JavaScript with `json_script`, not `{{ data|safe }}` inside `<script>`
- Use `naturaltime` / `intcomma` from `django.contrib.humanize` for friendly output
- Prefer `default_if_none` when `0` or `""` are valid values

QUICK SUMMARY

- `{{ a.b.c }}`: dict/attr/method/index lookup
- `|filter:arg`: transform output
- `|default`: fallback value
- `|safe`: opt out of escaping

Built-in Tags & Control Flow

TAGS FOR LOGIC

Tags in {% %} add logic: conditions, loops, URL building, CSRF tokens and more. {% if %} supports and, or, not, comparisons and in. {% for %} loops over any iterable, provides a forloop object and has an {% empty %} branch for empty lists. Block tags always need their closing tag ({% endif %}, {% endfor %}).

NOTES

- No parentheses in {% if %}; nest tags instead
- {% for k, v in d.items %} unpacks pairs
- forloop.length (6.0) gives the total number of items
- {% with %} caches an expensive value for a block
- {% querystring %} (5.1+) rebuilds the current query string

CONTROL FLOW

```
{% if user.is_authenticated and user.is_staff %}
  <a href="{% url 'admin:index' %}">Admin</a>
{% elif user.is_authenticated %}
  Hi {{ user.first_name|default:user.username }}
{% else %}
  <a href="{% url 'login' %}?next={{ request.path }}">Log in</a>
{% endif %}

<ul>
{% for post in posts %}
  <li class="{% cycle 'odd' 'even' %}">
    {{ forloop.counter }}/{{ forloop.length }} {{ post.title }}
    {% if forloop.last %}(newest){% endif %}
  </li>
{% empty %}
  <li>No posts yet.</li>
{% endfor %}
</ul>

{% with total=order.items.count %}{{ total }} item{{ total|pluralize }}{% endwith %}
<a href="{% querystring page=page_obj.next_page_number %}">Next</a>
<form method="post">{% csrf_token %} ... </form>
{% comment "TODO" %} hidden from output {% endcomment %}
```

CODE EXPLAINED

- → elif / else are optional branches
- → {% empty %} runs when the iterable has no items
- → {% cycle %} alternates values on each loop pass
- → {% with %} evaluates once and names the result
- → {% csrf_token %} is required in every POST form

FORLOOP VARIABLES

VARIABLE	VALUE
counter / counter0	1-based / 0-based
revcounter	Items left (1-based)
first / last	Bool
length	Total items (6.0)
parentloop	Outer loop

{% IF %} OPERATORS

OPERATOR	EXAMPLE
and / or / not	a and not b
== != < > <= >=	n > 0
in / not in	"x" in tags
is / is not	v is None

OTHER TAGS

TAG	DOES
{% url %}	Reverse a URL
{% now "Y" %}	Current date
{% spaceless %}	Strip whitespace
{% firstof a b %}	First truthy
{% regroup %}	Group a list
{% lorem %}	Placeholder text

⚠ COMMON MISTAKES

- {% if (a or b) and c %}: parentheses aren't allowed
- Calling .all in a loop condition repeatedly; use {% with %} or prepare it in the view
- Leaving out {% csrf_token %} in a POST form: 403 CSRF verification failed

✓ TIPS & BEST PRACTICES

- Use {% empty %} instead of a separate {% if posts %} check
- Use {% querystring %} for pagination links so filters are kept
- Use {% regroup %} to group a sorted list by a field

QUICK SUMMARY

- {% if %}…{% endif %}: conditions
- {% for %}…{% empty %}: loops
- forloop.*: loop state
- {% with %}: local variable

Inheritance, Includes & Partials

REUSING TEMPLATE CODE

Inheritance lets a child template `{% extends %}` a base layout and override its named `{% block %}`s. `{% include %}` drops another template in place with the current context. **Partials** (new in 6.0) define named fragments inside one file with `{% partialdef %}` and render them with `{% partial %}` or from a view as `"file.html#name"`, which pairs well with htmx.

NOTES

- `{% extends %}` must be the **first** tag in the child
- Blocks not overridden keep the parent's content
- `{{ block.super }}` adds to the parent block instead of replacing it
- `{% include "x.html" with a=1 only %}` limits the context
- Partials: `{% partialdef name inline %}` also renders it in place

BASE.HTML

```
<!doctype html>
<html><head><title>{% block title %}My Site{% endblock %}</title></head>
<body>
  {% include "partials/nav.html" %}
  <main>{% block content %}{% endblock content %}</main>
</body></html>
```

BLOG/POST_LIST.HTML

```
{% extends "base.html" %}
{% block title %}Posts · {{ block.super }}{% endblock %}
{% block content %}
  {% partialdef post-row %}
    <li id="post-{{ post.pk }}">{{ post.title }}</li>
  {% endpartialdef %}
  <ul>{% for post in posts %}{% partial post-row %}{% endfor %}</ul>
  {% include "blog/sidebar.html" with tags=tags only %}
{% endblock content %}
```

CODE EXPLAINED

- → Child output = parent layout with child blocks swapped in
- → `{{ block.super }}` → `Posts · My Site`
- → Naming the end tag (`endblock content`) helps readability
- → `#post-row` renders just that fragment: ideal for htmx swaps
- → `only` passes just the listed variables to the include

RENDER A PARTIAL FROM A VIEW

```
render(request, "blog/post_list.html#post-row", {"post": post})
```

REUSE TAGS

TAG	USE
extends	Inherit a layout
block	Overridable region
include	Insert a file
partialdef	Define a fragment
partial	Render a fragment

THREE-LEVEL PATTERN

LEVEL	FILE
1 Site	base.html
2 Section	blog/base_blog.html
3 Page	blog/post_list.html

WHICH TO USE

NEED	USE
Shared layout	extends + block
Shared widget	include
Repeated in one file	partialdef
htmx fragment	"file#name"

⚠ COMMON MISTAKES

- Any output or tag before `{% extends %}` ; it's ignored or errors
- Defining the same block name twice in one template: `TemplateSyntaxError`
- Content outside any `{% block %}` in a child; it never renders

✓ TIPS & BEST PRACTICES

- Keep one `base.html` with blocks for `title` , `content` , `extra_css` , `extra_js`
- Use partials instead of many tiny include files for htmx endpoints
- Use `{% include ... only %}` to make includes predictable

QUICK SUMMARY

- `{% extends %}` + `{% block %}` : layouts
- `{{ block.super }}` : extend, not replace
- `{% include %}` : insert a template
- `{% partialdef %}` / `#name` : fragments (6.0)

Static Files

CSS, JAVASCRIPT AND IMAGES

Static files are the assets you ship with the code: CSS, JavaScript, fonts and images. In development `django.contrib.staticfiles` serves them from each app's `static/` folder and from `STATICFILES_DIRS`. For production, `collectstatic` copies everything into `STATIC_ROOT`, where a web server, CDN or WhiteNoise serves it. User uploads are **media**, not static (Sheet 43).

NOTES

- Namespace app assets: `blog/static/blog/style.css`
- `{% static %}` builds the URL, so changing `STATIC_URL` never breaks links
- `runserver` serves static files only when `DEBUG = True`
- `ManifestStaticFilesStorage` adds content hashes for cache-busting
- `STORAGES["staticfiles"]` picks the storage backend

SETTINGS

```
STATIC_URL = "static/"                          # URL prefix
STATICFILES_DIRS = [BASE_DIR / "static"]        # project-level assets
STATIC_ROOT = BASE_DIR / "staticfiles"          # collectstatic target
STORAGES = {
    "default": {"BACKEND": "django.core.files.storage.FileSystemStorage"},
    "staticfiles": {"BACKEND":
        "django.contrib.staticfiles.storage.ManifestStaticFilesStorage"},
}
```

TEMPLATE

```
{% load static %}
<link rel="stylesheet" href="{% static 'css/site.css' %}">
<img src="{% static 'blog/logo.png' %}" alt="Logo">
<script src="{% static 'js/app.js' %}" defer></script>
```

COMMANDS

```
python manage.py findstatic css/site.css      # which file wins?
python manage.py collectstatic --noinput      # copy to STATIC_ROOT
```

CODE EXPLAINED

- → `{% load static %}` is needed once per template file
- → `{% static 'css/site.css' %}` → `/static/css/site.a1b2c3.css` with the manifest
- → `STATIC_ROOT` must differ from every `STATICFILES_DIRS` entry
- → `findstatic` shows search order when two files share a name
- → Run `collectstatic` on every deploy

SETTINGS

SETTING	ROLE
`STATIC_URL`	Public URL prefix
`STATICFILES_DIRS`	Extra source folders
`STATIC_ROOT`	Collected output
`STORAGES`	Storage backends

STATIC VS MEDIA

	STATIC / MEDIA
Who adds	Developer / users
In git	Yes / No
URL setting	STATIC_URL / MEDIA_URL
Folder	STATIC_ROOT / MEDIA_ROOT

SERVING IN PRODUCTION

OPTION	NOTES
WhiteNoise	Python serves, easy
Nginx	`location /static/`
CDN / S3	`django-storages`

⚠ COMMON MISTAKES

- Hard-coding `/static/css/site.css`; it breaks with a CDN or hashed filenames
- Pointing `STATIC_ROOT` at a `STATICFILES_DIRS` folder: `ImproperlyConfigured`
- Static files 404 after `DEBUG = False`; you need WhiteNoise or a web server

✓ TIPS & BEST PRACTICES

- Add WhiteNoise early; it makes static serving work the same everywhere
- Use hashed filenames + long cache headers for fast repeat loads
- Keep build tools (Vite, Tailwind) outputting into a `STATICFILES_DIRS` folder

QUICK SUMMARY

- `{% static 'path' %}`: asset URL
- `STATICFILES_DIRS`: source folders
- `collectstatic` → `STATIC_ROOT`
- Manifest storage: cache-busting

Custom Tags & Filters

EXTENDING THE TEMPLATE LANGUAGE

When built-ins aren't enough, write your own. Create a `templatetags/` package in an app, make a module with a `template.Library()` named `register`, and decorate functions as **filters**, **simple tags** or **inclusion tags**. Templates load the module by its file name with `{% load %}`. **Context processors** are the other extension point: functions that add variables to every template.

NOTES

- `templatetags/` needs an `__init__.py`; restart `runserver` after creating it
- Filters take one value and at most one argument
- `simple_tag` takes any arguments and returns a string
- `inclusion_tag` renders a small template with its own context
- Mark returned HTML with `format_html()` so it's escaped correctly

BLOG/TEMPLATETAGS/BLOG_EXTRAS.PY

```python
from django import template
from django.utils.html import format_html

register = template.Library()

@register.filter
def initials(value):                          # {{ name|initials }}
    return "".join(w[0].upper() for w in value.split())

@register.simple_tag(takes_context=True)
def active(context, url_name):                # {% active 'home' %}
    match = context["request"].resolver_match
    return "active" if match and match.url_name == url_name else ""

@register.simple_tag
def badge(text, color="blue"):                # {% badge "New" color="red" %}
    return format_html('<span class="{}">{}</span>', color, text)

@register.inclusion_tag("blog/_recent.html")
def recent_posts(count=5):                    # {% recent_posts 3 %}
    return {"posts": Post.objects.order_by("-created")[:count]}
```

CODE EXPLAINED

- → The module name (`blog_extras`) is what `{% load %}` uses
- → `takes_context=True` passes the template context first
- → `format_html` escapes the arguments but keeps the markup
- → Inclusion tags return a context dict for their own template
- → Context processor: `def site(request): return {"SITE_NAME": ...}`

USAGE

```
{% load blog_extras %}
{{ "Ada Lovelace"|initials }}  {% badge "New" %}  {% recent_posts 3 %}
```

EXTENSION TYPES

TYPE	SYNTAX
filter	{{ v|f:arg }}
simple_tag	{% t a b=1 %}
… as var	{% t a as x %}
inclusion_tag	Renders a template
context processor	Global vars

FILTER OPTIONS

OPTION	EFFECT
is_safe=True	Keeps safe input safe
needs_autoescape	Gets autoescape flag
expects_localtime	Gets local datetime
name="x"	Register under name

USEFUL LIBRARIES

{% LOAD %}	GIVES
humanize	intcomma, naturaltime
static	{% static %}
cache	{% cache %}
i18n	{% translate %}
tz	{% timezone %}

⚠ COMMON MISTAKES

- Returning raw f-string HTML from a tag; it's escaped, or worse, unsafe with `mark_safe`
- Naming the tag module the same as an app (`blog.py`), which confuses imports
- Running queries in a filter used inside a loop, creating N+1 queries

✓ TIPS & BEST PRACTICES

- Add `"builtins": ["blog.templatetags.blog_extras"]` to `OPTIONS` to skip `{% load %}`
- Keep context processors cheap; they run on every `render()`
- Test tags with `Template("{% load x %}...").render(Context({...}))`

QUICK SUMMARY

- `register = template.Library()`
- `@register.filter` : value transform
- `@register.simple_tag` : function tag
- `@register.inclusion_tag` : mini template

Model Basics

MODELS MAP TO TABLES

A **model** is a Python class that subclasses `models.Model`. Each model maps to one database table and each field attribute to one column. Django adds an auto-incrementing `id` primary key unless you define one, and a default **manager** called `objects` for queries. Instances are rows: create them, change attributes, then call `save()`.

NOTES

- Table name is `<app_label>_<modelname>`, e.g. `blog_post`
- 6.0+: the default primary key type is `BigAutoField`
- `__str__` controls how objects appear in the admin and shell
- `save()` does INSERT for new rows and UPDATE for existing ones
- Run `makemigrations` + `migrate` after any model change (Sheet 30)

DEFINING AND USING A MODEL

```python
# blog/models.py
from django.db import models

class Post(models.Model):
    title = models.CharField(max_length=200)
    body = models.TextField(blank=True)
    published = models.BooleanField(default=False)
    created = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title

# python manage.py shell
p = Post(title="Hello")              # not in the DB yet
p.save()                             # INSERT; p.id -> 1
p.title = "Hello, Django"
p.save(update_fields=["title"])      # UPDATE only that column
Post.objects.create(title="Two")     # build + save in one step
p.pk                                 # 1 (alias for the primary key)
p.refresh_from_db()                  # reload from the database
p.delete()                           # (1, {'blog.Post': 1})
Post.objects.count()                 # 1
```

CODE EXPLAINED

→ Field classes define the column type and validation

→ `auto_now_add=True` sets the time once, on creation

→ `update_fields` writes only the listed columns (faster, safer)

→ `objects.create()` returns the saved instance

→ `delete()` returns the count of deleted rows per model

INSTANCE METHODS

METHOD	DOES
`save()`	Insert or update
`delete()`	Delete row (+ cascades)
`refresh_from_db()`	Reload fields
`full_clean()`	Run validation
`get_FOO_display()`	Label for a choice

THINGS DJANGO ADDS

NAME	WHAT
`id / pk`	Primary key
`objects`	Default manager
`DoesNotExist`	Exception class
`MultipleObjectsReturned`	Exception class
`_meta`	Model metadata

SAVE() OPTIONS

ARGUMENT	EFFECT
`update_fields`	Only these columns
`force_insert`	Always INSERT
`force_update`	Always UPDATE
`using`	Pick a database

⚠ COMMON MISTAKES

- Changing a model and forgetting `makemigrations`: `no such column` errors
- Expecting `save()` to run `full_clean()`; validation only runs in forms or manually
- Overriding `save()` without calling `super().save(*args, **kwargs)`

✓ TIPS & BEST PRACTICES

- Always define `__str__`; it makes the admin and debugging far easier
- Use `update_fields` in background jobs to avoid overwriting concurrent edits
- Explore models live with `python manage.py shell` (models are auto-imported)

QUICK SUMMARY

- `class M(models.Model)`: one table
- Field attribute = column
- `save()` / `delete()`: write rows
- `M.objects`: the query manager

Field Types

CharField · TextField · IntegerField · DecimalField · DateTimeField · BooleanField · JSONField · FileField

CHOOSING FIELD TYPES

Each field class sets the **database column type**, the default **form widget** and basic **validation**. Pick the narrowest type that fits the data: `CharField` needs `max_length`, `DecimalField` needs `max_digits` and `decimal_places`, and file fields store a path while the file itself lives in storage. Relationship fields are covered on Sheets 25–26.

NOTES

- Use `DecimalField` for money; `FloatField` has rounding errors
- `DateTimeField` stores aware UTC datetimes when `USE_TZ = True`
- `JSONField` works on all supported databases
- `ImageField` needs the Pillow package
- `GeneratedField` (5.0+) is computed by the database

A MODEL USING MANY TYPES

```
import uuid
from django.db import models
from django.db.models.functions import Lower

class Product(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4)
    name = models.CharField(max_length=120)
    slug = models.SlugField(unique=True)
    description = models.TextField(blank=True)
    price = models.DecimalField(max_digits=8, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)
    rating = models.FloatField(null=True)
    active = models.BooleanField(default=True)
    released = models.DateField(null=True, blank=True)
    updated = models.DateTimeField(auto_now=True)
    website = models.URLField(blank=True)
    contact = models.EmailField()
    specs = models.JSONField(default=dict)
    manual = models.FileField(upload_to="manuals/", blank=True)
    photo = models.ImageField(upload_to="products/%Y/", blank=True)
    name_lower = models.GeneratedField(expression=Lower("name"),
        output_field=models.CharField(max_length=120), db_persist=True)
```

CODE EXPLAINED

- → `max_digits=8, decimal_places=2` holds up to `999999.99`
- → `auto_now=True` updates the timestamp on every `save()`
- → `default=dict` (a callable) gives each row its own empty dict
- → `upload_to` can include date format codes like `%Y`
- → `GeneratedField` is read-only; the DB keeps it in sync

TEXT & NUMBERS

FIELD	COLUMN / NOTE
`CharField`	VARCHAR(max_length)
`TextField`	Unlimited text
`SlugField`	Letters, digits, – _
`IntegerField`	32-bit int
`BigIntegerField`	64-bit int
`DecimalField`	Exact decimal

DATES & SPECIAL

FIELD	PYTHON TYPE
`DateField`	`date`
`DateTimeField`	`datetime`
`TimeField`	`time`
`DurationField`	`timedelta`
`UUIDField`	`UUID`
`GenericIPAddressField`	str

PRIMARY KEYS

FIELD	RANGE
`AutoField`	32-bit int
`BigAutoField`	64-bit (default 6.0)
`SmallAutoField`	16-bit int
`UUIDField(pk)`	Random 128-bit

⚠ COMMON MISTAKES

- `default={}` or `default=[]`: one shared mutable object; pass `dict` or `list`
- `FloatField` for prices; `0.1 + 0.2` gives `0.30000000000000004`
- `CharField` without `max_length`: system check error `fields.E120`

✓ TIPS & BEST PRACTICES

- Use `PositiveIntegerField` for counts so negatives are rejected
- Prefer `TextField` over large `CharField` on PostgreSQL; there's no speed difference
- Use `UUIDField` as a public ID and keep the integer `pk` internal

QUICK SUMMARY

- `CharField(max_length=)`: short text
- `DecimalField`: money
- `DateTimeField(auto_now_add=)`: timestamps
- `JSONField(default=dict)`: structured data

Field Options

OPTIONS EVERY FIELD ACCEPTS

Field options fine-tune a column. `null` is about the **database** (can the column hold NULL?), while `blank` is about **validation** (can a form leave it empty?). `choices` limits values and gives readable labels through `get_FOO_display()`. Others add uniqueness, indexes, defaults and the labels shown in forms and the admin.

NOTES

- For text fields use `blank=True` alone; empty string means "no value"
- `null=True, blank=True` is the usual pair for optional non-text fields
- `default` can be a value or a callable like `timezone.now`
- `db_default` (5.0+) sets the default in the database itself
- `editable=False` hides a field from forms and the admin

OPTIONS IN PRACTICE

```python
from django.db import models
from django.db.models.functions import Now
from django.utils import timezone

class Order(models.Model):
    class Status(models.TextChoices):
        PENDING = "P", "Pending"
        SHIPPED = "S", "Shipped"
        CANCELLED = "C", "Cancelled"

    number = models.CharField(max_length=20, unique=True)
    status = models.CharField(max_length=1, choices=Status,
                              default=Status.PENDING, db_index=True)
    note = models.TextField(blank=True, help_text="Shown on the invoice")
    shipped_at = models.DateTimeField(null=True, blank=True)
    placed_at = models.DateTimeField(default=timezone.now)
    created = models.DateTimeField(db_default=Now())
    code = models.CharField("Promo code", max_length=10, blank=True)

o = Order(number="A-1")
o.status                          # 'P'
o.get_status_display()            # 'Pending'
Order.Status.SHIPPED.label        # 'Shipped'
Order.objects.filter(status=Order.Status.SHIPPED)
```

CODE EXPLAINED

- → `TextChoices` members store the first value, display the second
- → 5.0+: pass the enum class directly as `choices=`
- → `default=timezone.now` (no parentheses) runs per row
- → `db_default=Now()` lets the DB fill the value on INSERT
- → First positional argument is `verbose_name`

NULL VS BLANK

COMBO	MEANING
`(default)`	Required everywhere
`blank=True`	Form may be empty
`null=True`	DB allows NULL
`both`	Optional non-text

COMMON OPTIONS

OPTION	EFFECT
`unique`	UNIQUE constraint
`db_index`	Adds an index
`choices`	Allowed values
`validators`	Extra checks
`db_column`	Column name
`db_comment`	Column comment

CHOICES TYPES

TYPE	STORED AS
`TextChoices`	str
`IntegerChoices`	int
`list of tuples`	First item
`dict`	Keys (5.0+)

⚠ COMMON MISTAKES

- `null=True` on a `CharField`; you get two "empty" values: `None` and `""`
- `default=timezone.now()` with parentheses; every row gets the server start time
- Thinking `choices` adds a DB constraint; it only validates in forms and `full_clean()`

✓ TIPS & BEST PRACTICES

- Add a `CheckConstraint` if `choices` must be enforced in the database
- Use `help_text` generously; it appears in forms and the admin
- Put choices classes inside the model so they travel with it

QUICK SUMMARY

- `null`: database; `blank`: forms
- `choices=TextChoices`: labeled values
- `default` vs `db_default`: Python vs DB
- `unique` / `db_index`: constraints, speed

ForeignKey Relationships

MANY-TO-ONE LINKS

A `ForeignKey` creates a **many-to-one** relationship: many comments belong to one post. The column stores the related row's primary key (`post_id`). `on_delete` is required and decides what happens to comments when their post is deleted. Django adds a **reverse accessor** on the other model, named `comment_set` by default or whatever `related_name` says.

NOTES

- Access the object with `c.post`; the raw id with `c.post_id` (no query)
- Refer to models by string (`"auth.User"`) to avoid circular imports
- Use `settings.AUTH_USER_MODEL` for links to users
- An index is created on every `ForeignKey` column automatically
- 6.1: `DB_CASCADE` and friends let the database handle deletes

DEFINING AND FOLLOWING A FOREIGNKEY

```
from django.conf import settings
from django.db import models

class Post(models.Model):
    title = models.CharField(max_length=200)

class Comment(models.Model):
    post = models.ForeignKey(Post, on_delete=models.CASCADE,
                             related_name="comments")
    author = models.ForeignKey(settings.AUTH_USER_MODEL,
                               on_delete=models.SET_NULL, null=True)
    category = models.ForeignKey("Category", on_delete=models.PROTECT)
    text = models.TextField()

c = Comment.objects.get(pk=1)
c.post                      # <Post: Hello>   (1 query)
c.post_id                   # 3               (no query)
post = Post.objects.get(pk=3)
post.comments.all()         # reverse: every comment on the post
post.comments.create(text="Nice!", category=cat)
post.delete()               # CASCADE deletes its comments too
```

CODE EXPLAINED

- → `related_name="comments"` replaces the default `comment_set`
- → `SET_NULL` requires `null=True` on the field
- → `PROTECT` raises `ProtectedError` if a category is still in use
- → `filter(post__title=...)` follows the FK in a lookup (a JOIN)
- → `post.comments.create()` sets `post` for you

ON_DELETE OPTIONS

OPTION	WHEN PARENT DELETED
`CASCADE`	Delete children
`PROTECT`	Block, `ProtectedError`
`RESTRICT`	Block unless cascaded
`SET_NULL`	Set to NULL
`SET_DEFAULT / SET()`	Set a value
`DO_NOTHING`	DB decides (risky)

DB-LEVEL (6.1)

OPTION	SQL
`DB_CASCADE`	`ON DELETE CASCADE`
`DB_SET_NULL`	`ON DELETE SET NULL`
`DB_SET_DEFAULT`	`ON DELETE SET DEFAULT`
`Note`	No `pre/post_delete` signals

OTHER OPTIONS

OPTION	PURPOSE
`related_name`	Reverse name
`related_query_name`	Reverse lookup name
`limit_choices_to`	Filter form choices
`to_field`	Point at a non-pk field
`db_constraint`	Create FK constraint?

⚠ COMMON MISTAKES

- `SET_NULL` without `null=True`: system check error
- Two FKs to the same model without `related_name`: reverse accessor clash
- Looping `for c in comments: c.post.title` without `select_related`, which is N+1 queries

✓ TIPS & BEST PRACTICES

- Default to `PROTECT` for lookup tables, `CASCADE` for owned children
- Use `c.post_id` when you only need the id
- Pick plural `related_name`s that read well: `post.comments`, `user.orders`

QUICK SUMMARY

- `ForeignKey(Model, on_delete=)`: many-to-one
- `obj.fk` / `obj.fk_id`: object / raw id
- `parent.related_name.all()`: reverse
- `CASCADE` / `PROTECT` / `SET_NULL`

Many-to-Many & One-to-One

M2M AND 1:1 LINKS

A `ManyToManyField` links many rows to many rows (posts ↔ tags) through a hidden **join table**. You manage links with `add()`, `remove()`, `set()` and `clear()` rather than assigning. Use `through=` to supply your own join model when the link needs extra data, such as a join date. A `OneToOneField` is a unique `ForeignKey`, ideal for extending a model like `User` with a profile.

NOTES

- Declare the M2M on either model; the other side gets a reverse accessor
- Both objects must be saved before calling `add()`
- `set()` replaces the whole set; `add()` keeps existing links
- With a `through` model, pass `through_defaults=` to `add()`
- One-to-one reverse access raises `DoesNotExist` if missing

M2M AND ONE-TO-ONE

```
class Tag(models.Model):
    name = models.CharField(max_length=30, unique=True)

class Post(models.Model):
    title = models.CharField(max_length=200)
    tags = models.ManyToManyField(Tag, related_name="posts", blank=True)

post.tags.add(django, python)          # link (no duplicates)
post.tags.remove(python)               # unlink
post.tags.set([django, web])           # replace all links
post.tags.clear()                      # remove every link
django.posts.all()                     # reverse side
Post.objects.filter(tags__name="django")

class Membership(models.Model):        # custom join table
    person = models.ForeignKey("Person", on_delete=models.CASCADE)
    group = models.ForeignKey("Group", on_delete=models.CASCADE)
    joined = models.DateField()
class Group(models.Model):
    members = models.ManyToManyField("Person", through="Membership")
group.members.add(ann, through_defaults={"joined": date.today()})

class Profile(models.Model):
    user = models.OneToOneField(settings.AUTH_USER_MODEL,
                                on_delete=models.CASCADE)
user.profile.bio                       # reverse 1:1 (no _set)
```

CODE EXPLAINED

- → `add()` / `remove()` write to the join table immediately
- → `tags__name` filters across the M2M (may duplicate rows; use `.distinct()`)
- → `through=` model holds the two FKs plus extra columns
- → `through_defaults` fills the extra columns on `add()`
- → One-to-one reverse uses the lowercase model name: `user.profile`

RELATED MANAGER

METHOD	EFFECT
`add(*objs)`	Create links
`remove(*objs)`	Delete links
`set(objs)`	Replace links
`clear()`	Delete all links
`create(**kw)`	New obj + link

RELATIONSHIP TYPES

FIELD	CARDINALITY
`ForeignKey`	Many → one
`ManyToManyField`	Many ↔ many
`OneToOneField`	One ↔ one
`GenericForeignKey`	Any model

M2M OPTIONS

OPTION	PURPOSE
`through`	Custom join model
`related_name`	Reverse name
`symmetrical`	Self-links both ways
`db_table`	Join table name

⚠ COMMON MISTAKES

- `post.tags = [t1, t2]`: direct assignment is not allowed, use `set()`
- Calling `add()` on an unsaved object: `ValueError` (needs a primary key)
- Forgetting `.distinct()` when filtering across M2M, so rows appear twice

✓ TIPS & BEST PRACTICES

- Start with `through=` if the link might ever need extra data; migrating later is painful
- Create profiles in a `post_save` signal or lazily with `get_or_create`
- Use `prefetch_related("tags")` when listing objects with their tags

QUICK SUMMARY

- `ManyToManyField`: join table
- `add` / `remove` / `set` / `clear`: manage links
- `through=`: extra link data
- `OneToOneField`: unique FK

Model Meta Options

THE INNER META CLASS

An inner `class Meta` holds model-level settings that aren't fields: default ordering, human-readable names, the table name, **indexes** and **constraints**. Constraints are enforced by the database and checked by `full_clean()`, so they catch bad data from any source. Changes to `Meta` usually need a migration.

NOTES

- `ordering` applies to every query unless overridden with `order_by()`
- `UniqueConstraint` replaces the older `unique_together`
- `CheckConstraint(condition=Q(...))` enforces rules in SQL (5.1+ uses `condition`)
- Conditional unique constraints use `condition=`
- Read metadata at runtime through `Model._meta`

META IN PRACTICE

```python
from django.db import models
from django.db.models import Q, F
from django.db.models.functions import Lower

class Booking(models.Model):
    room = models.ForeignKey("Room", on_delete=models.CASCADE)
    guest_email = models.EmailField()
    start = models.DateField()
    end = models.DateField()
    cancelled = models.BooleanField(default=False)

    class Meta:
        ordering = ["-start", "room"]
        verbose_name_plural = "bookings"
        indexes = [
            models.Index(fields=["room", "start"]),
            models.Index(Lower("guest_email"), name="booking_email_lower"),
        ]
        constraints = [
            models.CheckConstraint(condition=Q(end__gt=F("start")),
                                   name="booking_end_after_start"),
            models.UniqueConstraint(fields=["room", "start"],
                condition=Q(cancelled=False), name="one_active_booking"),
        ]
```

CODE EXPLAINED

→ `"-start"` sorts descending; ties broken by `room`

→ Indexes on expressions need an explicit `name`

→ `CheckConstraint` → `CHECK (end > start)` in the database

→ Conditional unique: only one non-cancelled booking per room/day

→ Violations raise `IntegrityError` on save, `ValidationError` in forms

COMMON META OPTIONS

OPTION	PURPOSE
`ordering`	Default sort
`verbose_name(_plural)`	Display names
`db_table`	Table name
`indexes`	Index list
`constraints`	Constraint list
`get_latest_by`	For `latest()`

ADVANCED META

OPTION	PURPOSE
`abstract`	No table (Sheet 29)
`proxy`	Same table
`managed`	Skip migrations
`permissions`	Custom perms
`default_related_name`	Reverse name

CONSTRAINT CLASSES

CLASS	SQL
`UniqueConstraint`	UNIQUE
`CheckConstraint`	CHECK
`ExclusionConstraint`	EXCLUDE (Postgres)

⚠ COMMON MISTAKES

- Heavy `ordering` on huge tables; every query sorts even when you don't need it
- Duplicate constraint names across models; names must be unique per database
- Using `unique_together` in new code; prefer `UniqueConstraint`

✓ TIPS & BEST PRACTICES

- Use `%(app_label)s_%(class)s` in names on abstract base classes
- Add indexes for columns you filter or sort by often; check with `.explain()`
- Set `managed = False` for tables owned by another system

QUICK SUMMARY

- `class Meta`: model-level settings
- `ordering`: default sort
- `indexes`: speed up lookups
- `constraints`: DB-enforced rules

Model Methods & Managers

PUTTING LOGIC ON MODELS

Keep row-level logic on the model as **methods** and **properties**, and table-level logic (reusable filters) on a **custom QuerySet** exposed through a **manager**. `QuerySet.as_manager()` gives you chainable methods like `Post.objects.published().recent()`. `get_absolute_url()` and `clean()` are hooks Django itself calls.

NOTES

- `get_absolute_url()` is used by `redirect(obj)`, the admin "View on site" and generic views
- `clean()` runs in `full_clean()` and ModelForm validation, not on `save()`
- The first manager defined becomes `_default_manager`
- A manager that filters rows hides them from the admin too
- Methods on a QuerySet chain; methods on a Manager don't

METHODS, PROPERTIES, QUERYSETS

```
from django.core.exceptions import ValidationError
from django.urls import reverse
from django.utils import timezone
class PostQuerySet(models.QuerySet):
    def published(self):
        return self.filter(status="P", pub_date__lte=timezone.now())

class Post(models.Model):
    title = models.CharField(max_length=200)
    slug = models.SlugField(unique=True)
    status = models.CharField(max_length=1, default="D")
    pub_date = models.DateTimeField(null=True, blank=True)
    objects = PostQuerySet.as_manager()

    def get_absolute_url(self):
        return reverse("blog:detail", kwargs={"slug": self.slug})
    @property
    def is_live(self):
        return self.status == "P" and self.pub_date <= timezone.now()
    def clean(self):
        if self.status == "P" and not self.pub_date:
            raise ValidationError({"pub_date": "Required to publish."})

Post.objects.published().filter(author=user).order_by("-pub_date")
```

CODE EXPLAINED

- → QuerySet methods return QuerySets, so they chain
- → `as_manager()` copies QuerySet methods onto `objects`
- → `@property` reads like a field in templates: `{{ post.is_live }}`
- → A dict in `ValidationError` attaches errors to specific fields
- → `get_absolute_url` keeps URL logic in one place

HOOKS DJANGO CALLS

METHOD	CALLED BY
`__str__()`	Admin, shell, `{{ obj }}`
`get_absolute_url()`	`redirect`, admin
`clean()`	`full_clean()`, forms
`save()` / `delete()`	Writes

MANAGER PATTERNS

PATTERN	CODE
Chainable	`QS.as_manager()`
Mixed	`from_queryset(QS)`
Filtered	Override `get_queryset()`
Second manager	`live = LiveManager()`

FULL_CLEAN() STEPS

STEP	CHECKS
`clean_fields()`	Each field
`clean()`	Cross-field
`validate_unique()`	Unique fields
`validate_constraints()`	Meta constraints

⚠ COMMON MISTAKES

- Putting filters in views repeatedly instead of a QuerySet method
- Replacing `objects` with a filtering manager; the admin and related lookups then miss rows
- Expecting `save()` to call `clean()`; call `full_clean()` yourself if needed

✓ TIPS & BEST PRACTICES

- Name QuerySet methods for business meaning: `.published()`, `.overdue()`
- Use `cached_property` for expensive per-instance values
- Keep the unfiltered manager first, add filtered ones after it

QUICK SUMMARY

- Model methods: row logic
- `QuerySet.as_manager()`: chainable filters
- `get_absolute_url()`: canonical URL
- `clean()`: cross-field validation

Model Inheritance

THREE KINDS OF INHERITANCE

Django supports three styles. **Abstract base classes** share fields and methods but create no table of their own. **Multi-table inheritance** gives each class its own table, joined by an automatic one-to-one link. **Proxy models** reuse the parent's table exactly and only change Python behavior: ordering, managers or methods.

NOTES

- Abstract bases are the most common and the cheapest
- Multi-table queries need a JOIN on every access
- Proxy models can't add fields
- Child `Meta` inherits from an abstract parent's `Meta` unless overridden
- `parent_link=True` names the auto one-to-one explicitly

ALL THREE STYLES

```
# 1. Abstract base: fields copied into each child table
class TimeStampedModel(models.Model):
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)
    class Meta:
        abstract = True
        ordering = ["-created"]

class Article(TimeStampedModel):          # table: has created, updated
    title = models.CharField(max_length=200)

# 2. Multi-table: two tables, joined 1:1
class Place(models.Model):
    name = models.CharField(max_length=50)
class Restaurant(Place):                  # adds place_ptr_id
    serves_pizza = models.BooleanField(default=False)

r = Restaurant.objects.create(name="Luigi's", serves_pizza=True)
Place.objects.get(name="Luigi's").restaurant   # downcast via 1:1

# 3. Proxy: same table, different behavior
class PublishedArticle(Article):
    class Meta:
        proxy = True
        ordering = ["title"]
    objects = PublishedQuerySet.as_manager()
```

CODE EXPLAINED

- → `abstract = True`: no migration creates a table for the base
- → Children of an abstract base inherit its `Meta.ordering`
- → Multi-table child gets `place_ptr` as its primary key
- → `place.restaurant` raises `DoesNotExist` if it isn't one
- → Proxy can register separately in the admin with its own options

COMPARISON

STYLE	TABLES / USE
`Abstract`	Child only · shared fields
`Multi-table`	Both · real subtypes
`Proxy`	Parent only · behavior

META FLAGS

FLAG	EFFECT
`abstract = True`	No table
`proxy = True`	Reuse table
`managed = False`	No migrations

RELATED_NAME IN ABSTRACT

PLACEHOLDER	BECOMES
`%(class)s`	lowercased child
`%(app_label)s`	app label
`Example`	`"%(class)s_set"`

⚠ COMMON MISTAKES

- Multi-table inheritance for shared fields only; every query pays for a JOIN
- A fixed `related_name` on an FK in an abstract base, which clashes across children
- Adding a field to a proxy model: `FieldError`

✓ TIPS & BEST PRACTICES

- Make a `TimeStampedModel` abstract base for nearly every project
- Prefer composition (`OneToOneField`) over multi-table inheritance
- Use proxies to give one table two admin views (e.g. staff vs customers)

QUICK SUMMARY

- `abstract = True`: shared fields, no table
- Multi-table: subclass with its own table
- `proxy = True`: same table, new behavior
- `%(class)s`: unique related names

Migrations — Basics

VERSION CONTROL FOR THE SCHEMA

Migrations are Python files that record changes to your models so the database schema can follow along. `makemigrations` compares models to the last migration state and writes a new file; `migrate` applies unapplied migrations in dependency order and records them in the `django_migrations` table. Commit migration files to git with the code that needs them.

NOTES

- Each app has its own `migrations/` folder, numbered `0001_initial.py`, `0002_...`
- Renames are detected interactively ("Did you rename...?")
- Adding a non-null field to existing rows needs a default
- Migrations run inside a transaction on PostgreSQL and SQLite
- Never edit a migration that's already applied elsewhere

EVERYDAY WORKFLOW

```
python manage.py makemigrations                 # all apps
python manage.py makemigrations blog --name add_status
python manage.py makemigrations --check --dry-run   # CI: fail if missing
python manage.py migrate                        # apply everything
python manage.py migrate blog 0003              # go to a specific state
python manage.py migrate blog zero              # unapply all of blog
python manage.py showmigrations blog            # [X] applied, [ ] not
python manage.py sqlmigrate blog 0004           # print the SQL
python manage.py migrate blog 0004 --fake       # mark applied, no SQL
```

A GENERATED MIGRATION

```
class Migration(migrations.Migration):
    dependencies = [("blog", "0003_post_slug")]
    operations = [
        migrations.AddField(model_name="post", name="status",
            field=models.CharField(default="D", max_length=1)),
    ]
```

CODE EXPLAINED

- → `--name` gives the file a readable name
- → `--check` exits non-zero if models and migrations differ
- → Migrating to an earlier number runs reverse operations
- → `sqlmigrate` is great for reviewing risky changes
- → `--fake` only updates `django_migrations`; use with care

COMMANDS

COMMAND	DOES
`makemigrations`	Write new files
`migrate`	Apply / unapply
`showmigrations`	Status list
`sqlmigrate`	Show SQL
`optimizemigration`	Simplify one file

COMMON OPERATIONS

OPERATION	FROM
`CreateModel`	New model
`AddField`	New field
`AlterField`	Changed field
`RenameField`	Renamed field
`RemoveField`	Deleted field
`AddConstraint`	New constraint

USEFUL FLAGS

FLAG	USE
`--plan`	Preview migrate
`--fake-initial`	Adopt existing tables
`--empty`	Blank file
`--merge`	Fix conflicts
`--database`	Pick a DB

⚠ COMMON MISTAKES

- Not committing migration files, so teammates generate different ones
- Deleting migrations to "start fresh" on a database that already applied them
- Running `--fake` to silence an error, leaving the schema and history out of sync

✓ TIPS & BEST PRACTICES

- Run `makemigrations --check` in CI to catch forgotten migrations
- Review `sqlmigrate` output before deploying changes to large tables
- Keep schema changes and data changes in separate migrations

QUICK SUMMARY

- `makemigrations`: models → migration file
- `migrate`: file → database
- `showmigrations` / `sqlmigrate`: inspect
- `migrate app zero`: roll back an app

Migrations — Data & Advanced

BEYOND AUTO-GENERATED MIGRATIONS

Data migrations change rows rather than schema, for example filling a new column from existing data. Create an empty migration and add `RunPython` (or `RunSQL`) with a forward and reverse function. Inside, always load models with `apps.get_model()` so you get the **historical** version matching that point in time. Over time, `squashmigrations` folds long histories into one file.

NOTES

- Historical models have fields but **no custom methods**
- Give a reverse function, or `RunPython.noop`, so rollbacks work
- Two developers adding `0005_*` creates a conflict; fix with `--merge`
- `atomic = False` lets big data migrations commit in batches
- `run_before` forces ordering against another app

A DATA MIGRATION

```python
# python manage.py makemigrations blog --empty --name fill_slugs
from django.db import migrations
from django.utils.text import slugify

def fill_slugs(apps, schema_editor):
    Post = apps.get_model("blog", "Post")      # historical model
    for post in Post.objects.filter(slug="").iterator():
        post.slug = slugify(post.title)
        post.save(update_fields=["slug"])

def clear_slugs(apps, schema_editor):
    apps.get_model("blog", "Post").objects.update(slug="")

class Migration(migrations.Migration):
    dependencies = [("blog", "0004_post_slug")]
    operations = [
        migrations.RunPython(fill_slugs, clear_slugs),
        migrations.RunSQL("UPDATE blog_post SET status = 'D' WHERE status = '';",
                          reverse_sql=migrations.RunSQL.noop),
    ]
```

CODE EXPLAINED

- → `apps.get_model()` returns the model as of this migration
- → `.iterator()` streams rows instead of loading them all
- → The second function makes the migration reversible
- → `RunSQL` runs raw SQL with an optional `reverse_sql`
- → Set `atomic = False` for SQL that can't run in a transaction

HOUSEKEEPING

```
python manage.py makemigrations --merge        # resolve 0005 conflict
python manage.py squashmigrations blog 0001 0020
```

SPECIAL OPERATIONS

OPERATION	USE
`RunPython`	Python data changes
`RunSQL`	Raw SQL
`SeparateDatabaseAndState`	Split DB vs state
`AddIndexConcurrently`	Postgres, no lock

ZERO-DOWNTIME STEPS

STEP	DEPLOY
1	Add nullable column
2	Backfill (data migration)
3	Make it NOT NULL
4	Remove old column later

SQUASHING

STEP	ACTION
`squash`	Creates `0001_squashed_…`
`deploy`	Both histories work
`clean up`	Delete old files
`finish`	Remove `replaces`

⚠ COMMON MISTAKES

- Importing models directly (`from blog.models import Post`) in a migration; it breaks once the model changes
- Calling custom model methods in `RunPython`; historical models don't have them
- No reverse function, so `migrate blog 0003` fails with `IrreversibleError`

✓ TIPS & BEST PRACTICES

- Process large tables in batches with `iterator(chunk_size=2000)`
- Test migrations both ways: `migrate` then `migrate app <previous>`
- Use `RunPython.noop` when the reverse truly has nothing to undo

QUICK SUMMARY

- `--empty`: start a data migration
- `RunPython(forward, reverse)`: data change
- `apps.get_model()`: historical model
- `squashmigrations` / `--merge`: housekeeping

QuerySet Basics

QUERYING WITH QUERYSETS

A **QuerySet** is a lazy, chainable description of a database query. Methods like `filter()` and `order_by()` return new QuerySets without touching the database; SQL only runs when you **evaluate** it by iterating, slicing with a step, calling `len()`, `list()` or `bool()`, or printing it. Evaluated results are cached on that QuerySet object.

NOTES

- `get()` returns one object or raises `DoesNotExist` / `MultipleObjectsReturned`
- `first()` / `last()` return `None` instead of raising
- Slicing `[:10]` becomes `LIMIT 10`; negative indexes aren't supported
- Chain freely: each call adds to the same SQL
- Async variants exist: `aget()`, `acreate()`, `async for`

CORE QUERYSET CALLS

```
Post.objects.all()                              # every row (lazy)
Post.objects.filter(published=True)             # WHERE published
Post.objects.exclude(status="D")                # WHERE NOT status = 'D'
Post.objects.filter(published=True).order_by("-created", "title")
Post.objects.get(pk=1)                          # one object or exception
Post.objects.filter(title="Hi").first()         # object or None
Post.objects.order_by("-created")[:5]           # LIMIT 5
Post.objects.all()[10:20]                       # OFFSET 10 LIMIT 10
Post.objects.create(title="New")                # INSERT, returns obj
Post.objects.get_or_create(slug="hi", defaults={"title": "Hi"})  # (obj, created)
Post.objects.filter(status="D").update(status="P")   # bulk UPDATE -> count
Post.objects.filter(old=True).delete()          # bulk DELETE -> (n, {...})
Post.objects.count()                            # SELECT COUNT(*)
Post.objects.filter(pk=1).exists()              # cheap existence test
Post.objects.values("id", "title")              # dicts
Post.objects.values_list("title", flat=True)    # ['Hi', 'New']
print(Post.objects.filter(pk=1).query)          # the SQL
```

CODE EXPLAINED

→ Nothing above hits the DB until evaluated (except create/get/update/delete/count/exists)

→ `"-created"` sorts descending

→ `get_or_create` returns a tuple `(object, created_bool)`

→ `update()` and `delete()` skip `save()` / `delete()` methods and signals on update

→ `.query` shows SQL without running it

RETURN QUERYSETS

METHOD	DOES
`filter / exclude`	Narrow rows
`order_by`	Sort
`distinct`	Remove dupes
`values / values_list`	Dicts / tuples
`none()`	Empty QuerySet
`reverse()`	Flip order

RETURN VALUES

METHOD	RETURNS
`get()`	Object
`first() / last()`	Object or None
`count()`	int
`exists()`	bool
`latest("f")`	Newest by field
`in_bulk([ids])`	dict id → obj

WHEN SQL RUNS

ACTION	EVALUATES?
`for x in qs`	Yes
`list(qs), len(qs)`	Yes
`if qs:`	Yes
`qs[:5]`	No (still lazy)
`qs[0]`	Yes (one row)

⚠ COMMON MISTAKES

- `if qs:` followed by `len(qs)` on big tables; use `exists()` / `count()` when you don't need rows
- Calling `get()` with a filter that may match several rows: `MultipleObjectsReturned`
- Expecting `update()` to call `save()` or `auto_now`; it writes SQL directly

✓ TIPS & BEST PRACTICES

- Reuse an evaluated QuerySet variable to benefit from its cache
- Use `values_list("id", flat=True)` when you only need ids
- Print `qs.query` or use `connection.queries` to see the SQL

QUICK SUMMARY

- QuerySets are lazy and chainable
- `filter()` / `exclude()` / `order_by()`
- `get()` raises; `first()` returns None
- `update()` / `delete()`: bulk writes

Field Lookups

DOUBLE-UNDERSCORE LOOKUPS

Filter keyword arguments use `field__lookup=value`. With no lookup, `exact` is assumed. Lookups cover text matching, comparisons, membership, ranges, NULL checks and date parts. The same double underscore also **follows relationships**: `author__username` joins to the author table, and you can chain as deep as you need.

NOTES

- Multiple kwargs in one `filter()` are combined with AND
- `i` prefix = case-insensitive (`iexact` , `icontains`)
- Date parts chain with comparisons: `created__year__gte=2025`
- Following a reverse relation may return duplicates; add `.distinct()`
- Use `Q` objects for OR logic (Sheet 34)

LOOKUP EXAMPLES

```
Post.objects.filter(title="Django")                    # exact
Post.objects.filter(title__iexact="django")            # ILIKE 'django'
Post.objects.filter(title__icontains="orm")            # ILIKE '%orm%'
Post.objects.filter(slug__startswith="how-to")
Post.objects.filter(views__gt=100, views__lte=1000)
Post.objects.filter(id__in=[1, 4, 9])
Post.objects.filter(created__range=(start, end))       # BETWEEN
Post.objects.filter(pub_date__isnull=True)
Post.objects.filter(created__year=2026, created__month=10)
Post.objects.filter(created__date=date.today())
Post.objects.filter(created__week_day=2)               # Monday
Post.objects.filter(title__regex=r"^\d+ tips")

# spanning relationships
Post.objects.filter(author__username="ann")            # FK join
Post.objects.filter(author__profile__country="US")     # two joins
Author.objects.filter(post__title__icontains="orm").distinct()  # reverse
Post.objects.filter(tags__name__in=["web", "api"]).distinct()   # M2M
Post.objects.filter(specs__color="red")                # JSONField key
```

CODE EXPLAINED

- → `views__gt=100` → `WHERE views > 100`
- → `__in` accepts a list or another QuerySet (subquery)
- → `__range` is inclusive on both ends
- → Reverse FK lookups use the `related_query_name` (default: model name)
- → JSONField keys chain like relations: `specs__color`

TEXT

LOOKUP	SQL-ISH
`exact` / `iexact`	= / ILIKE
`contains` / `icontains`	LIKE '%x%'
`startswith` / `endswith`	LIKE 'x%'
`regex` / `iregex`	Regex match

COMPARE & SETS

LOOKUP	MEANING
`gt` / `gte`	> / >=
`lt` / `lte`	< / <=
`in`	IN (…)
`range`	BETWEEN
`isnull`	IS NULL

DATE PARTS

LOOKUP	MATCHES
`date`	Date of a datetime
`year` / `month` / `day`	Parts
`week` / `week_day`	ISO week / 1=Sun
`quarter`	1–4
`hour` / `minute`	Time parts

⚠ COMMON MISTAKES

- Using `contains` when you meant case-insensitive search; use `icontains`
- `filter(tags__name="a", tags__name="b")` : duplicate kwargs; chain two `filter()` calls
- Forgetting `.distinct()` after reverse or M2M lookups, so the same row repeats

✓ TIPS & BEST PRACTICES

- Use `__date` on datetimes rather than building start/end ranges by hand
- Index columns you filter by `startswith` or `exact` often
- For real full-text search use `django.contrib.postgres.search`

QUICK SUMMARY

- `field__lookup=value` : filter syntax
- `icontains` / `in` / `range` / `isnull`
- `created__year=` : date parts
- `fk__field=` : follow relations

Q & F Expressions

COMPLEX CONDITIONS AND COLUMN REFERENCES

Q objects wrap lookups so they can be combined with | (OR), & (AND) and ~ (NOT). F expressions refer to **another column's value** inside the database, so you can compare two fields or update a counter without reading it into Python first, which also avoids race conditions. Case/When builds SQL CASE expressions.

NOTES

- `Q` objects must come **before** keyword arguments in `filter()`
- `F("views") + 1` runs as SQL: `views = views + 1`
- After an `F()` update, call `refresh_from_db()` to see the new value
- `Value()` wraps a literal for use in expressions
- Expressions also work in `order_by()` and `annotate()`

Q, F, CASE/WHEN

```
from django.db.models import Q, F, Value, Case, When, CharField
from django.db.models.functions import Concat

# OR / NOT
Post.objects.filter(Q(status="P") | Q(author=request.user))
Post.objects.filter(~Q(status="D"), views__gt=10)   # Q first
q = Q()
if term:  q &= Q(title__icontains=term)              # build dynamically
if tag:   q &= Q(tags__name=tag)
Post.objects.filter(q)

# compare two columns
Product.objects.filter(stock__lt=F("reorder_level"))
Event.objects.filter(end__gt=F("start") + timedelta(hours=1))

# atomic update, no race condition
Post.objects.filter(pk=pk).update(views=F("views") + 1)
post.views = F("views") + 1; post.save(update_fields=["views"])

# CASE WHEN
Order.objects.annotate(size=Case(
    When(total__gte=500, then=Value("large")),
    When(total__gte=100, then=Value("medium")),
    default=Value("small"), output_field=CharField()))
User.objects.annotate(full=Concat("first_name", Value(" "), "last_name"))
Post.objects.order_by(F("pub_date").desc(nulls_last=True))
```

CODE EXPLAINED

- → `Q() &= ...` lets you add conditions only when inputs exist
- → `F("start") + timedelta` does date math in SQL
- → `update(views=F(...) + 1)` is safe under concurrent requests
- → `Case/When` evaluates conditions top to bottom
- → `nulls_last=True` controls where NULLs sort

Q OPERATORS

OPERATOR	SQL
`Q(a) | Q(b)`	a OR b
`Q(a) & Q(b)`	a AND b
`Q(a)`	NOT a
`Q(a) ^ Q(b)`	XOR

F USES

USE	EXAMPLE
Compare	`filter(a__gt=F("b"))`
Update	`update(n=F("n")+1)`
Annotate	`annotate(t=F("a")*F("b"))`
Order	`F("x").asc()`
Span	`F("author__name")`

HANDY FUNCTIONS

FUNCTION	DOES
Coalesce	First non-null
Concat	Join strings
Lower / Upper	Case
Now	DB time
Greatest / Least	Max / min of args

⚠ COMMON MISTAKES

- Putting a keyword arg before a `Q` in `filter()`: `SyntaxError`
- `post.views += 1; post.save()` under load; two requests lose an increment, use `F()`
- Reading `post.views` right after an `F()` save; it's an expression until refreshed

✓ TIPS & BEST PRACTICES

- Build search filters with `q = Q()` and `&=` / `|=`
- Use `Coalesce("nickname", "username")` for fallbacks in SQL
- Mix `F()` with `ExpressionWrapper` when the output type differs

QUICK SUMMARY

- `Q(...) | Q(...)`: OR conditions
- `~Q(...)`: NOT
- `F("field")`: column reference in SQL
- `Case(When(...))`: conditional values

Aggregation & Annotation

SUMMARIES AND COMPUTED COLUMNS

`aggregate()` returns a **dict** of totals over the whole QuerySet. `annotate()` adds a computed value to **each row**, often a count or sum of related objects. Put `values()` before `annotate()` to group by those fields (SQL `GROUP BY`). `Subquery` and `OuterRef` embed a correlated query, such as each author's latest post title.

NOTES

- Annotations can be filtered and ordered like fields
- `Count("x", distinct=True)` avoids double counting across joins
- `filter=Q(...)` on an aggregate counts only matching rows
- Default ordering can break `GROUP BY`; clear it with `order_by()`
- 6.0: `StringAgg` works on every database; `AnyValue` is new

AGGREGATES IN PRACTICE

```
from django.db.models import Count, Sum, Avg, Max, Q, OuterRef, Subquery

Order.objects.aggregate(total=Sum("amount"), avg=Avg("amount"))
# {'total': Decimal('9120.50'), 'avg': Decimal('45.60')}

authors = Author.objects.annotate(
    n_posts=Count("post"),
    n_live=Count("post", filter=Q(post__status="P")),
).filter(n_posts__gte=5).order_by("-n_posts")
authors[0].n_posts                                  # 42

# GROUP BY status
Post.objects.values("status").annotate(n=Count("id")).order_by()
# <QuerySet [{'status': 'D', 'n': 7}, {'status': 'P', 'n': 30}]>

# GROUP BY month
from django.db.models.functions import TruncMonth
Order.objects.annotate(m=TruncMonth("created")).values("m") \
     .annotate(total=Sum("amount")).order_by("m")

# correlated subquery: newest post title per author
newest = Post.objects.filter(author=OuterRef("pk")).order_by("-created")
Author.objects.annotate(latest=Subquery(newest.values("title")[:1]))
```

CODE EXPLAINED

- → `aggregate` keys are the names you give (`total` , `avg`)
- → `filter=Q(...)` → `COUNT(...) FILTER (WHERE ...)`
- → Filtering on an annotation becomes `HAVING`
- → Empty `order_by()` stops `Meta.ordering` adding GROUP BY columns
- → `OuterRef("pk")` points at the outer query's row

AGGREGATE FUNCTIONS

FUNCTION	RETURNS
`Count`	int
`Sum / Avg`	number
`Max / Min`	value
`StdDev / Variance`	float
`StringAgg`	joined text (6.0)

AGGREGATE VS ANNOTATE

	RESULT
`aggregate()`	One dict
`annotate()`	Per-row value
`values().annotate()`	Per group
`annotate().filter()`	HAVING

DATE TRUNCATION

FUNCTION	GROUPS BY
`TruncDate`	Day
`TruncWeek`	Week
`TruncMonth`	Month
`TruncYear`	Year
`ExtractHour`	Hour number

⚠ COMMON MISTAKES

- Two `Count` s over different joins multiply each other; add `distinct=True` or use subqueries
- `values().annotate()` with model `ordering` set, producing one group per row
- `aggregate(Sum(...))` on no rows returns `None` ; use `default=0` (4.0+)

✓ TIPS & BEST PRACTICES

- Use `Sum("amount", default=0)` to avoid `None` checks
- Annotate counts in list views instead of calling `.count()` per row
- Check heavy queries with `qs.explain(analyze=True)` on PostgreSQL

QUICK SUMMARY

- `aggregate()` : dict of totals
- `annotate()` : per-row computed value
- `values(...).annotate(...)` : GROUP BY
- `Subquery(OuterRef)` : per-row subquery

Related Objects & Query Optimization

AVOIDING THE N+1 PROBLEM

Accessing a related object in a loop runs **one extra query per row**: the N+1 problem. `select_related()` follows ForeignKey and one-to-one links with a SQL JOIN in the same query. `prefetch_related()` fetches many-to-many and reverse relations in **one extra query** and joins them in Python. `only()` and `defer()` limit which columns load. 6.1 adds **fetch modes** that can fix N+1 automatically.

NOTES

- `select_related`: FK / 1:1 forward (JOIN)
- `prefetch_related`: M2M and reverse FK (separate query)
- `Prefetch()` customizes the prefetched QuerySet
- Calling `.filter()` on a prefetched relation ignores the prefetch
- 6.1: `fetch_mode(models.FETCH_PEERS)` batch-loads on first access

BEFORE AND AFTER

```python
# N+1: 1 query for posts + 1 per post for author
for post in Post.objects.all():
    print(post.author.name)

# 1 query with a JOIN
for post in Post.objects.select_related("author", "author__profile"):
    print(post.author.name)

# 2 queries total: posts, then all their tags
posts = Post.objects.prefetch_related("tags")
for p in posts:
    print([t.name for t in p.tags.all()])

from django.db.models import Prefetch
Author.objects.prefetch_related(Prefetch("post_set",
    queryset=Post.objects.filter(status="P").order_by("-created"),
    to_attr="live_posts"))                # author.live_posts is a list

Post.objects.only("id", "title")          # load just these columns
Post.objects.defer("body")                # load all but body

from django.db import models               # Django 6.1
Book.objects.fetch_mode(models.FETCH_PEERS)   # auto batch-load
Book.objects.fetch_mode(models.FETCH_RAISE)   # error on lazy load
```

CODE EXPLAINED

- → `author__profile` chains joins through two FKs
- → Prefetch results are cached on each object
- → `to_attr` stores filtered results as a plain list
- → Accessing a deferred field later triggers another query
- → `FETCH_RAISE` is useful in tests to catch N+1

WHICH ONE

RELATION	USE
ForeignKey (forward)	select_related
OneToOne	select_related
Reverse FK	prefetch_related
ManyToMany	prefetch_related
Generic FK	prefetch_related

FETCH MODES (6.1)

MODE	ON LAZY ACCESS
FETCH_ONE	One query (default)
FETCH_PEERS	Load for all peers
FETCH_RAISE	Raise an error

FINDING SLOW QUERIES

TOOL	SHOWS
Debug Toolbar	Queries per page
connection.queries	SQL list (DEBUG)
assertNumQueries	Test guard
qs.explain()	Query plan

⚠ COMMON MISTAKES

- `select_related("tags")` on an M2M: `FieldError`; use `prefetch_related`
- `p.tags.filter(...)` inside the loop, which ignores the prefetch and queries again
- Using `only()` then touching other fields in templates, causing hidden extra queries

✓ TIPS & BEST PRACTICES

- Add `select_related` in list views' `get_queryset()` by default
- Guard key views with `assertNumQueries` in tests
- Use `exists()` / `count()` instead of loading rows just to check

QUICK SUMMARY

- `select_related()`: JOIN for FK/1:1
- `prefetch_related()`: extra query for M2M/reverse
- `only()` / `defer()`: fewer columns
- `fetch_mode()` (6.1): auto N+1 handling

Bulk Ops, Raw SQL & Transactions

WRITING EFFICIENTLY AND SAFELY

Bulk methods write many rows in a few queries instead of one per object. When the ORM can't express a query, `raw()` maps SQL results back to models and `connection.cursor()` runs any SQL. `transaction.atomic()` groups writes so they all succeed or all roll back; use it for anything that must stay consistent, like moving money between accounts.

NOTES

- `bulk_create` skips `save()` and signals
- `update_conflicts=True` turns `bulk_create` into an upsert
- By default each query autocommits; `atomic` opens a transaction
- Nested `atomic` blocks create savepoints
- `on_commit()` runs code only after a successful commit

BULK, RAW AND ATOMIC

```python
from django.db import connection, transaction

Tag.objects.bulk_create([Tag(name=n) for n in names],
                        batch_size=500, ignore_conflicts=True)
Product.objects.bulk_create(rows, update_conflicts=True,
    unique_fields=["sku"], update_fields=["price"])   # upsert
for p in products: p.price *= 1.1
Product.objects.bulk_update(products, ["price"], batch_size=500)
obj, created = Product.objects.update_or_create(
    sku="A1", defaults={"price": 9.99}, create_defaults={"stock": 0})

Post.objects.raw("SELECT * FROM blog_post WHERE views > %s", [100])
with connection.cursor() as cur:
    cur.execute("UPDATE blog_post SET views = 0 WHERE id = %s", [pk])
    cur.execute("SELECT COUNT(*) FROM blog_post")
    (count,) = cur.fetchone()

@transaction.atomic
def transfer(src, dst, amount):
    src = Account.objects.select_for_update().get(pk=src.pk)  # row lock
    src.balance -= amount; src.save()
    dst.balance += amount; dst.save()
    transaction.on_commit(lambda: send_receipt.enqueue(src.pk))

with transaction.atomic():                # block form
    order.save(); order.items.create(sku="A1")
```

CODE EXPLAINED

- → `ignore_conflicts=True` skips duplicates silently
- → `update_or_create` 5.0+: `create_defaults` only used on insert
- → Always pass parameters as a list (`%s`), never f-strings
- → `select_for_update()` locks rows until the transaction ends
- → An exception inside `atomic` rolls back everything in it

BULK METHODS

METHOD	SIGNALS?
`bulk_create`	No
`bulk_update`	No
`qs.update()`	No
`qs.delete()`	Yes (per object)
`in_bulk()`	Read, dict

TRANSACTION TOOLS

TOOL	USE
`atomic()`	Decorator / block
`on_commit(fn)`	After commit
`select_for_update()`	Row lock
`ATOMIC_REQUESTS`	Wrap every view
`set_rollback(True)`	Force rollback

RAW SQL OPTIONS

OPTION	RETURNS
`raw(sql, params)`	Model instances
`cursor.execute`	Rows (tuples)
`RawSQL()`	Expression in ORM
`extra()`	Legacy, avoid

⚠ COMMON MISTAKES

- Building SQL with f-strings: SQL injection; pass `params` instead
- Catching exceptions **inside** `atomic` and continuing; the transaction is broken
- Sending emails inside a transaction that later rolls back; use `on_commit`

✓ TIPS & BEST PRACTICES

- Use `batch_size` for very large bulk writes to keep queries reasonable
- Enable `ATOMIC_REQUESTS` for simple apps that want one transaction per request
- Wrap data imports in one `atomic()` so a bad row leaves nothing half-done

QUICK SUMMARY

- `bulk_create` / `bulk_update` : many rows, few queries
- `update_or_create` : upsert one row
- `raw()` / `cursor()` : SQL with params
- `transaction.atomic()` : all or nothing

Form Basics

HANDLING USER INPUT

A `forms.Form` class declares fields once and gives you HTML rendering, validation and cleaned Python values. A form is **unbound** when created empty (for display) and **bound** when given `request.POST`. Call `is_valid()` to run validation; valid data is then in `form.cleaned_data`, already converted to the right types. Invalid forms re-render with their errors.

NOTES

- `cleaned_data` only exists after `is_valid()` returns
- Errors are in `form.errors` (a dict of field → messages)
- Always redirect after a successful POST (PRG pattern)
- Pass `request.FILES` as the second argument for uploads
- Forms work without models; use `ModelForm` for model data (Sheet 41)

FORMS.PY + VIEW

```python
from django import forms

class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    message = forms.CharField(widget=forms.Textarea)
    subscribe = forms.BooleanField(required=False)

def contact(request):
    if request.method == "POST":
        form = ContactForm(request.POST)            # bound
        if form.is_valid():
            data = form.cleaned_data                # {'name': 'Ann', ...}
            send_message(data["email"], data["message"])
            return redirect("contact-thanks")       # PRG
    else:
        form = ContactForm(initial={"subscribe": True})   # unbound
    return render(request, "contact.html", {"form": form})
```

CODE EXPLAINED

- → `required=False` makes a field optional (default is required)
- → `initial=` sets display values on an unbound form
- → Invalid POST falls through to `render()` with errors attached
- → `{{ form }}` renders with the default `div` template (5.0+)
- → `novalidate` turns off browser checks so you can test server validation

CONTACT.HTML

```html
<form method="post" novalidate>
  {% csrf_token %}
  {{ form }}                              {# fields + labels + errors #}
  <button type="submit">Send</button>
</form>
```

FORM API

MEMBER	GIVES
`is_bound`	Has data?
`is_valid()`	Runs validation
`cleaned_data`	Typed values
`errors`	Field → messages
`has_changed()`	Differs from initial?
`non_field_errors()`	Form-level errors

RENDERING OPTIONS

SYNTAX	OUTPUT
`{{ form }}`	Default (div)
`{{ form.as_div }}`	Wrapped in divs
`{{ form.as_p }}`	Paragraphs
`{{ form.name }}`	One widget
`{{ form.name.errors }}`	Its errors

BOUND FIELD PARTS

ATTRIBUTE	IS
`label_tag`	<label> element
`help_text`	Help string
`value()`	Current value
`id_for_label`	HTML id
`as_field_group`	Label+widget+errors

⚠ COMMON MISTAKES

- Reading `form.cleaned_data` before calling `is_valid()`: `AttributeError`
- Using `request.POST["email"]` directly, skipping validation and type conversion
- Creating a new unbound form after a failed POST, which throws the errors away

✓ TIPS & BEST PRACTICES

- Use `ContactForm(request.POST or None)` to collapse the GET/POST branches
- Render fields individually with `{{ form.name.as_field_group }}` for custom layouts
- Use `django-crispy-forms` or a custom `FORM_RENDERER` for styled forms

QUICK SUMMARY

- `forms.Form`: declare fields
- Bound = has data; unbound = empty
- `is_valid()` → `cleaned_data`
- `{{ form }}`: render + errors

Form Fields & Widgets

FIELDS VALIDATE, WIDGETS RENDER

Each **form field** validates and converts one input (string → `int`, `date`, `Decimal`…). Each field has a default **widget**, the class that renders its HTML. Swap widgets with `widget=` and add HTML attributes (CSS classes, placeholders) with `attrs`. Field arguments control labels, help text, initial values and whether the field is required.

NOTES

- All fields are `required=True` by default
- `ChoiceField` needs `choices`; `ModelChoiceField` uses a QuerySet
- `TypedChoiceField(coerce=int)` converts the chosen value
- Use `DateInput(attrs={"type": "date"})` for a native date picker
- Disabled fields ignore submitted data and use `initial`

FIELDS AND WIDGETS

```python
from django import forms

class SignupForm(forms.Form):
    username = forms.CharField(min_length=3, max_length=30,
        widget=forms.TextInput(attrs={"placeholder": "ann_dev",
                                      "class": "input"}))
    email = forms.EmailField(label="Email address",
                             help_text="We never share it.")
    password = forms.CharField(widget=forms.PasswordInput)
    age = forms.IntegerField(min_value=13, required=False)
    birthday = forms.DateField(widget=forms.DateInput(attrs={"type": "date"}))
    plan = forms.ChoiceField(choices=[("free", "Free"), ("pro", "Pro")],
                             widget=forms.RadioSelect, initial="free")
    topics = forms.MultipleChoiceField(choices=TOPICS,
                                       widget=forms.CheckboxSelectMultiple)
    country = forms.ModelChoiceField(queryset=Country.objects.all(),
                                     empty_label="Choose...")
    terms = forms.BooleanField(label="I accept the terms")

form.cleaned_data["age"]       # 30 (int) or None
form.cleaned_data["country"]   # <Country: Canada>
```

CODE EXPLAINED

- → `attrs` become HTML attributes on the rendered input
- → `PasswordInput` doesn't re-fill its value on error
- → `RadioSelect` / `CheckboxSelectMultiple` change the look, not the data
- → `ModelChoiceField` returns a model instance in `cleaned_data`
- → A required `BooleanField` must be checked to validate

COMMON FIELDS

FIELD	CLEANS TO
`CharField`	str
`IntegerField`	int
`DecimalField`	Decimal
`DateField`	date
`ChoiceField`	str
`FileField`	UploadedFile

WIDGETS

WIDGET	HTML
`TextInput`	text input
`Textarea`	<textarea>
`Select`	<select>
`CheckboxInput`	checkbox
`HiddenInput`	hidden
`ClearableFileInput`	file + clear

CORE ARGUMENTS

ARGUMENT	PURPOSE
`required`	Must be filled
`label`	Label text
`initial`	Starting value
`help_text`	Hint text
`disabled`	Read-only
`validators`	Extra checks

⚠ COMMON MISTAKES

- Expecting `ChoiceField` to return an int; it returns a string unless you use `TypedChoiceField`
- Setting `widget=forms.Textarea()` on an `IntegerField`; widget changes don't change cleaning
- Building a `ModelChoiceField` queryset at import time with user-specific filters; set it in `__init__`

✓ TIPS & BEST PRACTICES

- Set queryset per request: `self.fields["country"].queryset = ...` in `__init__`
- Add CSS classes for all fields in a loop over `self.fields` in `__init__`
- Use `forms.URLField` / `EmailField` rather than regex on `CharField`

QUICK SUMMARY

- Field = validation + type
- Widget = HTML rendering
- `attrs={...}`: HTML attributes
- `ModelChoiceField(queryset=)`: pick a row

Form Validation

VALIDATION ORDER

Validation runs in layers when you call `is_valid()`. First each field converts and checks its own value, plus any `validators`. Then each `clean_<fieldname>()` method runs for single-field rules and must **return** the cleaned value. Finally `clean()` runs for rules that involve several fields. Raise `ValidationError` to reject input, or call `add_error()` to attach a message to a specific field.

NOTES

- `clean_<field>` only runs if that field passed its own checks
- In `clean()`, fields that failed earlier are missing from `cleaned_data`
- Errors raised in `clean()` become **non-field errors**
- Give errors a `code` so tests and templates can check them
- Reusable validators are plain functions that raise `ValidationError`

FIELD AND FORM VALIDATION

```python
from django import forms
from django.core.exceptions import ValidationError

def no_profanity(value):                                  # reusable validator
    if "darn" in value.lower():
        raise ValidationError("Please keep it clean.", code="profanity")

class EventForm(forms.Form):
    title = forms.CharField(validators=[no_profanity])
    start = forms.DateField()
    end = forms.DateField()

    def clean_title(self):                                # one field
        title = self.cleaned_data["title"].strip()
        if Event.objects.filter(title__iexact=title).exists():
            raise ValidationError("That title is taken.", code="duplicate")
        return title                                      # must return!

    def clean(self):                                      # several fields
        cleaned = super().clean()
        start, end = cleaned.get("start"), cleaned.get("end")
        if start and end and end < start:
            self.add_error("end", "End must be after start.")
        return cleaned
```

CODE EXPLAINED

→ Validators run during field cleaning, before `clean_<field>`

→ Use `.get()` in `clean()`: earlier failures remove keys

→ `add_error("end", ...)` attaches to the `end` field

→ `add_error(None, ...)` adds a non-field error

→ Returning a modified value (`strip()`) changes `cleaned_data`

ORDER OF CHECKS

STEP	METHOD
1	`field.to_python()`
2	`field.validate()`
3	`validators`
4	`clean_<field>()`
5	`clean()`

BUILT-IN VALIDATORS

VALIDATOR	CHECKS
`MinValueValidator`	≥ n
`MaxLengthValidator`	len ≤ n
`RegexValidator`	Pattern
`EmailValidator`	Email
`FileExtensionValidator`	Extensions

ERROR ACCESS

CALL	RETURNS
`form.errors`	ErrorDict
`{{ form.non_field_errors }}`	In templates
`form.has_error("f")`	bool
`form.add_error(f, msg)`	Attach error
`field.errors`	Field's list

⚠ COMMON MISTAKES

- Forgetting `return` in `clean_<field>()`, which sets the value to `None`
- `self.cleaned_data["start"]` in `clean()` when start failed: `KeyError`
- Raising in `clean()` for a single-field problem; the error shows at the top, not by the field

✓ TIPS & BEST PRACTICES

- Put rules that also apply outside forms on the model's `clean()` instead
- Pass `params` to `ValidationError` for translatable messages: `"%(value)s is taken"`
- Test validation by asserting on `form.errors` and `form.has_error("f", code=...)`

QUICK SUMMARY

- `validators=[...]`: reusable checks
- `clean_<field>()`: one field, return value
- `clean()`: cross-field rules
- `add_error(field, msg)`: targeted errors

ModelForm

FORMS BUILT FROM MODELS

A `ModelForm` builds its fields from a model, so field types, `max_length`, `choices`, `blank` and `help_text` carry over automatically. Its `save()` creates or updates the model instance. Pass `instance=obj` to edit an existing row. Use `save(commit=False)` to get the unsaved object, set fields the user shouldn't control, then save it yourself.

NOTES

- List `fields` explicitly; avoid `"__all__"` and `exclude`
- Model validation (`clean()` , unique, constraints) runs in `is_valid()`
- After `commit=False` , call `form.save_m2m()` for many-to-many data
- Override labels, widgets and help text in `Meta`
- Add extra non-model fields like any normal form field

MODELFORM CREATE AND EDIT

```
from django import forms
from .models import Post

class PostForm(forms.ModelForm):
    notify = forms.BooleanField(required=False)          # extra field
    class Meta:
        model = Post
        fields = ["title", "body", "tags", "pub_date"]
        labels = {"body": "Content"}
        widgets = {"pub_date": forms.DateInput(attrs={"type": "date"})}
        help_texts = {"tags": "Pick up to five."}

def post_edit(request, pk=None):
    post = get_object_or_404(Post, pk=pk, author=request.user) if pk else None
    form = PostForm(request.POST or None, instance=post)
    if form.is_valid():
        post = form.save(commit=False)                   # not in DB yet
        post.author = request.user
        post.save()
        form.save_m2m()                                  # tags
        if form.cleaned_data["notify"]:
            notify_followers(post)
        return redirect(post)
    return render(request, "blog/post_form.html", {"form": form})
```

CODE EXPLAINED

- → `instance=None` creates; `instance=post` edits
- → `commit=False` returns the object without saving
- → `save_m2m()` is only needed after `commit=False`
- → `widgets` / `labels` dicts override per field
- → Filtering the lookup by `author` stops editing others' posts

META OPTIONS

OPTION	PURPOSE
`model`	Source model
`fields`	Which fields
`exclude`	Which to skip
`widgets`	Widget overrides
`labels / help_texts`	Text overrides
`error_messages`	Custom messages

MODEL → FORM FIELD

MODEL FIELD	FORM FIELD
`CharField`	CharField
`TextField`	CharField + Textarea
`ForeignKey`	ModelChoiceField
`ManyToManyField`	ModelMultipleChoiceField
`BooleanField`	BooleanField

SAVE() PATHS

CALL	RESULT
`save()`	Saves + M2M
`save(commit=False)`	Unsaved obj
`…then obj.save()`	Row saved
`…then save_m2m()`	M2M saved

⚠ COMMON MISTAKES

- `fields = "__all__"` ; a later model field like `is_admin` becomes editable
- Using `commit=False` and forgetting `save_m2m()` , so tags are never saved
- Setting `author` from a hidden input; users can change it, set it in the view

✓ TIPS & BEST PRACTICES

- Use `modelform_factory(Post, fields=[...])` for quick one-off forms
- Override `__init__(self, *args, user=None, **kwargs)` to pass the user in
- Reuse one ModelForm for create and update by passing `instance`

QUICK SUMMARY

- `ModelForm` : fields from a model
- `Meta.fields` : explicit whitelist
- `instance=` : edit existing row
- `save(commit=False)` : set extra fields first

Formsets

MANY FORMS ON ONE PAGE

A **formset** manages several copies of the same form, such as rows of order items. Factories build the formset class: `formset_factory` for plain forms, `modelformset_factory` for a QuerySet and `inlineformset_factory` for children of one parent object. Every formset renders a hidden **management form** that tells Django how many forms were submitted.

NOTES

- `extra` = number of blank forms added for new rows
- `can_delete=True` adds a DELETE checkbox to each form
- `min_num` / `max_num` with `validate_min` / `validate_max` enforce limits
- Formset-level checks go in a `BaseFormSet.clean()` override
- Adding rows in JavaScript means cloning the empty form and bumping `TOTAL_FORMS`

INLINE FORMSET FOR ORDER ITEMS

```python
from django.forms import inlineformset_factory

ItemFormSet = inlineformset_factory(
    Order, OrderItem,                    # parent, child
    fields=["product", "quantity"],
    extra=2, can_delete=True, min_num=1, validate_min=True,
)

def order_edit(request, pk):
    order = get_object_or_404(Order, pk=pk)
    form = OrderForm(request.POST or None, instance=order)
    formset = ItemFormSet(request.POST or None, instance=order)
    if form.is_valid() and formset.is_valid():
        with transaction.atomic():
            form.save()
            formset.save()               # creates, updates, deletes rows
        return redirect(order)
    return render(request, "orders/edit.html",
                  {"form": form, "formset": formset})
```

CODE EXPLAINED

- → The inline formset sets the FK to `order` for you
- → `min_num=1, validate_min=True` requires at least one item
- → Validate both form and formset before saving either
- → `formset.save()` handles new, changed and deleted rows
- → `{{ formset.empty_form }}` gives a blank row (`__prefix__` index) for JS

TEMPLATE

```html
<form method="post">{% csrf_token %}
  {{ form }}
  {{ formset.management_form }}
  {% for f in formset %}<div class="item">{{ f }}</div>{% endfor %}
  <button>Save</button>
</form>
```

FACTORIES

FACTORY	FOR
`formset_factory`	Plain Form
`modelformset_factory`	A QuerySet
`inlineformset_factory`	Children of a parent

FACTORY ARGUMENTS

ARGUMENT	EFFECT
`extra`	Blank forms
`can_delete`	DELETE box
`can_order`	ORDER field
`min_num / max_num`	Limits
`formset`	Custom base class

MANAGEMENT FORM

FIELD	HOLDS
`TOTAL_FORMS`	Forms submitted
`INITIAL_FORMS`	Existing rows
`MIN_NUM_FORMS`	Minimum
`MAX_NUM_FORMS`	Maximum

⚠ COMMON MISTAKES

- Leaving out `{{ formset.management_form }}`: "ManagementForm data is missing"
- Saving the parent form before checking `formset.is_valid()`, leaving partial data
- Adding forms in JS without updating `TOTAL_FORMS`, so new rows are ignored

✓ TIPS & BEST PRACTICES

- Wrap parent + formset saves in `transaction.atomic()`
- Use `prefix=` when a page has more than one formset
- For dynamic rows, consider htmx partials instead of hand-written JS

QUICK SUMMARY

- `formset_factory(Form, extra=)`: many forms
- `inlineformset_factory(Parent, Child)`: children
- `management_form`: required hidden fields
- `formset.save()`: create/update/delete

File & Image Uploads

ACCEPTING USER FILES

Uploaded files arrive in `request.FILES`, but only when the form uses `method="post"` and `enctype="multipart/form-data"`. A model `FileField` or `ImageField` stores the **path** in the database and saves the file through the storage backend under `MEDIA_ROOT`. `MEDIA_URL` is the public URL prefix; in development you add a URL pattern to serve those files.

NOTES

- Pass `request.FILES` as the second argument to the form
- `ImageField` checks the file really is an image (needs Pillow)
- `upload_to` can be a string with date codes or a function
- Deleting a model row does **not** delete its file
- Use `django-storages` to store files on S3 or similar

SETTINGS, MODEL, VIEW

```
# settings.py
MEDIA_URL = "media/"
MEDIA_ROOT = BASE_DIR / "media"

# models.py
def avatar_path(instance, filename):
    return f"avatars/user_{instance.user_id}/{filename}"

class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    avatar = models.ImageField(upload_to=avatar_path, blank=True)
    resume = models.FileField(upload_to="resumes/%Y/%m/", blank=True,
        validators=[FileExtensionValidator(["pdf"])])

# views.py
form = ProfileForm(request.POST, request.FILES, instance=profile)

profile.avatar.url        # '/media/avatars/user_7/me.png'
profile.resume.delete(save=True)   # remove file and clear field

# urls.py (development only)
from django.conf import settings
from django.conf.urls.static import static
urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

CODE EXPLAINED

- → `upload_to` function gets the instance and original filename
- → Duplicate names get a random suffix automatically
- → `.url` is for templates; `.path` only works on local storage
- → `FileExtensionValidator` checks the extension (not the contents)
- → `static()` helper only works when `DEBUG = True`

UPLOADEDFILE

ATTRIBUTE	VALUE
`name`	Original filename
`size`	Bytes
`content_type`	Browser-reported
`chunks()`	Iterate pieces
`read()`	Whole file

FIELDFILE

MEMBER	GIVES
`url`	Public URL
`name`	Stored path
`size`	Bytes
`open() / read()`	Contents
`save(name, file)`	Store new file
`delete()`	Remove file

SETTINGS

SETTING	PURPOSE
`MEDIA_ROOT`	Disk folder
`MEDIA_URL`	URL prefix
`DATA_UPLOAD_MAX_MEMORY_SIZE`	Body limit
`FILE_UPLOAD_PERMISSIONS`	File mode

⚠ COMMON MISTAKES

- Missing `enctype="multipart/form-data"`: `request.FILES` is empty and the form says "required"
- Trusting `content_type` or the extension; validate contents for anything risky
- Serving user uploads from the same domain without care; HTML/SVG files can run scripts

✓ TIPS & BEST PRACTICES

- Store uploads on object storage (S3, GCS) via `django-storages` in production
- Put uploads under per-user folders with an `upload_to` function
- Clean up orphaned files with a `post_delete` signal or a periodic task

QUICK SUMMARY

- `enctype="multipart/form-data"`: required
- `Form(request.POST, request.FILES)`
- `FileField` / `ImageField(upload_to=)`
- `MEDIA_ROOT` / `MEDIA_URL`: storage + URL

Admin Basics

THE BUILT-IN ADMIN

`django.contrib.admin` generates a full CRUD interface from your models. Register a model in the app's `admin.py`, create a superuser, and visit `/admin/`. It is meant for **trusted staff**, not end users. Only accounts with `is_staff = True` can log in, and they see only the models their permissions allow.

NOTES

- The admin URL is wired in `config/urls.py` as `path("admin/", admin.site.urls)`
- Model `__str__` is what the admin shows for each row
- `verbose_name` / `verbose_name_plural` set the labels
- Superusers have every permission automatically
- 6.0: admin icons now come from Font Awesome Free

REGISTER MODELS

```
# blog/admin.py
from django.contrib import admin
from .models import Post, Tag

admin.site.register(Tag)                          # default options

@admin.register(Post)                             # with options (Sheet 45)
class PostAdmin(admin.ModelAdmin):
    list_display = ["title", "status", "created"]

# config/urls.py — customize the site
admin.site.site_header = "Blog Administration"
admin.site.site_title = "Blog Admin"
admin.site.index_title = "Dashboard"
urlpatterns = [path("manage/", admin.site.urls)]   # move off /admin/
```

CODE EXPLAINED

- → `register(Model)` uses a plain `ModelAdmin`
- → `@admin.register` attaches a customized `ModelAdmin`
- → `site_header` shows in the top bar and login page
- → Changing the URL prefix reduces automated login attacks
- → The admin needs `auth`, `contenttypes`, `sessions` and `messages`

CREATE AN ADMIN USER

```
python manage.py createsuperuser
# Username: ann   Email: ann@example.com   Password: ********
python manage.py runserver   # then open http://127.0.0.1:8000/manage/
```

ADMINSITE ATTRIBUTES

ATTRIBUTE	SETS
`site_header`	Top bar text
`site_title`	`<title>` suffix
`index_title`	Home heading
`site_url`	"View site" link
`empty_value_display`	Shown for empty

DEFAULT PERMISSIONS

CODENAME	ALLOWS
`add_<model>`	Create
`change_<model>`	Edit
`delete_<model>`	Delete
`view_<model>`	Read only

USER FLAGS

FLAG	MEANS
`is_active`	Can log in at all
`is_staff`	Can use admin
`is_superuser`	All permissions

⚠ COMMON MISTAKES

- Building the customer-facing site on the admin; it's for staff workflows
- Leaving `/admin/` with a weak superuser password on a public server
- No `__str__` on models, so every row shows as "Post object (1)"

✓ TIPS & BEST PRACTICES

- Give staff accounts only the permissions they need, via groups
- Override `templates/admin/base_site.html` for branding
- Add 2FA with `django-otp` or similar for admin logins

QUICK SUMMARY

- `@admin.register(Model)` : add to admin
- `createsuperuser` : first login
- `is_staff` : admin access
- `admin.site.site_header` : branding

ModelAdmin Options

SHAPING THE CHANGE LIST AND FORM

A `ModelAdmin` class controls two pages per model. The **change list** shows rows in a table with columns, filters, search and pagination. The **change form** edits one object, with field order, grouping and read-only fields. Most customization is declarative: set class attributes and the admin builds the UI.

NOTES

- `list_display` can include model fields, methods or `ModelAdmin` methods
- `search_fields` supports lookups: `"author__username"`, `"=sku"`
- `list_select_related` stops N+1 queries on FK columns
- `date_hierarchy` adds year/month/day drill-down
- `@admin.display` sets a column's header, sort field and boolean icon

A CUSTOMIZED MODELADMIN

```python
@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ["title", "author", "status", "created", "is_live"]
    list_display_links = ["title"]
    list_editable = ["status"]
    list_filter = ["status", "created",
                   ("author", admin.RelatedOnlyFieldListFilter)]
    search_fields = ["title", "body", "author__username"]
    date_hierarchy = "created"
    ordering = ["-created"]
    list_per_page = 50
    list_select_related = ["author"]
    prepopulated_fields = {"slug": ["title"]}
    readonly_fields = ["created", "updated"]
    autocomplete_fields = ["author"]
    fieldsets = [
        (None, {"fields": ["title", "slug", "body"]}),
        ("Publishing", {"fields": ["status", "pub_date", "author"]}),
        ("Metadata", {"fields": ["created", "updated"],
                      "classes": ["collapse"]}),
    ]

    @admin.display(boolean=True, description="Live?", ordering="pub_date")
    def is_live(self, obj):
        return obj.is_live
```

CODE EXPLAINED

- → `list_editable` fields can't also be `list_display_links`
- → `RelatedOnlyFieldListFilter` lists only authors who have posts
- → `prepopulated_fields` fills the slug as you type the title
- → `autocomplete_fields` needs `search_fields` on the related admin
- → `classes: ["collapse"]` makes a fieldset collapsible

CHANGE LIST

OPTION	DOES
`list_display`	Columns
`list_filter`	Sidebar filters
`search_fields`	Search box
`list_editable`	Inline edit
`date_hierarchy`	Date drill-down
`list_per_page`	Page size

CHANGE FORM

OPTION	DOES
`fields`	Field order
`fieldsets`	Grouped fields
`readonly_fields`	Display only
`exclude`	Hide fields
`raw_id_fields`	ID input for FK
`filter_horizontal`	M2M widget

SEARCH_FIELDS PREFIXES

PREFIX	LOOKUP
`(none)`	icontains
`^`	istartswith
`=`	iexact
`@`	Full-text (Postgres)

⚠ COMMON MISTAKES

- A `ForeignKey` in `list_display` without `list_select_related`: one query per row
- A huge FK dropdown (thousands of users); use `autocomplete_fields` or `raw_id_fields`
- Putting a field in both `fields` and `exclude`: `ImproperlyConfigured`

✓ TIPS & BEST PRACTICES

- Add `show_facets = admin.ShowFacets.ALWAYS` (5.0+) to see filter counts
- Use `readonly_fields` for computed values via methods
- Run `manage.py check`; admin options are validated at startup

QUICK SUMMARY

- `list_display` / `list_filter` / `search_fields`
- `fieldsets`: group the edit form
- `readonly_fields`: show, don't edit
- `@admin.display`: custom columns

Inlines & Custom Actions

RELATED ROWS AND BULK ACTIONS

Inlines edit child objects on the parent's page, such as order items inside an order. `TabularInline` uses a compact table and `StackedInline` a full form per row. **Actions** run on rows selected in the change list, like "Mark as published". Override hook methods to filter visible rows or tighten permissions per user.

NOTES

- Inlines need a `ForeignKey` from child to parent
- `extra` controls how many blank inline rows appear
- Actions receive `(modeladmin, request, queryset)`
- Report results with `self.message_user()`
- `delete_selected` is the one built-in action

INLINES, ACTIONS, HOOKS

```python
class OrderItemInline(admin.TabularInline):
    model = OrderItem
    extra = 1
    fields = ["product", "quantity", "price"]

@admin.register(Order)
class OrderAdmin(admin.ModelAdmin):
    inlines = [OrderItemInline]
    actions = ["mark_shipped"]

    @admin.action(description="Mark selected orders as shipped",
                  permissions=["change"])
    def mark_shipped(self, request, queryset):
        n = queryset.update(status="S", shipped_at=timezone.now())
        self.message_user(request, f"{n} orders shipped.", messages.SUCCESS)

    def get_queryset(self, request):          # staff see their region only
        qs = super().get_queryset(request)
        if request.user.is_superuser:
            return qs
        return qs.filter(region=request.user.region)
```

CODE EXPLAINED

- → `inlines` lists inline classes shown under the main form
- → `permissions=["change"]` hides the action from users without change perm
- → `queryset` is exactly the rows the user ticked
- → `get_queryset` limits the change list and object access
- → `save_model(request, obj, form, change)` sets fields on save

INLINE OPTIONS

OPTION	DOES
`model`	Child model
`extra`	Blank rows
`min_num / max_num`	Row limits
`fk_name`	Pick the FK
`show_change_link`	Link to child page

MODELADMIN HOOKS

METHOD	OVERRIDE TO
`get_queryset`	Filter rows
`save_model`	Set fields on save
`get_readonly_fields`	Dynamic read-only
`get_form`	Swap the form
`formfield_for_…`	Limit FK/M2M choices

PERMISSION HOOKS

METHOD	CONTROLS
`has_view_permission`	See
`has_add_permission`	Add
`has_change_permission`	Edit
`has_delete_permission`	Delete
`has_module_permission`	App on index

⚠ COMMON MISTAKES

- Actions using `queryset.update()` and expecting `save()` / signals to run
- Filtering `get_queryset` but not `formfield_for_foreignkey`, so dropdowns leak other regions
- Too many inline rows with FK dropdowns; each row repeats the choice query

✓ TIPS & BEST PRACTICES

- Loop and call `obj.save()` in actions when signals or `save()` logic matter
- Return an `HttpResponseRedirect` from an action to show a confirmation page
- Use `TabularInline` for short rows, `StackedInline` for wide forms

QUICK SUMMARY

- `TabularInline` / `StackedInline`: child rows
- `@admin.action`: bulk operations
- `get_queryset()`: per-user rows
- `has_*_permission()`: fine-grained access

Users & Authentication

THE AUTH SYSTEM

`django.contrib.auth` provides a `User` model, password hashing, sessions-based login and permissions. `authenticate()` checks credentials against the configured backends and returns a user or `None`. `login()` stores the user's id in the session, after which `AuthenticationMiddleware` puts that user on `request.user` for every request. `logout()` clears the session.

NOTES

- Passwords are stored hashed (PBKDF2 by default, Argon2 optional)
- `request.user` is an `AnonymousUser` when not logged in
- `login()` rotates the session key to stop session fixation
- Use `User.objects.create_user()` so the password is hashed
- Get the active user model with `get_user_model()` (Sheet 50)

LOG IN AND OUT BY HAND

```python
from django.contrib.auth import authenticate, login, logout, get_user_model
from django.contrib.auth.decorators import login_required

User = get_user_model()
u = User.objects.create_user("ann", "ann@example.com", "s3cret!pass")
u.check_password("s3cret!pass")          # True
u.set_password("new-pass-123"); u.save() # hashes it

def login_view(request):
    if request.method == "POST":
        user = authenticate(request, username=request.POST["username"],
                            password=request.POST["password"])
        if user is not None:
            login(request, user)         # session cookie set
            return redirect(request.GET.get("next", "home"))
    return render(request, "login.html")

@login_required
def dashboard(request):
    return render(request, "dash.html", {"name": request.user.get_full_name()})
```

CODE EXPLAINED

- → `create_user` hashes the password; `create()` would store plain text
- → `authenticate` returns `None` for bad credentials or inactive users
- → Redirecting to raw `next` is unsafe; built-in `LoginView` validates it
- → `logout` must be POST when using the built-in `LogoutView`
- → In templates, the `auth` context processor adds `user` and `perms`

USER FIELDS

FIELD	TYPE
`username`	Unique, 150 chars
`email`	Optional
`first_name / last_name`	Text
`is_active / is_staff`	bool
`last_login / date_joined`	datetime

FUNCTIONS

FUNCTION	DOES
`authenticate()`	Check credentials
`login()`	Start session
`logout()`	End session
`alogin() etc.`	Async versions

PASSWORD HASHERS

HASHER	NOTES
`PBKDF2`	Default
`Argon2`	Recommended
`Scrypt`	Built in
`BCryptSHA256`	Needs bcrypt

⚠ COMMON MISTAKES

- `User.objects.create(password="x")` stores an unhashed password that never matches
- Changing a password without `update_session_auth_hash`, logging the user out
- Redirecting to `next` without `url_has_allowed_host_and_scheme`: open redirect

✓ TIPS & BEST PRACTICES

- Use the built-in auth views (Sheet 48) instead of hand-written login views
- Switch to Argon2 by listing it first in `PASSWORD_HASHERS`
- Set `AUTH_PASSWORD_VALIDATORS` to enforce password strength

QUICK SUMMARY

- `authenticate()` → user or None
- `login()` / `logout()`: session
- `request.user`: current user
- `create_user()` / `set_password()`: hashing

Built-in Auth Views

LOGIN, LOGOUT AND PASSWORD FLOWS

Django ships class-based views for the whole account flow: login, logout, password change and the four-step password reset by email. Include `django.contrib.auth.urls` to get them all at once; you only supply the templates in `templates/registration/`. Settings control where users go after login and logout.

NOTES

- Templates live in `registration/` (e.g. `registration/login.html`)
- `LogoutView` only accepts **POST** (5.0+)
- Password reset emails use the `EMAIL_BACKEND` setting
- `LoginView` validates `next` against allowed hosts
- Sign-up isn't built in; write a view with `UserCreationForm`

URLS AND SETTINGS

```python
# config/urls.py
urlpatterns = [
    path("accounts/", include("django.contrib.auth.urls")),
    path("accounts/signup/", SignUpView.as_view(), name="signup"),
]
# settings.py
LOGIN_URL = "login"                  # where @login_required sends users
LOGIN_REDIRECT_URL = "dashboard"     # after login (no ?next=)
LOGOUT_REDIRECT_URL = "home"

# accounts/views.py
from django.contrib.auth.forms import UserCreationForm
class SignUpView(CreateView):
    form_class = UserCreationForm
    template_name = "registration/signup.html"
    success_url = reverse_lazy("login")
```

REGISTRATION/LOGIN.HTML

```html
<form method="post">{% csrf_token %}
  {{ form }}
  <input type="hidden" name="next" value="{{ next }}">
  <button>Log in</button>  <a href="{% url 'password_reset' %}">Forgot?</a>
</form>
<form method="post" action="{% url 'logout' %}">
  {% csrf_token %}<button>Log out</button>
</form>
```

CODE EXPLAINED

- → One `include` adds 8 named URLs (table below)
- → `LOGIN_URL` accepts a URL name or path
- → `UserCreationForm` handles password confirmation and validators
- → The hidden `next` input sends users back where they started
- → Logout is a POST form, not a link

URL NAMES

NAME	VIEW
`login`	LoginView
`logout`	LogoutView
`password_change`	Change form
`password_reset`	Enter email
`…reset_confirm`	Set new password

TEMPLATES NEEDED

TEMPLATE	FOR
`login.html`	Login form
`password_reset_form`	Email form
`password_reset_email`	Email body
`…reset_confirm`	New password
`.html suffix`	on each name

AUTH FORMS

FORM	PURPOSE
`AuthenticationForm`	Login
`UserCreationForm`	Sign-up
`PasswordChangeForm`	Change
`SetPasswordForm`	Set w/o old
`PasswordResetForm`	Send email

⚠ COMMON MISTAKES

- A logout `<a href>` link; GET to `LogoutView` returns 405 in 5.0+
- Missing `registration/login.html`: `TemplateDoesNotExist`
- Testing password reset with no email backend; use the console backend in dev

✓ TIPS & BEST PRACTICES

- Set `EMAIL_BACKEND = "django.core.mail.backends.console.EmailBackend"` in dev
- Subclass `UserCreationForm` to add fields like email
- Use django-allauth if you need social login or email verification

QUICK SUMMARY

- `include("django.contrib.auth.urls")`
- Templates in `registration/`
- `LOGIN_URL` / `LOGIN_REDIRECT_URL`
- `UserCreationForm`: DIY sign-up

Permissions & Groups

WHO CAN DO WHAT

Every model gets four permissions automatically: **add**, **change**, **delete** and **view**. Permissions are named `"app_label.codename"` and can be given to users directly or through **groups** (roles). Check them in code with `user.has_perm()`, in views with decorators or mixins, and in templates with `{{ perms }}`. Superusers pass every check.

NOTES

- Add custom permissions in the model's `Meta.permissions`
- Permissions are cached on the user object per request
- Model-level only by default; object-level needs django-guardian or rules
- Inactive users have no permissions
- Create groups in a data migration so every environment matches

DEFINING AND CHECKING

```python
class Report(models.Model):
    class Meta:
        permissions = [("publish_report", "Can publish reports")]

from django.contrib.auth.models import Group, Permission
editors, _ = Group.objects.get_or_create(name="Editors")
perm = Permission.objects.get(codename="publish_report")
editors.permissions.add(perm)
user.groups.add(editors)

user.has_perm("reports.publish_report")          # True
user.has_perms(["reports.view_report", "reports.change_report"])
user.get_all_permissions()                       # {'reports.publish_report', ...}
user.groups.filter(name="Editors").exists()      # role check

@permission_required("reports.publish_report", raise_exception=True)
def publish(request, pk): ...
```

CODE EXPLAINED

- → `(codename, name)` tuples in `Meta.permissions`
- → Group members inherit all the group's permissions
- → `has_perms` requires every listed permission
- → `raise_exception=True` returns 403 instead of redirecting
- → `perms.app` is true if the user has any permission in that app

TEMPLATES

```
{% if perms.reports.publish_report %}<button>Publish</button>{% endif %}
{% if perms.reports %}<a href="...">Reports</a>{% endif %}  {# any perm in app #}
```

AUTO PERMISSIONS

CODENAME	ACTION
add_report	Create
change_report	Edit
delete_report	Delete
view_report	Read

WHERE TO CHECK

PLACE	TOOL
FBV	@permission_required
CBV	PermissionRequiredMixin
Code	user.has_perm()
Template	{{ perms.app.code }}
Admin	Automatic

OBJECT-LEVEL OPTIONS

OPTION	APPROACH
Queryset filter	filter(owner=user)
django-guardian	Per-object perms
django-rules	Predicate functions
Custom backend	has_perm(obj)

⚠ COMMON MISTAKES

- Checking `user.is_staff` for feature access; use specific permissions
- Wrong label format: `has_perm("publish_report")` must be `"reports.publish_report"`
- Adding a permission then reading `has_perm` on the same user object; the cache is stale, re-fetch

✓ TIPS & BEST PRACTICES

- Model roles as groups (Editors, Support) and assign users to groups
- Create groups and their permissions in a data migration
- Filter querysets by owner for simple per-object access

QUICK SUMMARY

- `app_label.codename` : permission name
- `Group` : role = bundle of permissions
- `user.has_perm()` : check in code
- `{{ perms.app.codename }}` : check in templates

Custom User Model

START WITH A CUSTOM USER

Django's docs recommend creating a **custom user model at the start of every project**, even if it's identical to the default, because switching later means rewriting migrations. Subclass `AbstractUser` to keep everything and add fields, or `AbstractBaseUser` + `PermissionsMixin` to design the model from scratch (for example, email login with no username). Point `AUTH_USER_MODEL` at it before the first `migrate`.

NOTES

- Set `AUTH_USER_MODEL = "accounts.User"` before the first migration
- Refer to it as `settings.AUTH_USER_MODEL` in ForeignKeys
- Use `get_user_model()` everywhere else
- Register it in admin with `UserAdmin` (or a subclass)
- Profile data unrelated to auth can live in a separate `Profile` model

EMAIL LOGIN USER

```python
# accounts/models.py
from django.contrib.auth.models import AbstractUser, BaseUserManager

class UserManager(BaseUserManager):
    def create_user(self, email, password=None, **extra):
        user = self.model(email=self.normalize_email(email), **extra)
        user.set_password(password)
        user.save(using=self._db)
        return user
    def create_superuser(self, email, password, **extra):
        extra.update(is_staff=True, is_superuser=True)
        return self.create_user(email, password, **extra)

class User(AbstractUser):
    username = None                          # drop the username field
    email = models.EmailField(unique=True)

    USERNAME_FIELD = "email"                 # log in with email
    REQUIRED_FIELDS = []                     # asked by createsuperuser
    objects = UserManager()

# settings.py
AUTH_USER_MODEL = "accounts.User"

# accounts/admin.py
from django.contrib.auth.admin import UserAdmin
admin.site.register(User, UserAdmin)
```

CODE EXPLAINED

- → Setting `username = None` removes the inherited field
- → `USERNAME_FIELD` names the unique login field
- → `REQUIRED_FIELDS` excludes `USERNAME_FIELD` and password
- → A custom manager is needed once `username` is gone
- → `UserAdmin` may need `ordering` / `fieldsets` tweaks without `username`

BASE CLASSES

CLASS	INCLUDES
`AbstractUser`	Full default user
`AbstractBaseUser`	Password + last_login
`PermissionsMixin`	Groups + perms
`BaseUserManager`	Manager helpers

REFERRING TO THE USER

WHERE	USE
`ForeignKey`	`settings.AUTH_USER_MODEL`
`Code`	`get_user_model()`
`Forms`	Subclass with `model = User`

REQUIRED ATTRIBUTES

ATTRIBUTE	PURPOSE
`USERNAME_FIELD`	Login identifier
`EMAIL_FIELD`	Email field name
`REQUIRED_FIELDS`	createsuperuser prompts
`is_active`	Can log in

⚠ COMMON MISTAKES

- Switching `AUTH_USER_MODEL` mid-project; existing migrations and FKs break
- `from django.contrib.auth.models import User` in your code; it ignores the custom model
- Forgetting to update `UserCreationForm` / `UserChangeForm` for the new model

✓ TIPS & BEST PRACTICES

- Even if unsure, create `class User(AbstractUser): pass` on day one
- Keep auth fields on `User`; put app-specific data on a `Profile`
- Make email lookups case-insensitive with a `UniqueConstraint(Lower("email"))`

QUICK SUMMARY

- `AbstractUser`: extend the default
- `AUTH_USER_MODEL`: set before first migrate
- `get_user_model()`: get the class
- `USERNAME_FIELD`: login field

Middleware

HOOKS AROUND EVERY REQUEST

Middleware is a chain of callables that wrap every request and response. Requests flow **down** the `MIDDLEWARE` list to the view, and responses flow back **up** in reverse order. Each middleware can inspect or change the request, short-circuit with its own response, or modify the response. Built-in middleware handles security headers, sessions, CSRF, auth and messages.

NOTES

- Order matters: sessions before auth, auth before anything using `request.user`
- Returning a response early skips the rest of the chain and the view
- `__init__` runs **once** at startup, not per request
- Raise `MiddlewareNotUsed` in `__init__` to disable conditionally
- Mark sync/async support with `sync_capable` / `async_capable`

WRITING MIDDLEWARE

```python
import time, logging
log = logging.getLogger(__name__)

class TimingMiddleware:
    def __init__(self, get_response):          # once, at startup
        self.get_response = get_response

    def __call__(self, request):               # every request
        start = time.perf_counter()
        response = self.get_response(request)  # call next layer / view
        ms = (time.perf_counter() - start) * 1000
        response["Server-Timing"] = f"app;dur={ms:.1f}"
        return response

    def process_view(self, request, view_func, view_args, view_kwargs):
        request.view_name = view_func.__name__ # None = continue

    def process_exception(self, request, exception):
        log.exception("Unhandled error on %s", request.path)
        return None                            # let Django handle it

# settings.py
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    # ... sessions, common, csrf, auth, messages, clickjacking
    "core.middleware.TimingMiddleware",            # yours, last
]
```

CODE EXPLAINED

→ Code before `get_response` runs on the way in

→ Code after it runs on the way out

→ `process_view` runs just before the view is called

→ `process_exception` runs only when the view raises

→ Returning `None` from hooks means "carry on"

BUILT-IN (…MIDDLEWARE)

NAME	ADDS
`Security`	HTTPS, HSTS
`Session`	`request.session`
`Common`	APPEND_SLASH
`CsrfView`	CSRF checks
`Authentication`	`request.user`
`GZip`	Compression

OPTIONAL HOOKS

HOOK	WHEN
`process_view`	Before view
`process_exception`	View raised
`process_template…`	TemplateResponse

NEWER BUILT-INS

MIDDLEWARE	SINCE
`LoginRequired…`	5.1
`ContentSecurityPolicy…`	6.0

⚠ COMMON MISTAKES

- Custom middleware using `request.user` placed above `AuthenticationMiddleware`
- Heavy work in `__call__` (DB queries) that slows every single request
- Storing per-request data on `self`; one instance is shared by all requests

✓ TIPS & BEST PRACTICES

- Attach data to `request` (`request.tenant = ...`) for views to use
- Use a function-based middleware for tiny wrappers
- Check order with the debug toolbar or by logging in each layer

QUICK SUMMARY

- `MIDDLEWARE`: ordered list
- `__call__`: wrap `get_response(request)`
- `process_view` / `process_exception`: hooks
- Down for requests, up for responses

Sessions & Cookies

REMEMBERING USERS BETWEEN REQUESTS

Sessions store per-visitor data on the server and send the browser only a session id cookie. Use `request.session` like a dict; values must be JSON-serializable. The storage backend (`SESSION_ENGINE`) can be the database, cache or signed cookies. For small, non-secret values you can set **cookies** directly on the response, optionally **signed** to detect tampering.

NOTES

- Sessions are saved only when modified; nested changes need `modified = True`
- Default engine stores sessions in the `django_session` table
- Run `clearsessions` regularly to delete expired rows
- `login()` rotates the session key; `logout()` flushes it
- Signed cookies can be read by the user; don't store secrets in them

SESSIONS AND COOKIES

```
def add_to_cart(request, pk):
    cart = request.session.get("cart", {})       # dict-like
    cart[str(pk)] = cart.get(str(pk), 0) + 1
    request.session["cart"] = cart               # reassign: marks modified
    request.session.set_expiry(60 * 60 * 24 * 7) # one week
    return redirect("cart")

request.session["cart"]["5"] = 2                 # nested change...
request.session.modified = True                  # ...needs this
del request.session["cart"]                      # remove a key
request.session.flush()                          # delete all + new key
request.session.session_key                      # 'k3j2...'

def set_theme(request):
    resp = redirect("home")
    resp.set_cookie("theme", "dark", max_age=31536000,
                    secure=True, httponly=True, samesite="Lax")
    resp.set_signed_cookie("ref", "news", salt="ref")
    return resp
request.COOKIES.get("theme")                     # 'dark'
request.get_signed_cookie("ref", default=None, salt="ref")
resp.delete_cookie("theme")
```

CODE EXPLAINED

- → JSON keys are strings, so use `str(pk)` as dict keys
- → Reassigning the key automatically marks the session modified
- → `set_expiry(0)` ends the session when the browser closes
- → `httponly=True` hides the cookie from JavaScript
- → Signed cookie reads return `default` if tampered

SESSION_ENGINE

ENGINE	STORES IN
`…sessions.backends.db`	Database
`…backends.cache`	Cache only
`…backends.cached_db`	Cache + DB
`…backends.signed_cookies`	Cookie
`…backends.file`	Files

SESSION_COOKIE_*

SETTING	DEFAULT
`…_AGE`	2 weeks
`…_SECURE`	False
`…_HTTPONLY`	True
`…_SAMESITE`	"Lax"
`…_DOMAIN`	None

SESSION API

CALL	DOES
`get(k, d)`	Read
`pop(k)`	Read + remove
`set_expiry(s)`	Lifetime
`cycle_key()`	New key, keep data
`aget() / aset()`	Async (5.1+)

⚠ COMMON MISTAKES

- Storing model instances or `Decimal` in the session; they aren't JSON-serializable
- Mutating a nested dict without `modified = True`, so changes are lost
- Letting `django_session` grow forever; schedule `clearsessions`

✓ TIPS & BEST PRACTICES

- Use `cached_db` for speed with durability
- Set `SESSION_COOKIE_SECURE = True` in production
- Store ids in the session and look objects up per request

QUICK SUMMARY

- `request.session[k] = v` : per-visitor data
- `SESSION_ENGINE` : storage backend
- `set_cookie()` / `COOKIES` : raw cookies
- `set_signed_cookie()` : tamper-evident

Messages Framework

ONE-TIME FLASH MESSAGES

The **messages framework** shows one-time notices like "Profile saved" on the next page the user sees. Add a message in a view before redirecting; it's stored (in the session or a cookie) and removed once displayed. Templates get a `messages` variable from the context processor. Each message has a **level** and **tags** you can map to CSS classes.

NOTES

- Enabled by default: app, middleware and context processor
- Messages are consumed when iterated in a template
- Default storage tries cookies, then falls back to the session
- `SuccessMessageMixin` adds messages to generic edit views
- `fail_silently=True` avoids errors when the framework is off

ADDING MESSAGES

```
from django.contrib import messages
from django.contrib.messages.views import SuccessMessageMixin

def profile_save(request):
    ...
    messages.success(request, "Profile updated.")
    messages.warning(request, "Your email is not verified.")
    messages.add_message(request, messages.INFO, "Tip: add a photo.",
                         extra_tags="dismissible")
    return redirect("profile")

class PostCreate(SuccessMessageMixin, CreateView):
    model = Post
    fields = ["title", "body"]
    success_message = "\"%(title)s\" was created."
```

CODE EXPLAINED

- → Shortcut functions exist for each level
- → `extra_tags` adds classes alongside the level tag
- → `%(title)s` pulls from `cleaned_data`
- → `MESSAGE_TAGS = {m.ERROR: "danger"}` renames a level's CSS tag
- → Iterating `messages` marks them as shown

BASE.HTML

```
{% if messages %}<ul class="messages">
  {% for message in messages %}
    <li class="alert alert-{{ message.tags }}">{{ message }}</li>
  {% endfor %}
</ul>{% endif %}
```

LEVELS

LEVEL	VALUE / TAG
DEBUG	10 · debug
INFO	20 · info
SUCCESS	25 · success
WARNING	30 · warning
ERROR	40 · error

MESSAGE_STORAGE

BACKEND	USES
FallbackStorage	Cookie → session
CookieStorage	Signed cookie
SessionStorage	Session

MESSAGE ATTRIBUTES

ATTRIBUTE	VALUE
message	Text
level	int
tags	Level + extra tags
level_tag	Level tag only

⚠ COMMON MISTAKES

- Adding a message and rendering (not redirecting); it shows now **and** on the next page
- Not displaying `messages` in the base template, so they pile up and appear later
- `DEBUG` messages never showing; `MESSAGE_LEVEL` defaults to INFO

✓ TIPS & BEST PRACTICES

- Render messages once in `base.html` so every page shows them
- Map `ERROR` to `danger` when using Bootstrap
- For htmx, return messages in a response header or an out-of-band swap

QUICK SUMMARY

- `messages.success(request, msg)` : queue one
- `{% for message in messages %}` : show them
- `message.tags` : CSS class
- `SuccessMessageMixin` : for CBVs

Signals

REACTING TO EVENTS

Signals let decoupled code react when something happens elsewhere: a model is saved, a user logs in, a request finishes. A **receiver** function is connected to a signal, usually with `@receiver`, and is called synchronously by whoever sends it. Register receivers in your `AppConfig.ready()` so they're connected exactly once at startup.

NOTES

- Receivers run in the same thread and transaction as the sender
- `queryset.update()` and `bulk_create()` don't send model signals
- Pass `sender=` to listen to one model only
- Use `dispatch_uid` to avoid duplicate connections
- Prefer explicit calls when the code is in the same app

RECEIVERS AND REGISTRATION

```python
# accounts/signals.py
from django.conf import settings
from django.db.models.signals import post_save, pre_delete
from django.dispatch import receiver, Signal

@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def create_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)

@receiver(pre_delete, sender=Document)
def delete_file(sender, instance, **kwargs):
    instance.file.delete(save=False)

order_paid = Signal()                          # custom signal
order_paid.send(sender=Order, order=order)     # send it

# accounts/apps.py
class AccountsConfig(AppConfig):
    name = "accounts"
    def ready(self):
        from . import signals                  # connect receivers
```

CODE EXPLAINED

→ Receivers must accept `**kwargs` for future arguments

→ `created` is True only on the first save

→ `save=False` deletes the file without re-saving the row

→ Importing `signals` in `ready()` runs the decorators

→ `send()` returns a list of `(receiver, response)`

MODEL SIGNALS

SIGNAL	WHEN
`pre_save / post_save`	Around `save()`
`pre_delete / post_delete`	Around delete
`m2m_changed`	M2M add/remove
`pre_init / post_init`	Instance created

OTHER SIGNALS

SIGNAL	WHEN
`request_started`	Request begins
`request_finished`	Response sent
`user_logged_in`	After login
`user_login_failed`	Bad credentials
`post_migrate`	After migrate

SIGNAL API

CALL	DOES
`connect(fn)`	Register
`disconnect(fn)`	Unregister
`send(sender, **kw)`	Fire
`send_robust()`	Catch errors
`asend()`	Async fire

⚠ COMMON MISTAKES

- Defining receivers in a module that's never imported, so they never fire
- Expecting `post_save` after `queryset.update()`; no signal is sent
- Doing slow work (emails, APIs) in a receiver; it blocks the request, enqueue a task instead

✓ TIPS & BEST PRACTICES

- Use `transaction.on_commit` inside receivers for side effects
- Signals suit cross-app hooks; within an app, call the function directly
- Disconnect noisy receivers in tests with `post_save.disconnect`

QUICK SUMMARY

- `@receiver(signal, sender=)`: listen
- `post_save(created=...)`: after save
- `AppConfig.ready()`: register once
- `Signal().send()`: custom events

CSRF, XSS & Clickjacking

BUILT-IN PROTECTIONS

Django blocks three classic web attacks by default. **CSRF** protection rejects POST/PUT/DELETE requests that lack a secret token, so other sites can't submit forms as your users. **XSS** protection auto-escapes template output so user text can't inject scripts. **Clickjacking** protection sends `X-Frame-Options: DENY` so your pages can't be framed. The ORM's parameterized queries also prevent SQL injection.

NOTES

- Every POST form needs `{% csrf_token %}`
- JavaScript requests send the token in the `X-CSRFToken` header
- Cross-origin HTTPS POSTs need the site in `CSRF_TRUSTED_ORIGINS`
- `mark_safe` / `|safe` turn escaping off; never use them on user input
- `format_html()` escapes arguments while building HTML safely

CSRF IN FORMS AND FETCH()

```
<form method="post">{% csrf_token %} ... </form>

<script>
  const token = document.querySelector("[name=csrfmiddlewaretoken]").value;
  fetch("/api/like/", {method: "POST", headers: {"X-CSRFToken": token}});
</script>
<div hx-headers='{"X-CSRFToken": "{{ csrf_token }}"}'> ... </div>  {# htmx #}

{{ comment.text }}                {# <script> becomes &lt;script&gt; #}
```

PYTHON SIDE

```
from django.utils.html import format_html, escape
from django.views.decorators.csrf import csrf_exempt
from django.views.decorators.clickjacking import xframe_options_sameorigin

format_html("<b>{}</b>", user_input)          # input escaped
escape("<i>hi</i>")                           # '&lt;i&gt;hi&lt;/i&gt;'

@csrf_exempt                                  # e.g. signed webhook only
def stripe_webhook(request): ...

@xframe_options_sameorigin                    # allow framing by own site
def embed(request): ...

X_FRAME_OPTIONS = "DENY"                      # settings (default)
CSRF_TRUSTED_ORIGINS = ["https://app.example.com"]
```

CODE EXPLAINED

- → The token is tied to a cookie and checked by `CsrfViewMiddleware`
- → `{{ csrf_token }}` gives the raw value for headers
- → Auto-escaping converts `< > & ' "` to entities
- → Only exempt views that verify requests another way (signatures)
- → Trusted origins must include the scheme

ATTACK → DEFENSE

ATTACK	DJANGO DEFENSE
CSRF	Token + middleware
XSS	Auto-escaping
SQL injection	Parameterized ORM
Clickjacking	X-Frame-Options
Host header	ALLOWED_HOSTS

CSRF SETTINGS

SETTING	DEFAULT
CSRF_COOKIE_SECURE	False
CSRF_COOKIE_HTTPONLY	False
CSRF_USE_SESSIONS	False
CSRF_TRUSTED_ORIGINS	[]
CSRF_FAILURE_VIEW	403 page

ESCAPING TOOLS

TOOL	EFFECT
escape()	Escape a string
format_html()	Safe HTML builder
mark_safe()	Trust a string
|escapejs	For JS strings
|json_script	Data to JS

⚠ COMMON MISTAKES

- Adding `@csrf_exempt` to fix a 403 instead of sending the token
- `mark_safe(f"<b>{name}</b>")`: user input becomes live HTML; use `format_html`
- Putting `{{ data }}` inside `<script>`; escaping rules differ there, use `json_script`

✓ TIPS & BEST PRACTICES

- Read the CSRF failure page in DEBUG; it explains the exact cause
- Add Content Security Policy (Sheet 56) as a second XSS layer
- Run `manage.py check --deploy` before every launch

QUICK SUMMARY

- `{% csrf_token %}`: every POST form
- `X-CSRFToken`: AJAX header
- Auto-escape: XSS defense
- `X_FRAME_OPTIONS`: no framing

Content Security Policy

CSP BUILT IN (DJANGO 6.0)

A **Content Security Policy** header tells the browser which sources of scripts, styles, images and frames are allowed, so injected scripts can't run even if escaping fails. Django 6.0 adds native support: enable `ContentSecurityPolicyMiddleware`, describe the policy as a dict in `SECURE_CSP`, and use `CSP.NONCE` with `{{ csp_nonce }}` to allow specific inline scripts.

NOTES

- `SECURE_CSP` sends an enforced `Content-Security-Policy` header
- `SECURE_CSP_REPORT_ONLY` only reports violations (use it first)
- Constants live in `django.utils.csp.CSP`
- Per-view overrides **replace** the policy; `{}` disables it
- Django doesn't collect reports; point `report-uri` at your endpoint

SETTINGS.PY

```
from django.utils.csp import CSP

MIDDLEWARE += ["django.middleware.csp.ContentSecurityPolicyMiddleware"]
TEMPLATES[0]["OPTIONS"]["context_processors"] += [
    "django.template.context_processors.csp"]       # adds csp_nonce
SECURE_CSP = {
    "default-src": [CSP.SELF],
    "script-src": [CSP.SELF, CSP.NONCE],
    "style-src": [CSP.SELF, "https://fonts.googleapis.com"],
    "img-src": [CSP.SELF, "data:", "https:"],
    "frame-ancestors": [CSP.NONE],
}
SECURE_CSP_REPORT_ONLY = {"default-src": [CSP.SELF], "report-uri": "/csp/"}
```

CODE EXPLAINED

- `CSP.NONCE` becomes `'nonce-<random>'` per request
- Only scripts with the matching `nonce` attribute run
- Header: `script-src 'self' 'nonce-…'`
- `frame-ancestors 'none'` is the modern clickjacking guard
- Report-only and enforced headers can be sent together

TEMPLATE

```
<script nonce="{{ csp_nonce }}">initPage();</script>
```

PER-VIEW

```
from django.views.decorators.csp import csp_override
@csp_override({"default-src": [CSP.SELF], "img-src": [CSP.SELF, "data:"]})
def gallery(request): ...
@csp_override({})                                   # no CSP header
def legacy_widget(request): ...
```

CSP CONSTANTS

CONSTANT	VALUE
SELF	'self'
NONE	'none'
NONCE	Per-request nonce
UNSAFE_INLINE	'unsafe-inline'
STRICT_DYNAMIC	'strict-dynamic'
UNSAFE_EVAL	'unsafe-eval'

KEY DIRECTIVES

DIRECTIVE	CONTROLS
default-src	Fallback for all
script-src	JavaScript
style-src	CSS
img-src	Images
connect-src	fetch / XHR / WS
frame-ancestors	Who can frame you

ROLLOUT PLAN

STEP	ACTION
1	Report-only policy
2	Collect violations
3	Add nonces / sources
4	Switch to SECURE_CSP

⚠ COMMON MISTAKES

- Enforcing a strict policy on day one; the admin or third-party scripts break
- Adding `CSP.UNSAFE_INLINE` to script-src, which removes most XSS protection
- Caching full pages that contain `{{ csp_nonce }}`; reused nonces weaken the policy

✓ TIPS & BEST PRACTICES

- Start with `SECURE_CSP_REPORT_ONLY` and watch the browser console
- Move inline scripts to static files so few nonces are needed
- 6.1 adds a `{% csp_nonce_attr %}` tag and nonces in built-in templates

QUICK SUMMARY

- `ContentSecurityPolicyMiddleware`: sends headers
- `SECURE_CSP = {directive: [sources]}`
- `CSP.NONCE` + `{{ csp_nonce }}`: inline scripts
- `@csp_override`: per-view policy

Security Settings & Deployment Checklist

HARDENING FOR PRODUCTION

Development defaults favor convenience, so production needs a short list of settings turned on. `python manage.py check --deploy` inspects your settings and warns about each missing one. The core items: `DEBUG = False`, secrets from the environment, HTTPS everywhere with HSTS, secure cookies and correct `ALLOWED_HOSTS`.

NOTES

- Behind a proxy, set `SECURE_PROXY_SSL_HEADER` so Django knows the request was HTTPS
- Start HSTS with a small value (3600), then raise to a year
- `SECURE_HSTS_PRELOAD` is hard to undo; be sure first
- Rotate `SECRET_KEY` with `SECRET_KEY_FALLBACKS`
- Keep Django on a supported release for security patches

PRODUCTION SETTINGS

```
import os

DEBUG = False
SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
SECRET_KEY_FALLBACKS = [os.environ.get("DJANGO_OLD_SECRET_KEY", "")]
ALLOWED_HOSTS = ["example.com", "www.example.com"]
CSRF_TRUSTED_ORIGINS = ["https://example.com", "https://www.example.com"]

SECURE_SSL_REDIRECT = True                  # http -> https
SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")
SECURE_HSTS_SECONDS = 31536000              # one year
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True
SESSION_COOKIE_SECURE = True                # cookies only over HTTPS
CSRF_COOKIE_SECURE = True
SECURE_REFERRER_POLICY = "same-origin"
SECURE_CONTENT_TYPE_NOSNIFF = True          # default True
```

CODE EXPLAINED

- → `SECRET_KEY_FALLBACKS` keeps old sessions valid during rotation
- → `SECURE_SSL_REDIRECT` handles redirects in `SecurityMiddleware`
- → The proxy header must only be trusted if your proxy sets it
- → HSTS tells browsers to use HTTPS for the whole period
- → `check --deploy` reports `security.W0xx` warnings

CHECK IT

```
DJANGO_SETTINGS_MODULE=config.settings.prod python manage.py check --deploy
pip-audit                                   # known-vulnerable packages
```

CHECKLIST

ITEM	SETTING
Debug off	DEBUG = False
Secret in env	not in git
Hosts set	ALLOWED_HOSTS
HTTPS only	…SSL_REDIRECT
HSTS	…HSTS_SECONDS
Secure cookies	*_COOKIE_SECURE

HEADERS SENT

HEADER	FROM
HSTS	HSTS settings
X-Content-Type	NOSNIFF
Referrer-Policy	REFERRER_POLICY
X-Frame-Options	X_FRAME_OPTIONS
COOP	same-origin

ALSO CONSIDER

ITEM	HOW
Admin URL	Non-default path
Rate limiting	django-ratelimit
Error alerts	Sentry
Dependencies	pip-audit / Dependabot
Backups	Tested restores

⚠ COMMON MISTAKES

- Leaving `DEBUG = True`: error pages leak settings, SQL and source code
- `SECURE_SSL_REDIRECT` behind a proxy without `SECURE_PROXY_SSL_HEADER`: infinite redirects
- `ALLOWED_HOSTS = ["*"]` in production, which enables host-header attacks

✓ TIPS & BEST PRACTICES

- Run `check --deploy` in CI against production settings
- Keep a separate `prod.py` settings file with only safe defaults
- Subscribe to the Django security announcements mailing list

QUICK SUMMARY

- `check --deploy`: find gaps
- `DEBUG = False` + env secrets
- `SECURE_SSL_REDIRECT` + HSTS
- `*_COOKIE_SECURE = True`

Testing Basics

DJANGO'S TEST RUNNER

Django's test framework builds on `unittest`. `manage.py test` finds files named `test*.py`, creates a **separate test database**, runs each test inside a transaction and rolls it back afterwards, so tests don't affect each other. Use `TestCase` for anything touching the database and `SimpleTestCase` for tests that don't.

NOTES

- Test DB is named `test_<name>` and destroyed after the run
- `setUpTestData()` runs once per class: fast shared fixtures
- `setUp()` runs before **every** test method
- Use `--keepdb` to reuse the test database between runs
- pytest-django is a popular alternative runner

BLOG/TESTS.PY

```python
from django.test import TestCase, SimpleTestCase
from django.urls import reverse
from .models import Post

class PostModelTests(TestCase):
    @classmethod
    def setUpTestData(cls):                    # once for the class
        cls.post = Post.objects.create(title="Hello World", status="P")

    def test_str(self):
        self.assertEqual(str(self.post), "Hello World")

    def test_published_manager(self):
        Post.objects.create(title="Draft", status="D")
        self.assertQuerySetEqual(Post.objects.published(), [self.post])

    def test_list_view(self):
        resp = self.client.get(reverse("blog:list"))
        self.assertEqual(resp.status_code, 200)
        self.assertContains(resp, "Hello World")
        self.assertTemplateUsed(resp, "blog/post_list.html")

class SlugTests(SimpleTestCase):               # no database
    def test_slugify(self): self.assertEqual(slugify("Hi!"), "hi")
```

CODE EXPLAINED

→ Test methods must start with `test`

→ Objects from `setUpTestData` are deep-copied per test

→ `assertQuerySetEqual` compares a QuerySet to a list

→ `assertContains` checks status 200 **and** text

→ Run one test with its dotted path

RUNNING

```
python manage.py test                          # everything
python manage.py test blog.tests.PostModelTests.test_str --keepdb -v 2
coverage run manage.py test && coverage report
```

TEST CLASSES

CLASS	USE
`SimpleTestCase`	No DB
`TestCase`	DB, rolled back
`TransactionTestCase`	Real commits
`LiveServerTestCase`	Browser tests

DJANGO ASSERTIONS

ASSERTION	CHECKS
`assertContains`	Text in response
`assertRedirects`	Redirect target
`assertTemplateUsed`	Template
`assertNumQueries(n)`	Query count
`assertFormError`	Form errors

TEST FLAGS

FLAG	EFFECT
`--keepdb`	Reuse DB
`--parallel`	Run in processes
`--failfast`	Stop on first fail
`--tag / -k`	Filter tests
`--debug-sql`	Print SQL

⚠ COMMON MISTAKES

- Creating data in `setUp()` for every test when `setUpTestData` would do it once
- Tests that depend on run order or on each other's data
- Hitting real external APIs in tests; mock them with `unittest.mock.patch`

✓ TIPS & BEST PRACTICES

- Use factory_boy or model_bakery to create test data quickly
- Guard performance with `assertNumQueries` on list views
- Run tests in CI on every push, with `--parallel` for speed

QUICK SUMMARY

- `TestCase`: DB tests with rollback
- `setUpTestData()`: shared fixtures
- `manage.py test`: run the suite
- `assertContains` / `assertNumQueries`

Test Client & Fixtures

TESTING VIEWS END TO END

The **test client** (`self.client`) acts like a browser without a server: it sends requests through URLs, middleware and views and returns the response with extra attributes such as `context` and `templates`. Log users in instantly with `force_login()`. `RequestFactory` builds bare requests to call a view directly. Fixtures, mocks and `override_settings` control the environment.

NOTES

- `response.context["form"]` lets you inspect template variables
- `follow=True` follows redirects and records `redirect_chain`
- The test client skips CSRF checks by default
- `mail.outbox` collects emails sent during tests
- `captureOnCommitCallbacks` runs `on_commit` hooks in `TestCase`

CLIENT TESTS

```python
from django.core import mail
from django.test import TestCase, RequestFactory, override_settings

class PostViewTests(TestCase):
    fixtures = ["users.json"]                    # loaded per test class

    def setUp(self):
        self.user = User.objects.get(username="ann")
        self.client.force_login(self.user)
    def test_create_redirects(self):
        resp = self.client.post(reverse("blog:create"),
                                {"title": "New", "body": "Text"})
        post = Post.objects.get(title="New")
        self.assertRedirects(resp, post.get_absolute_url())

    def test_invalid_form(self):
        resp = self.client.post(reverse("blog:create"), {"title": ""})
        self.assertFormError(resp.context["form"], "title",
                             "This field is required.")
    @override_settings(DEFAULT_FROM_EMAIL="t@x.io")  # temp setting
    def test_email(self):                            # locmem backend in tests
        self.client.post(reverse("contact"), {"msg": "Hi"})
        self.assertEqual(len(mail.outbox), 1)
    def test_view_directly(self):
        request = RequestFactory().get("/dash/")
        request.user = self.user
        self.assertEqual(dashboard(request).status_code, 200)
```

CODE EXPLAINED

- → `force_login` skips the password check (fast)
- → `assertRedirects` also fetches the target and checks it loads
- → `post(url, data, content_type="application/json")` sends JSON
- → `resp.json()` parses a JSON response
- → `RequestFactory` skips middleware: set `request.user` yourself

CLIENT METHODS

METHOD	DOES
`get(url, data)`	GET with query
`post(url, data)`	Form POST
`force_login(u)`	Log in
`logout()`	Log out
`cookies / session`	State

RESPONSE EXTRAS

ATTRIBUTE	HOLDS
`status_code`	int
`context`	Template context
`templates`	Templates used
`redirect_chain`	With `follow=True`
`json()`	Parsed JSON

TEST DATA OPTIONS

OPTION	NOTES
`fixtures = [...]`	JSON/YAML files
`setUpTestData`	Python objects
`factory_boy`	Factories
`model_bakery`	Auto-fill models

⚠ COMMON MISTAKES

- Relying on large JSON fixtures that break every time a model changes
- Using `RequestFactory` and expecting `request.user` or `session` to exist
- Comparing `resp.content` byte strings instead of using `assertContains`

✓ TIPS & BEST PRACTICES

- Prefer factories over fixtures; they adapt to model changes
- Use `self.client.get(url, headers={"accept": "application/json"})` for header tests
- Use `freezegun` or `time-machine` to control `timezone.now()`

QUICK SUMMARY

- `self.client`: request through the stack
- `force_login()`: authenticate fast
- `assertRedirects` / `assertFormError`
- `RequestFactory`: call views directly

Pagination

SPLITTING LISTS INTO PAGES

`Paginator` slices any list or QuerySet into pages. `get_page(number)` is forgiving: it returns the first page for junk input and the last page for numbers too high. The `Page` object it returns is iterable and knows its neighbors. `ListView` does all of this for you when you set `paginate_by`, adding `page_obj` and `paginator` to the context.

NOTES

- Paginate an **ordered** QuerySet; otherwise pages can repeat rows
- Each page runs a `COUNT(*)` plus a `LIMIT/OFFSET` query
- `page(n)` raises `EmptyPage` / `PageNotAnInteger`; `get_page(n)` doesn't
- `orphans=3` folds a tiny last page into the previous one
- Use `{% querystring %}` to keep filters in page links

VIEW

```python
from django.core.paginator import Paginator

def post_list(request):
    qs = Post.objects.published().order_by("-created")
    paginator = Paginator(qs, 20, orphans=3)
    page_obj = paginator.get_page(request.GET.get("page"))
    return render(request, "blog/list.html", {"page_obj": page_obj})

page_obj.number                      # 3
paginator.num_pages                  # 12
paginator.count                      # 231 rows
page_obj.start_index()               # 41
paginator.get_elided_page_range(3, on_each_side=2)  # 1 2 3 4 5 … 11 12
```

CODE EXPLAINED

- → `get_page(None)` and `get_page("abc")` return page 1
- → Iterate `page_obj` directly for the rows on that page
- → `get_elided_page_range` adds `…` for long page lists
- → `{% querystring page=... %}` keeps `?q=` and other params
- → In `ListView`, just set `paginate_by = 20`

TEMPLATE

```django
{% for post in page_obj %}<p>{{ post.title }}</p>{% endfor %}
<nav>
  {% if page_obj.has_previous %}
    <a href="{% querystring page=page_obj.previous_page_number %}">‹ Prev</a>
  {% endif %}
  Page {{ page_obj.number }} of {{ page_obj.paginator.num_pages }}
  {% if page_obj.has_next %}
    <a href="{% querystring page=page_obj.next_page_number %}">Next ›</a>
  {% endif %}
</nav>
```

PAGINATOR

MEMBER	GIVES
`count`	Total items
`num_pages`	Total pages
`page_range`	1..num_pages
`get_page(n)`	Safe Page
`page(n)`	Strict Page

PAGE

MEMBER	GIVES
`number`	Current page
`has_next / has_previous`	bool
`next_page_number()`	int
`start_index / end_index`	Item numbers
`object_list`	Rows

LARGE TABLES

APPROACH	WHY
`Keyset / cursor`	No big OFFSET
`DRF CursorPagination`	APIs
`Cached count`	Skip slow COUNT

⚠ COMMON MISTAKES

- Paginating an unordered QuerySet: `UnorderedObjectListWarning`, rows shift between pages
- Building links with `?page=2`, which drops the search filters
- Deep `OFFSET` pages on million-row tables; switch to keyset pagination

✓ TIPS & BEST PRACTICES

- Reuse one `_pagination.html` include across the site
- Use `get_elided_page_range` for compact page lists
- Add `rel="next"` / `rel="prev"` for accessibility and crawlers

QUICK SUMMARY

- `Paginator(qs, n)`: split
- `get_page(request.GET.get("page"))`
- `page_obj.has_next` / `next_page_number`
- `paginate_by`: ListView shortcut

Caching

CACHING AT EVERY LEVEL

Caching stores expensive results so later requests reuse them. Configure a backend in CACHES (Redis and Memcached for production, local memory for development). Then cache at the right granularity: whole views with `@cache_page`, template fragments with `{% cache %}`, or any value with the low-level `cache` API. The per-site cache wraps everything via middleware.

NOTES

- `LocMemCache` is per-process; it isn't shared between workers
- Redis backend is built in (4.0+): `django.core.cache.backends.redis.RedisCache`
- Timeouts are seconds; `None` means forever, `0` means don't cache
- Cache keys vary by URL; use `vary_on_cookie` for per-user pages
- Async API: `await cache.aget()` / `aset()`

CONFIGURATION AND APIS

```
CACHES = {"default": {
    "BACKEND": "django.core.cache.backends.redis.RedisCache",
    "LOCATION": "redis://127.0.0.1:6379/1",
    "TIMEOUT": 300,                                  # default 5 min
}}

from django.core.cache import cache
cache.set("stats", compute_stats(), timeout=600)
cache.get("stats")                                   # value or None
cache.get_or_set("top", lambda: Post.objects.top()[:10], 3600)
cache.add("lock:job", 1, 60)                         # only if missing
cache.incr("hits")                                   # atomic +1
cache.delete("stats"); cache.delete_many(["a", "b"])

from django.views.decorators.cache import cache_page
from django.views.decorators.vary import vary_on_cookie
@cache_page(60 * 15)                                  # whole response
@vary_on_cookie
def dashboard(request): ...
path("about/", cache_page(3600)(TemplateView.as_view(template_name="about.html")))
```

CODE EXPLAINED

- → `get_or_set` computes and stores only on a miss
- → `add` is a simple lock: fails if the key exists
- → `@cache_page` keys on the full URL
- → `vary_on_cookie` separates logged-in users
- → Fragment cache keys include the extra args (`user.pk`)

TEMPLATE FRAGMENT

```
{% load cache %}
{% cache 600 sidebar request.user.pk %} ...expensive sidebar... {% endcache %}
```

BACKENDS

BACKEND	USE
RedisCache	Production
PyMemcacheCache	Production
LocMemCache	Dev, per process
DatabaseCache	No extra service
DummyCache	Disable caching

GRANULARITY

LEVEL	TOOL
Site	Update/FetchFromCache MW
View	@cache_page
Fragment	{% cache %}
Value	cache.get/set
Instance	@cached_property

CACHE API

CALL	DOES
get_many([..])	dict of hits
set_many({..})	Bulk set
touch(k, t)	New timeout
clear()	Wipe everything
caches["other"]	Pick a cache

⚠ COMMON MISTAKES

- `@cache_page` on views that show per-user data without `vary_on_cookie`
- Caching a lazy QuerySet object; cache `list(qs)` or values instead
- Never invalidating; stale data persists until timeout, delete keys on save

✓ TIPS & BEST PRACTICES

- Delete related keys in `post_save` signals or model `save()`
- Prefix keys with a version (`KEY_PREFIX` , `VERSION`) to bust everything on deploy
- Measure first; the slowest thing is often a missing `select_related`

QUICK SUMMARY

- `CACHES` : backend config
- `cache.get_or_set()` : value caching
- `@cache_page(s)` : view caching
- `{% cache s name %}` : fragments

Sending Email

EMAIL FROM DJANGO

`send_mail()` sends a simple message in one call, while `EmailMessage` and `EmailMultiAlternatives` give full control: CC, BCC, reply-to, HTML versions and attachments. Delivery goes through the configured **email backend**: SMTP in production, the console backend in development and the in-memory backend in tests. Django 6.0 moved to Python's modern email API; 6.1 adds the `MAILERS` setting for multiple backends.

NOTES

- Console backend prints emails to the terminal: perfect for dev
- Tests automatically use `locmem`; read `mail.outbox`
- Use keyword arguments; 6.0 deprecates extra positional ones
- Send from a background task so requests don't wait on SMTP
- Transactional services (SES, Postmark) via django-anymail

SETTINGS AND SENDING

```
# settings.py
EMAIL_BACKEND = "django.core.mail.backends.smtp.EmailBackend"
EMAIL_HOST = "smtp.example.com"
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_HOST_USER = os.environ["SMTP_USER"]
EMAIL_HOST_PASSWORD = os.environ["SMTP_PASSWORD"]
DEFAULT_FROM_EMAIL = "Shop <no-reply@example.com>"

from django.core.mail import send_mail, EmailMultiAlternatives
from django.template.loader import render_to_string

send_mail(subject="Welcome!", message="Thanks for joining.",
          from_email=None, recipient_list=["ann@example.com"])

ctx = {"order": order}
msg = EmailMultiAlternatives(
    subject=f"Order #{order.pk}",
    body=render_to_string("emails/order.txt", ctx),
    to=[order.email], reply_to=["support@example.com"])
msg.attach_alternative(render_to_string("emails/order.html", ctx), "text/html")
msg.attach("invoice.pdf", pdf_bytes, "application/pdf")
msg.send()
```

CODE EXPLAINED

- → `from_email=None` uses `DEFAULT_FROM_EMAIL`
- → Always include a plain-text body with the HTML version
- → `attach(name, content, mimetype)` adds a file
- → `send()` returns the number of messages sent
- → Render email bodies from templates, like pages

BACKENDS (…BACKENDS.)

BACKEND	USE
`smtp.EmailBackend`	Production
`console.EmailBackend`	Dev: print
`filebased.EmailBackend`	Dev: files
`locmem.EmailBackend`	Tests
`dummy.EmailBackend`	Discard

FUNCTIONS

FUNCTION	FOR
`send_mail()`	One message
`send_mass_mail()`	Many, one connection
`mail_admins()`	To ADMINS
`EmailMessage`	Full control
`get_connection()`	Reuse SMTP

SETTINGS

SETTING	PURPOSE
`EMAIL_HOST / PORT`	Server
`EMAIL_USE_TLS`	STARTTLS (587)
`EMAIL_USE_SSL`	SSL (465)
`DEFAULT_FROM_EMAIL`	Sender
`MAILERS (6.1)`	Named backends

⚠ COMMON MISTAKES

- Setting both `EMAIL_USE_TLS` and `EMAIL_USE_SSL`; pick the one your port uses
- Sending email inside the request; slow SMTP makes pages hang
- Putting SMTP passwords in `settings.py` instead of the environment

✓ TIPS & BEST PRACTICES

- Send via `django.tasks` or Celery, after commit with `on_commit`
- Use django-anymail to switch providers without code changes
- Preview HTML emails with the file-based backend

QUICK SUMMARY

- `send_mail(subject, message, from, to)`
- `EmailMultiAlternatives`: HTML + text
- `EMAIL_BACKEND`: delivery method
- `mail.outbox`: assert in tests

Background Tasks

THE TASKS FRAMEWORK (DJANGO 6.0)

Django 6.0 adds `django.tasks`, a standard API for defining work to run outside the request: decorate a function with `@task` and call `.enqueue()`. Django defines tasks and queues them through a configured **backend**, but **does not ship a worker**. The built-in backends are for development and testing; production needs a third-party backend and worker, such as the `django-tasks` package's database backend.

NOTES

- Arguments must be JSON-serializable (pass ids, not model objects)
- Built-in: `ImmediateBackend` (runs now) and `DummyBackend` (stores, never runs)
- `priority` ranges from -100 to 100
- `run_after` defers a task (backend must support it)
- Celery and RQ remain options for heavy workloads

DEFINE, CONFIGURE, ENQUEUE

```python
# settings.py
TASKS = {
    "default": {"BACKEND": "django.tasks.backends.immediate.ImmediateBackend"},
}

# orders/tasks.py
from django.core.mail import send_mail
from django.tasks import task

@task(priority=10, queue_name="emails")
def send_receipt(order_id):
    order = Order.objects.get(pk=order_id)
    send_mail(subject=f"Receipt #{order.pk}", message="Thanks!",
              from_email=None, recipient_list=[order.email])
    return order.pk

# in a view
from django.db import transaction
transaction.on_commit(lambda: send_receipt.enqueue(order.pk))

result = send_receipt.enqueue(order.pk)       # TaskResult
result.id                                     # backend-specific id
result.status                                 # TaskResultStatus.READY
result.refresh(); result.return_value         # after it finishes
send_receipt.using(priority=50, run_after=timedelta(minutes=5)).enqueue(7)
```

CODE EXPLAINED

- → `@task` turns the function into a `Task` object
- → `enqueue()` returns a `TaskResult` right away
- → `on_commit` avoids queueing work for a rolled-back order
- → `return_value` raises `ValueError` until the task succeeds
- → `using()` returns a copy with new options

TASKRESULTSTATUS

STATUS	MEANS
`READY`	Queued / retry
`RUNNING`	Executing
`SUCCESSFUL`	Done
`FAILED`	Raised / couldn't start

TASKRESULT

ATTRIBUTE	VALUE
`id / status`	Identity, state
`return_value`	Function result
`errors`	List of TaskError
`attempts`	Runs so far
`enqueued_at / finished_at`	Timestamps

@TASK OPTIONS

OPTION	DEFAULT
`priority`	0
`queue_name`	"default"
`backend`	"default"
`takes_context`	False

⚠ COMMON MISTAKES

- Passing a model instance to `enqueue()`; arguments must survive a JSON round-trip
- Using `ImmediateBackend` in production; work still runs inside the request
- Expecting Django to start workers; you must run the backend's worker process

✓ TIPS & BEST PRACTICES

- Install `django-tasks` for a database backend and `db_worker` command
- Use `takes_context=True` to read `context.attempt` for retry logic
- Use `DummyBackend` in tests and assert on its `results` list

QUICK SUMMARY

- `@task`: define background work
- `.enqueue(*args)` → `TaskResult`
- `TASKS`: backend config
- No built-in worker: add one

Async Django

ASYNC VIEWS AND THE ORM

Views can be `async def` coroutines. Under an **ASGI** server they run on the event loop, which helps when a view waits on several slow I/O calls at once (external APIs, streaming). The ORM offers async versions of query methods (prefix `a`) and `async for` iteration. Sync-only code must be wrapped with `sync_to_async`; calling async code from sync code uses `async_to_sync`.

NOTES

- Under WSGI, async views still work but gain nothing
- Middleware that is sync-only forces a thread switch per request
- Transactions aren't async yet; wrap them in a sync function
- Lazy QuerySet methods (`filter`) have no `a` version; evaluation does
- Use `httpx.AsyncClient` for concurrent outbound HTTP

ASYNC VIEW WITH CONCURRENT CALLS

```
import asyncio, httpx
from asgiref.sync import sync_to_async, async_to_sync
from django.http import JsonResponse

async def dashboard(request):
    async with httpx.AsyncClient(timeout=5) as client:
        weather, news = await asyncio.gather(
            client.get("https://api.example.com/weather"),
            client.get("https://api.example.com/news"))
    user = await request.auser()                    # async user lookup
    post = await Post.objects.filter(author=user).afirst()
    count = await Post.objects.acount()
    titles = [p.title async for p in Post.objects.order_by("-id")[:5]]
    return JsonResponse({"weather": weather.json(), "posts": count,
                         "titles": titles})

@sync_to_async
def charge_card(order_id):                          # sync code w/ atomic
    with transaction.atomic(): ...

async def checkout(request, pk):
    await charge_card(pk)

async_to_sync(notify_channel)("orders", {"id": 7})  # from sync code
```

CODE EXPLAINED

- → `asyncio.gather` runs both requests at the same time
- → `request.auser()` (5.0+) loads the user without blocking
- → `afirst` , `acount` , `aget` , `acreate` are awaitable
- → `async for` iterates a QuerySet asynchronously
- → `sync_to_async` runs blocking code in a thread

RUN UNDER ASGI

```
uvicorn config.asgi:application --workers 4
```

ASYNC ORM METHODS

SYNC	ASYNC
`get()`	aget()
`create()`	acreate()
`count()` / `exists()`	acount() / aexists()
`first()`	afirst()
`save()` / `delete()`	asave() / adelete()
`for x in qs`	async for x in qs

WHEN ASYNC HELPS

CASE	BENEFIT
`Many API calls`	High
`Streaming / SSE`	High
`WebSockets`	Use Channels
`CPU work`	None
`Plain CRUD`	Little

ASGI SERVERS

SERVER	NOTE
`Uvicorn`	Fast, common
`Daphne`	Channels default
`Hypercorn`	HTTP/2, HTTP/3
`Granian`	Rust-based

⚠ COMMON MISTAKES

- Calling sync ORM methods inside `async def` : `SynchronousOnlyOperation`
- Using `requests` in an async view; it blocks the event loop, use `httpx`
- Making every view async under WSGI and expecting a speedup

✓ TIPS & BEST PRACTICES

- Go async only for views that wait on I/O; keep the rest sync
- Use `StreamingHttpResponse` with an async generator for server-sent events
- Use Django Channels for WebSockets

QUICK SUMMARY

- `async def view` : runs on the event loop
- `aget` / `acount` / `async for` : async ORM
- `sync_to_async` : call sync code
- ASGI server: Uvicorn, Daphne

Logging & Debugging

SEEING WHAT YOUR APP DOES

Django uses Python's standard `logging` module, configured through the `LOGGING` setting as a `dictConfig` dict. **Loggers** produce records, **handlers** send them somewhere (console, file, email) and **formatters** shape each line. Django logs request errors to `django.request`. For interactive debugging, use `breakpoint()`, the debug error page and the Django Debug Toolbar.

NOTES

- Keep `disable_existing_loggers: False` so Django's loggers still work
- 5xx errors log to `django.request` at ERROR level
- `django.db.backends` at DEBUG logs every SQL query (needs `DEBUG = True`)
- Get a logger per module: `logging.getLogger(__name__)`
- In containers, log to stdout and let the platform collect it

LOGGING AND USAGE

```
LOGGING = {
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {"plain": {
        "format": "{asctime} {levelname} {name}: {message}", "style": "{"}},
    "handlers": {
        "console": {"class": "logging.StreamHandler", "formatter": "plain"},
        "mail_admins": {"class": "django.utils.log.AdminEmailHandler",
                        "level": "ERROR"},
    },
    "root": {"handlers": ["console"], "level": "INFO"},
    "loggers": {
        "django.request": {"handlers": ["mail_admins"], "level": "ERROR"},
        "django.db.backends": {"level": "WARNING"},   # DEBUG to see SQL
        "shop": {"level": "DEBUG"},
    },
}

import logging
log = logging.getLogger(__name__)                     # e.g. "shop.views"
log.info("Order %s paid", order.pk)                   # lazy formatting
try: charge(order)
except PaymentError:
    log.exception("Charge failed for %s", order.pk)   # adds traceback
breakpoint()                                          # pdb in runserver
```

CODE EXPLAINED

- → `"style": "{"` lets formats use `{name}` placeholders
- → `AdminEmailHandler` emails tracebacks to `ADMINS`
- → Logger names are dotted: `shop` covers `shop.views`
- → Pass args separately so formatting only happens if logged
- → `log.exception` must be called inside `except`

LEVELS

LEVEL	USE
DEBUG	Details for devs
INFO	Normal events
WARNING	Unexpected, OK
ERROR	Failed action
CRITICAL	App broken

DJANGO LOGGERS

LOGGER	COVERS
django	Everything
django.request	4xx/5xx
django.server	runserver
django.db.backends	SQL
django.security.*	Security events

DEBUG TOOLS

TOOL	SHOWS
Debug Toolbar	SQL, templates, time
breakpoint()	Interactive pdb
shell_plus	REPL (extensions)
Sentry	Prod errors

⚠ COMMON MISTAKES

- `disable_existing_loggers: True`, which silences Django's own loggers
- f-strings in log calls: `log.info(f"...")` formats even when the level is off
- Leaving SQL debug logging on in production, flooding logs

✓ TIPS & BEST PRACTICES

- Use JSON logs (python-json-logger) for log aggregators
- Attach a request id in middleware and include it in every line
- Install django-debug-toolbar in development only

QUICK SUMMARY

- `LOGGING`: dictConfig setting
- `getLogger(__name__)`: per-module logger
- `django.request`: server errors
- `log.exception()`: traceback included

Internationalization & Time Zones

LANGUAGES AND TIME ZONES

i18n marks strings for translation in Python (`gettext`) and templates (`{% translate %}`); `makemessages` extracts them into `.po` files, translators fill them in, and `compilemessages` builds `.mo` files Django reads. `LocaleMiddleware` picks the language per request. For time, keep `USE_TZ = True`: datetimes are stored in UTC and converted to the active time zone for display.

NOTES

- Use `gettext_lazy` for strings defined at import time (models, forms)
- `makemessages` needs GNU gettext installed
- Use `timezone.now()`, never `datetime.now()`, when `USE_TZ = True`
- `timezone.activate(tz)` sets the display zone per request
- `i18n_patterns()` adds `/en/`, `/fr/` URL prefixes

PYTHON AND SETTINGS

```python
from django.utils.translation import gettext as _, gettext_lazy, ngettext
from django.utils import timezone
import zoneinfo

class Post(models.Model):
    title = models.CharField(gettext_lazy("title"), max_length=200)

msg = _("Welcome back!")
items = ngettext("%(n)d item", "%(n)d items", n) % {"n": n}

LANGUAGE_CODE = "en"
LANGUAGES = [("en", "English"), ("es", "Español")]
LOCALE_PATHS = [BASE_DIR / "locale"]     # makemessages -l es / compilemessages
MIDDLEWARE += ["django.middleware.locale.LocaleMiddleware"]  # after Session
USE_TZ, TIME_ZONE = True, "UTC"

timezone.now()                                  # aware UTC datetime
timezone.localtime(post.created)                # in the active zone
timezone.activate(zoneinfo.ZoneInfo("America/Chicago"))
```

CODE EXPLAINED

- → `_` is the usual alias for `gettext`
- → `ngettext` picks singular or plural
- → `LocaleMiddleware` goes after `SessionMiddleware`, before `CommonMiddleware`
- → Templates convert to the active zone automatically
- → `{% blocktranslate count %}` handles plurals in templates

TEMPLATE AND COMMANDS

```
{% load i18n tz %}
<h1>{% translate "Latest posts" %}</h1>
{% blocktranslate count n=posts|length %}{{ n }} post{% plural %}
  {{ n }} posts{% endblocktranslate %}
{% timezone "Europe/Paris" %}{{ post.created }}{% endtimezone %}
```

TRANSLATION FUNCTIONS

FUNCTION	USE
`gettext / _`	Runtime strings
`gettext_lazy`	Import-time strings
`ngettext`	Plurals
`pgettext`	With context
`activate("es")`	Switch language

TIMEZONE HELPERS

FUNCTION	RETURNS
`now()`	Aware UTC
`localtime(dt)`	In active zone
`localdate()`	Today, local
`make_aware(dt)`	Naive → aware
`is_aware(dt)`	bool

SETTINGS

SETTING	PURPOSE
`LANGUAGE_CODE`	Default language
`LANGUAGES`	Allowed list
`LOCALE_PATHS`	.po locations
`USE_I18N`	Enable i18n
`USE_TZ / TIME_ZONE`	Time handling

⚠ COMMON MISTAKES

- `gettext` (not lazy) in model fields or forms, so labels freeze in the startup language
- `datetime.now()` with `USE_TZ = True`: naive datetime warnings and wrong times
- Forgetting `compilemessages`; edited `.po` files have no effect

✓ TIPS & BEST PRACTICES

- Store each user's time zone and activate it in middleware
- Use Rosetta or Weblate to let translators edit `.po` files
- Wrap strings for translation from day one, even with one language

QUICK SUMMARY

- `gettext_lazy`: model/form labels
- `{% translate %}`: template strings
- `makemessages` → `.po` → `compilemessages`
- `timezone.now()`: aware UTC

Django REST Framework

BUILDING JSON APIS

Django REST Framework (DRF) is the standard third-party package for APIs. **Serializers** convert model instances to JSON and validate incoming data, much like forms. **Views** and **ViewSets** handle requests; a `ModelViewSet` gives full CRUD in a few lines, and a **router** generates its URLs. Authentication, permissions, pagination and a browsable HTML API are built in.

NOTES

- Install `djangorestframework` and add `"rest_framework"` to `INSTALLED_APPS`
- `ModelSerializer` builds fields from a model like `ModelForm`
- `serializer.is_valid(raise_exception=True)` returns 400 automatically
- Global defaults live in the `REST_FRAMEWORK` setting
- Django Ninja is a popular type-hint-based alternative

SERIALIZER, VIEWSET, ROUTER

```python
from rest_framework import serializers, viewsets, permissions
from rest_framework.routers import DefaultRouter

class PostSerializer(serializers.ModelSerializer):
    author = serializers.ReadOnlyField(source="author.username")
    class Meta:
        model = Post
        fields = ["id", "title", "body", "author", "created"]
        read_only_fields = ["created"]

class PostViewSet(viewsets.ModelViewSet):
    serializer_class = PostSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly]
    def get_queryset(self):
        return Post.objects.select_related("author").order_by("-created")
    def perform_create(self, serializer):
        serializer.save(author=self.request.user)

router = DefaultRouter()
router.register("posts", PostViewSet, basename="post")
urlpatterns = [path("api/", include(router.urls))]
```

CODE EXPLAINED

- → `source="author.username"` reads a nested attribute
- → `ModelViewSet` provides list, create, retrieve, update, destroy
- → `perform_create` sets server-side fields on save
- → The router creates `/api/posts/` and `/api/posts/<pk>/`
- → `IsAuthenticatedOrReadOnly`: anyone reads, users write

ROUTER URLS

REQUEST	ACTION
`GET /posts/`	list
`POST /posts/`	create
`GET /posts/1/`	retrieve
`PUT / PATCH /posts/1/`	update
`DELETE /posts/1/`	destroy

VIEW TYPES

CLASS	USE
`@api_view`	FBV
`APIView`	Custom CBV
`generics.ListCreateAPIView`	Common combos
`ModelViewSet`	Full CRUD
`@action`	Extra endpoints

PERMISSIONS

CLASS	ALLOWS
`AllowAny`	Everyone
`IsAuthenticated`	Logged in
`IsAdminUser`	Staff
`IsAuthenticatedOrReadOnly`	Read for all
`DjangoModelPermissions`	Model perms

⚠ COMMON MISTAKES

- `fields = "__all__"` on serializers, exposing internal fields
- Nested serializers without `select_related` / `prefetch_related`: N+1 queries
- Accepting `author` from the client instead of setting it in `perform_create`

✓ TIPS & BEST PRACTICES

- Add drf-spectacular for an OpenAPI schema and Swagger UI
- Use token or session auth for first-party apps; JWT via simplejwt if needed
- Write object-level permissions with `has_object_permission()`

QUICK SUMMARY

- `ModelSerializer`: model ↔ JSON
- `ModelViewSet`: CRUD endpoints
- `DefaultRouter`: URLs
- `permission_classes`: access rules

Deployment

RUNNING DJANGO IN PRODUCTION

In production Django runs behind an application server: **Gunicorn** for WSGI, or **Uvicorn**/Daphne for ASGI. A reverse proxy (Nginx, Caddy or the platform's load balancer) handles TLS and slow clients. Static files are collected and served by WhiteNoise or a CDN. Configuration comes from **environment variables**, and each deploy runs migrations and `collectstatic`.

NOTES

- Typical worker count: `2 × CPU cores + 1`
- Run `migrate` once per deploy, not per worker
- Health check endpoint: a tiny view returning 200
- PaaS options: Fly.io, Render, Railway, Heroku; or a VPS with Docker
- Use PostgreSQL in production, not SQLite, for multi-user sites

DEPLOY STEPS

```
pip install -r requirements.txt
python manage.py check --deploy
python manage.py collectstatic --noinput
python manage.py migrate --noinput
gunicorn config.wsgi:application --bind 0.0.0.0:8000 --workers 3
# or ASGI:
gunicorn config.asgi:application -k uvicorn_worker.UvicornWorker
```

WHITENOISE + ENV SETTINGS

```
MIDDLEWARE.insert(1, "whitenoise.middleware.WhiteNoiseMiddleware")
STORAGES["staticfiles"] = {"BACKEND":
    "whitenoise.storage.CompressedManifestStaticFilesStorage"}
DATABASES = {"default": dj_database_url.config(conn_max_age=600)}
```

CODE EXPLAINED

- → WhiteNoise goes right after `SecurityMiddleware`
- → `dj-database-url` reads `DATABASE_URL` from the environment
- → `conn_max_age` keeps DB connections open between requests
- → `uvicorn_worker` runs Uvicorn under Gunicorn's process manager
- → Build static files into the image; run migrations at release time

DOCKERFILE

```
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN python manage.py collectstatic --noinput
CMD ["gunicorn", "config.wsgi:application", "--bind", "0.0.0.0:8000"]
```

TYPICAL STACK

LAYER	CHOICE
Proxy / TLS	Nginx, Caddy, LB
App server	Gunicorn / Uvicorn
Static	WhiteNoise / CDN
Database	PostgreSQL
Cache / queue	Redis
Errors	Sentry

ENVIRONMENT VARIABLES

VARIABLE	FEEDS
DJANGO_SECRET_KEY	SECRET_KEY
DATABASE_URL	DATABASES
DJANGO_ALLOWED_HOSTS	ALLOWED_HOSTS
REDIS_URL	CACHES
SENTRY_DSN	Error reporting

RELEASE CHECKLIST

STEP	COMMAND
Checks	`check --deploy`
Static	`collectstatic`
Schema	`migrate`
Restart	Reload workers
Verify	Health check

⚠ COMMON MISTAKES

- Running `runserver` in production; it isn't built for load or security
- Each container running `migrate` on start, causing race conditions
- Forgetting `CSRF_TRUSTED_ORIGINS` behind HTTPS, so every form POST fails

✓ TIPS & BEST PRACTICES

- Keep settings twelve-factor style: everything that varies comes from env
- Set `CONN_MAX_AGE` or use a pooler (PgBouncer, 5.1+ pool option)
- Automate deploys from CI so every release runs the same steps

QUICK SUMMARY

- Gunicorn (WSGI) / Uvicorn (ASGI)
- WhiteNoise or CDN: static files
- Env vars: secrets and config
- `collectstatic` + `migrate` each deploy